MODULE 00 · EXAM STRATEGY - ALL BLOCKS

PCEP-30-02: Certified Entry-Level Python Programmer

A complete exam course: four blocks of teaching, three review modules and four practice papers.

PART Introduction COURSE 6h 27m QUESTIONS 314 worked

admin@sthithapragna.com

Almost every PCEP question is code and a question: **what happens?** This course teaches you to answer it by tracing, accurately, in your head.

The exam

ITEM	DETAIL
Questions and time	30 questions in 40 minutes, plus a 5 minute NDA and tutorial
Pass mark	70%, which is 21 out of 30
Question types	single and multiple select, drag and drop, gap fill, code insertion
Delivery	TestNow: online, with a proctor; no notes of any kind
Validity and cost	5 years; from 69 US dollars; retake after 7 days

Check the version: PCEP-30-03 is coming

- **PCEP-30-02** is the active exam, and the one this course teaches.
- **PCEP-30-03** is listed as in development. Its target was Q3 2026, which has passed without a release.
- Check the version code on your voucher. If 30-03 is live, compare its syllabus with this course.
- The update does not affect the 5-year validity of certificates already earned.

The four blocks, weighted as the exam weights them

PART	EXAM WEIGHT	MODULES	MINUTES
Block 1: Fundamentals	18%	1 to 5	68
Block 2: Control flow	29%	6 to 10	63
Block 3: Data collections	25%	11 to 15	60
Block 4: Functions and exceptions	28%	16 to 21	67
Review	all blocks	22 to 24	32
Practice papers	7 / 8 / 7 / 8 questions	25 to 28	92

Modules 1 to 14

MODULE	TITLE	LENGTH
01	How Python runs your code	13 min
02	Literals, variables and numeral systems	13 min
03	Arithmetic operators and precedence	15 min
04	Comparison, Boolean and bitwise operators	15 min
05	Console input, output and type casting	12 min
06	Making decisions: if, elif and else	12 min
07	while loops	12 min
08	for loops and range()	13 min
09	Nested loops, break, continue and else	13 min
10	Loop patterns the exam loves	13 min
11	Lists I: indexing, slicing and methods	13 min
12	Lists II: references, copies and comprehensions	13 min
13	Tuples	9 min
14	Dictionaries	11 min

Modules 15 to 28

MODULE	TITLE	LENGTH
15	Strings	13 min
16	Functions I: def, return and None	11 min
17	Functions II: passing arguments	12 min
18	Scope and the global keyword	11 min
19	Recursion and generators	11 min
20	The exception hierarchy	8 min
21	Handling exceptions	13 min
22	Spot the error	14 min
23	Confusion pairs	9 min
24	Exam mechanics and question formats	10 min
25	Practice paper 1	23 min
26	Practice paper 2	23 min
27	Practice paper 3	24 min
28	Practice paper 4: final rehearsal	23 min

Original questions, official sources, real Python

- A bank of 481 practice questions was analysed for **topics only**. No question was reused.
- Facts come from docs.python.org, the Python Institute's syllabus and policies, and Python Essentials 1.
- **Every code example was run on Python 3.12 and Python 3.14** before it was written into a slide.
- Where the two versions differ, the slide says so, and gives the exam's answer.

Where Python 3.14 and the exam differ

CHANGE	PYTHON 3.14	ANSWER ON THE EXAM
`except A, B:` without brackets	legal (PEP 758)	use brackets: except (A, B):
`return` inside `finally`	SyntaxWarning (PEP 765)	behaviour is unchanged
several error messages	reworded	answer by the exception's name
soft keywords `match`, `case`, `_`, `type`	legal names	there are still 35 keywords

Where study material can mislead you

- Logical **not** sits below every comparison, not beside unary minus. Module 4.
- Only one except branch ever runs for an exception. Module 21.
- The exam treats a full string slice as a copy; CPython may not. Module 15.
- The certificate lasts 5 years. Some websites wrongly say it never expires.

A study route

01

Learn

Modules 1 to 21 in order. Commit to every worked question before the answer.

02

Review

Modules 22 to 24: errors, look-alikes, and the exam itself.

03

Practise

Papers 1 to 3, without paper. Revisit the block your misses point to.

04

Rehearse

Paper 4 under exam conditions, a few days before you sit.

ONE HABIT

Run the code yourself as you go: Python, IDLE, or the free Edube sandbox. Seeing it run fixes the rule.

Module 1 starts with the idea that decides more PCEP answers than any other: Python checks the whole file before it runs a single line.

MODULE 01 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

How Python runs your code

Interpreters and compilers, the four parts of a language, keywords, lines, indentation and comments.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 01 of 28

admin@sthithapragna.com

Python checks the **whole file** before it runs a single line. Grammar mistakes stop everything. Other mistakes stop only when they are reached.

Why translation is needed at all

01

Instruction list

The complete set of commands a processor knows. Its alphabet.

02

Machine code

Programs written in that alphabet. Fast, and unreadable to people.

03

Source code

A program in a high-level language, saved in a source file.

04

Translation

A compiler or an interpreter turns source into something runnable.

ON THE EXAM

A processor understands only its own instruction list. Everything else has to be translated first.

The four elements of every language

ELEMENT	WHAT IT IS	IN ENGLISH	IN PYTHON
Alphabet	The symbols you may use	The letters A to Z	Characters in a UTF-8 source file
Lexis	The dictionary of valid words	cat, runs, quickly	Keywords, names, numbers, operators
Syntax	Rules for combining words	'Cat the runs' is broken	print("hi" is missing a bracket
Semantics	Whether a valid phrase makes sense	'The sky ate Tuesday'	10 / 0 is well formed, but meaningless

ON THE EXAM

Syntax errors are caught before anything runs. Semantic errors surface only when that line executes.

admin@sthithapragna.com

Compilation versus interpretation

COMPILATION

Translate once, ship the result

- Source becomes a machine code file, once
- Runs fast, without the compiler present
- Every change means compiling again
- One compiler per hardware platform

INTERPRETATION

Translate every time it runs

- Source is translated each time it runs
- Runs as soon as you finish writing
- The same source runs on any platform
- Slower, and users need the interpreter

THE EXAM ANSWER

Python is an interpreted language. Programs in interpreted languages are often called scripts.

What CPython actually does

01

Source

Your .py file, read as UTF-8 text.

›

02

Compile

The whole file is parsed and checked. Syntax errors stop here.

›

03

Bytecode

Compact instructions for a virtual machine, not for the processor.

›

04

Execute

The Python Virtual Machine runs the bytecode.

WHY IT MATTERS

This is why a syntax error on line 50 stops line 1 from running. And bytecode is not machine code.

.pyc files and the __pycache__ folder

- When you **import** a module, CPython saves its bytecode in a **.pyc** file.
- The .pyc files live in a folder called **__pycache__**, beside the module.
- Names record the interpreter, for example **helper.cpython-312.pyc**.
- The script you run directly is **not** cached. Only imported modules are.

> ON THE EXAM
>
> A .pyc file holds compiled bytecode. Not source code, not machine code, and not an interpreter.

Recognition facts older banks love

- **CPython** is the reference implementation of Python, written in C.
- **Guido van Rossum** created Python and named it after Monty Python's Flying Circus.
- Python 3 is **not** backward compatible with Python 2.
- **IDLE** is the Integrated Development and Learning Environment that ships with Python.
- Source files end in **.py**. Python is free to download and use.

Before running versus while running

SYNTAX ERROR

Found before line 1 runs

```
1  print("first")
2  print("second"
```

Output: nothing at all. SyntaxError: '(' was never closed.

RUNTIME ERROR

Found when the line runs

```
1  print("first")
2  print(1 / 0)
3  print("third")
```

Output: first, then ZeroDivisionError. Line 3 never runs.

THE TIEBREAKER

Syntax errors, including indentation errors, stop the whole file. Runtime errors stop at the line that fails.

Python's 35 keywords

```
False     None      True      and       as        assert    async
await     break     class     continue  def       del       elif
else      except    finally   for       from      global    if
import    in        is        lambda    nonlocal  not       or
pass      raise     return    try       while     with      yield
```

1 Only three start with a capital letter: False, None and True.

1-5 None of these can be a variable or function name. That is a SyntaxError.

Soft keywords match, case, _ and type are not on this list. They are still legal names.

Keywords versus built-in names

KEYWORD

Rejected before running

```
1  True = 1
2  class = "3B"
3  in = [1, 2]
```

Each line is a SyntaxError. Nothing runs.

BUILT-IN NAME

Legal, and quietly harmful

```
1  print = 3
2  print("hi")
```

Line 1 runs. Line 2 fails: TypeError, 'int' object is not callable.

ON THE EXAM

print, input, len and list are names, not keywords. You can reassign them, and then they stop working.

One instruction per line, with three exceptions

```
1  x = 1; y = 2
2  total = x + \
3          y
4  colours = ["red",
5             "green"]
6  print(total, colours)
```

1 A semicolon puts two instructions on one line. Legal, but discouraged.

2-3 A backslash at the end of a line continues the instruction.

4-5 Inside any open bracket, a new line continues automatically.

Two instructions side by side with nothing between them, like print(1) print(2), is a SyntaxError.

Indentation is grammar, not decoration

```
1  score = 82
2  if score >= 70:
3      print("Pass")
4      print("Well done")
5  print("Recorded")
```

2 The colon promises that an indented block follows.

3-4 Same indentation, same block. Both run only if the test is true.

5 Back at the left edge, the block has ended. This line always runs.

PEP 8 recommends four spaces per level. Tabs work, but never mix them with spaces.

Four ways to break indentation

WHAT YOU WROTE	WHAT PYTHON RAISES
A line indented for no reason	IndentationError: unexpected indent
A colon with nothing indented after it	IndentationError: expected an indented block
A block containing only a comment	IndentationError: expected an indented block
Tabs and spaces mixed inconsistently	TabError: inconsistent use of tabs and spaces

THE TRAP

Every one of these is a kind of SyntaxError, so nothing runs. A comment is not a statement: use pass.

What counts as a comment, and what does not

```
1  #!/usr/bin/env python3
2  # A comment runs to the end of the line.
3  total = 10  # This one starts mid-line.
4  label = "# Not a comment, just text"
5  """A string on its own. Python evaluates
6  it and throws it away."""
7  print(total, label)
```

1 The #! line tells Unix-like systems how to run the file. To Python, it is a comment.

3 Code before the # runs. Everything after it is ignored.

4 A # inside quotes is just a character in the string.

5-6 Python has no block comment. A triple-quoted string is a string.

Newer than the syllabus

- Python 3.13 rebuilt the interactive prompt: colour, multi-line editing, and **exit** without brackets.
- Since 3.11, error messages mark the exact failing part of a line.
- Soft keywords arrived: **match**, **case** and **_** in 3.10, **type** in 3.12.
- CPython has an experimental JIT compiler from 3.13. It is off by default.

> ON THE EXAM
>
> None of this changes an exam answer. Python is still interpreted, and there are still 35 keywords.

What to carry into the exam

- Python is **interpreted**. CPython compiles to bytecode, then the PVM runs it.
- Grammar errors, indentation included, stop the file **before line 1** runs.
- 35 keywords. Only **False**, **None** and **True** are capitalised.
- print and input are built-in names, **not** keywords.
- A block holding only a comment is an IndentationError. Use pass.

What happens when this program runs?

```
1  print("Start")
2  total = 10
3  print("Total:", total
4  print("End")
```

A Start is printed, then a SyntaxError is reported

B Start and Total: 10 are printed, then the program fails

C Nothing is printed; the program fails with a SyntaxError

D Start, Total: 10 and End are all printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("Start")
2  total = 10
3  print("Total:", total
4  print("End")
```

SyntaxError: '(' was never closed, reported on line 3.

A Start is printed, then a SyntaxError is reported

Pictures Python running line by line before checking. It compiles the whole file first.

B Start and Total: 10 are printed, then the program fails

True of a runtime error. A missing bracket is a grammar error.

C Nothing is printed; the program fails with a SyntaxError

Line 3 never closes its bracket, so compiling fails and line 1 never runs.

D Start, Total: 10 and End are all printed

Python never skips a broken line. One grammar error stops the file.

THE SIGNAL

A grammar error anywhere means no output at all. Scan brackets, colons and quotes before you trace.

Each line is run on its own. Which two run without an error? (Choose two.)

```
1  in_ = 2
2  True = 1
3  Print = 4
4  class = 3
```

A Line 1

B Line 2

C Line 3

D Line 4

◆ PAUSE AND DECIDE

Each line is run on its own. Which two run without an error? (Choose two.)

```
1  in_ = 2
2  True = 1
3  Print = 4
4  class = 3
```

A Line 1

in is a keyword, but in_ is a different name. Underscores are allowed.

B Line 2

True is a keyword. SyntaxError: cannot assign to True.

C Line 3

Names are case-sensitive. Print is neither a keyword nor print.

D Line 4

class is a keyword, so this is a SyntaxError.

THE SIGNAL

One character turns a keyword into an ordinary name. Case and underscores both count.

What happens when this program runs?

```
1  score = 72
2  if score >= 70:
3      # passed
4  print("Result recorded")
```

A Result recorded is printed

B Nothing is printed; an IndentationError is raised

C Nothing is printed, and the program ends normally

D Result recorded is printed, then an IndentationError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  score = 72
2  if score >= 70:
3      # passed
4  print("Result recorded")
```

IndentationError: expected an indented block after 'if' statement on line 2.

A Result recorded is printed

What you would get if line 3 said pass instead of a comment.

B Nothing is printed; an IndentationError is raised

Comments vanish before the grammar check, so the if block is empty.

C Nothing is printed, and the program ends normally

An empty block is never silently accepted. It is a grammar error.

D Result recorded is printed, then an IndentationError

Indentation errors are found before anything runs, not after.

THE SIGNAL

A comment is not a statement. A block that must do nothing needs pass.

> A developer runs main.py, which imports her own module, helper.py. Afterwards, a new folder named __pycache__ has appeared beside helper.py.
>
> Inside it is a single file: helper.cpython-314.pyc.

Which statement about this situation is true?

A The .pyc holds machine code, so helper.py no longer needs an interpreter

B The .pyc holds the source of helper.py, compressed to save space

C main.py is cached there too, because every file that runs is cached

D The .pyc holds compiled bytecode for helper.py, reused by later imports

◆ PAUSE AND DECIDE

Which statement about this situation is true?

A The .pyc holds machine code, so helper.py no longer needs an interpreter

Bytecode targets the Python virtual machine, not the processor. The interpreter is still needed.

B The .pyc holds the source of helper.py, compressed to save space

The source stays in helper.py. The cache holds the result of compiling it.

C main.py is cached there too, because every file that runs is cached

Only imported modules are cached, and the folder holds a single file.

D The .pyc holds compiled bytecode for helper.py, reused by later imports

Importing compiled helper.py to bytecode, cached so the next import skips that step.

THE SIGNAL

A .pyc file contains bytecode. Only imported modules get one, and it never replaces the interpreter.

MODULE 02 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Literals, variables and numeral systems

How a value's spelling decides its type, four ways to write a number, and the rules for naming things.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 02 of 28

admin@sthithapragna.com

A literal is a value written directly in code. **How it is spelled** decides its type, before your program does anything with it.

What the spelling tells Python

LITERAL	TYPE	WHAT DECIDES IT
`42 -7 1_000_000`	int	Digits only, no point, no E
`3.14 .5 4. 4e2`	float	A decimal point, or an E exponent
`"42" 'hi' ""`	str	Quotes, whatever is inside them
`True False`	bool	Exactly these two words, capitalised
`None`	NoneType	The single value meaning no value

THE TRAP

4e2 is a float and "42" is a string. Neither looks like it, and both are favourite exam traps.

type() tells you, and shows it as a class

```
1  print(type(42))       # <class 'int'>
2  print(type(42.0))     # <class 'float'>
3  print(type(4e2))      # <class 'float'>
4  print(type("42"))     # <class 'str'>
5  print(type(True))     # <class 'bool'>
6  print(type(None))     # <class 'NoneType'>
```

3 An E exponent always makes a float, even when the value is whole.

4 Quotes win. The digits inside do not make it a number.

Output is written as <class '...'>. Older material shows <type '...'>, which is Python 2.

E means times ten to the power

```
1  print(3E8)          # 300000000.0
2  print(4.2e-3)       # 0.0042
3  print(+42E7)        # 420000000.0
4  print(0.0001)       # 0.0001
5  print(0.00001)      # 1e-05
6  print(1e16)         # 1e+16
```

1-3 3E8 means 3 times 10 to the 8. Upper or lower case E, always a float.

5-6 Python chooses how to print a float. Very small or very large values come back in E notation.

The exponent must be a whole number: 1E2.5 is a SyntaxError.

Underscores inside numbers

```
1  salary = 1_250_000      # fine: 1250000
2  mask = 0b1111_0000      # fine in any base
3  _1000 = 5               # a variable name, not a number
4  bad = 1__000            # SyntaxError
5  worse = 1000_           # SyntaxError
```

1-2 One underscore between digits is ignored. It only helps the reader.

3 Starting with an underscore makes it a name, not a number.

4-5 Two underscores in a row, or one at the end, is invalid.

The same kind of number, four ways to write it

SYSTEM	BASE	PREFIX	DIGITS ALLOWED	EXAMPLE
Binary	2	0b	0 and 1	0b1010 is 10
Octal	8	0o	0 to 7	0o17 is 15
Decimal	10	none	0 to 9	42 is 42
Hexadecimal	16	0x	0 to 9, A to F	0x1F is 31

ON THE EXAM

The prefix only changes how you write the number. It is still an int, and print shows it in decimal.

Place values: each column is worth base times the next

```
# binary 0b1101:  1*8  + 1*4 + 0*2 + 1*1  = 13
# octal  0o17:    1*8  + 7*1              = 15
# hex    0x1F:    1*16 + 15*1             = 31
print(0b1101, 0o17, 0x1F)      # 13 15 31
```

1 Binary columns, right to left: 1, 2, 4, 8, 16. Add the columns holding a 1.

3 In hex, F is 15. Multiply each digit by its column value and add.

Invalid digits are SyntaxErrors: 0b102, 0o8. And 017 is rejected: use 0o17 for octal.

bin(), oct() and hex() return strings

```
1  print(bin(10))          # 0b1010
2  print(oct(8))           # 0o10
3  print(hex(255))         # 0xff
4  print(type(hex(255)))   # <class 'str'>
5  print(0XFF, 0B11)       # 255 3
```

1-3 Each function gives the text form, prefix included, lower case letters.

4 The result is a string, not a number. You cannot do arithmetic on it.

bool and None

- **True** and **False** are the only bool values. In arithmetic they act as 1 and 0.
- So True + True is 2, and True * 10 is 10.
- **None** is the single value of type NoneType. It means no value.
- None is not 0, not False and not an empty string. None == 0 is False.

> ON THE EXAM
>
> None can be assigned, printed and compared. It cannot be used in arithmetic.

Strings, briefly, before module 15

```
print("Python" == 'Python')   # True
print("Py" "thon")            # Python
print("4" + "2")              # 42
print(4 + 2)                  # 6
```

1. Single and double quotes make exactly the same string.
2. Two string literals side by side are joined into one.
3. Plus on strings joins them. The digits never become numbers.

Rules for a legal name

- Letters, digits and underscores only. Any length.
- **Cannot start with a digit.** 2nd_place is a SyntaxError.
- Cannot be one of the 35 keywords.
- **Case-sensitive:** total, Total and TOTAL are three different names.
- No spaces or hyphens. first-name reads as first minus name.

> ON THE EXAM
>
> Letters include non-English ones: cafe with an accent is a legal name. A leading underscore is legal.

Legal names versus good names

THE RULES

Break them: SyntaxError

- No leading digit
- No keywords
- Only letters, digits, _
- Checked by the interpreter

PEP 8 CONVENTIONS

Break them: still runs

- snake_case for variables
- UPPER_CASE for constants
- CapWords for class names
- Checked only by people and tools

THE TIEBREAKER

If a question asks whether a name is legal, apply only the rules. MyVar is ugly but legal.

A variable is a name stuck on a value

```
1  x = 10          # the name x now refers to 10
2  x = "ten"       # x moves to a string. Types follow values
3  print(x)        # ten
4  y = x           # y refers to the same value as x
5  print(count)    # NameError: count is not defined
```

1-2 Assignment creates the name on first use. The type belongs to the value, not the name.

5 Using a name before assigning it is a NameError, raised when the line runs.

Chained and multiple assignment

```
1  a = b = c = 0          # chained: three names, one value
2  x, y = 1, 2            # multiple: x is 1, y is 2
3  x, y = y, x            # swap in one line
4  print(a, b, c, x, y)   # 0 0 0 2 1
```

1 Chained assignment gives every name the same value.

3 The whole right side is evaluated first, then the names are bound. That is why the swap works.

The counts must match: x, y = 1, 2, 3 raises ValueError: too many values to unpack.

What to carry into the exam

- A point or an E makes a float. Quotes make a str. **4e2 is 400.0.**
- 0b, 0o, 0x change the spelling only. The value is an int, printed in decimal.
- bin(), oct() and hex() return **strings**.
- Names: no leading digit, no keywords, case-sensitive. PEP 8 is style, not law.
- In a, b = x, y the right side is evaluated completely before binding.

Which types do distance, rate and code have?

```
1  distance = 2_500
2  rate = 4E2
3  code = "42"
```

A int, int, str

B int, float, str

C str, float, int

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

Which types do distance, rate and code have?

```
1  distance = 2_500
2  rate = 4E2
3  code = "42"
```

type(rate) is <class 'float'>; its value is 400.0.

A int, int, str

4E2 is whole in value, but an E exponent always makes a float.

B int, float, str

Underscore is fine, E means float, quotes mean str.

C str, float, int

The underscore does not make a string, and quotes never make an int.

D The code raises a SyntaxError

One underscore between digits is legal since Python 3.6.

THE SIGNAL

Read the spelling, not the value. E or a point: float. Quotes: str.

What is the output of this program?

```
1  a = 0b1010
2  b = 0o17
3  c = 0x1F
4  print(a + b + c)
```

A 58

B 46

C 56

D 0b10100o170x1F

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 0b1010
2  b = 0o17
3  c = 0x1F
4  print(a + b + c)
```

10 + 15 + 31 = 56

A **58**

Reads 0o17 as decimal 17. Octal 17 is 8 + 7 = 15.

B **46**

Reads 0x1F's first digit as 10s. In hex, the 1 is worth 16.

C **56**

8 + 2 = 10, then 8 + 7 = 15, then 16 + 15 = 31.

D **0b10100o170x1F**

These are ints, not strings. Plus adds them.

THE SIGNAL

Convert each literal with place values, then do ordinary arithmetic.

Each line is run on its own. Which two are legal assignments? (Choose two.)

```
1  _total = 1
2  2nd_place = 2
3  first-name = 3
4  Class = 4
```

A Line 1

B Line 2

C Line 3

D Line 4

◆ PAUSE AND DECIDE

Each line is run on its own. Which two are legal assignments? (Choose two.)

```
1  _total = 1
2  2nd_place = 2
3  first-name = 3
4  Class = 4
```

A Line 1

A name may start with an underscore.

B Line 2

A name cannot start with a digit. SyntaxError.

C Line 3

The hyphen is a minus sign. You cannot assign to first - name.

D Line 4

class is a keyword, but Class is not. Case matters.

THE SIGNAL

Check each name against three rules: first character, allowed characters, keywords with exact case.

What is the output of this program?

```
1  a = b = 5
2  a, b = b + 1, a * 2
3  print(a, b)
```

A 6 12

B 6 10

C 5 10

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = b = 5
2  a, b = b + 1, a * 2
3  print(a, b)
```

Right side first: (5 + 1, 5 * 2) is (6, 10). Then a and b are bound.

A `6 12`

Updates a first, then uses the new a. Python does not.

B `6 10`

Both expressions use the old values, 5 and 5.

C `5 10`

Treats the first assignment as skipped. Both names change.

D The code raises a SyntaxError

Chained and multiple assignment are both legal.

THE SIGNAL

In multiple assignment, the whole right side is evaluated before any name changes.

What happens when this program runs?

```
1  Total = 10
2  total = Total + 5
3  print(total)
4  print(TOTAL)
```

A 15 is printed twice

B 15 is printed, then 10

C Nothing is printed; the program raises a SyntaxError

D 15 is printed, then the program raises a NameError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  Total = 10
2  total = Total + 5
3  print(total)
4  print(TOTAL)
```

NameError: name 'TOTAL' is not defined. Did you mean: 'Total'?

A 15 is printed twice

Treats total and TOTAL as one name. Names are case-sensitive.

B 15 is printed, then 10

Treats TOTAL as Total. It is a third, separate name.

C Nothing is printed; the program raises a SyntaxError

The grammar is fine. The problem only appears when line 4 runs.

D 15 is printed, then the program raises a NameError

TOTAL was never assigned. Line 3 has already printed 15.

THE SIGNAL

Different case, different name. An unknown name is a runtime NameError, so earlier lines have run.

> A physics program needs the value thirty point one one times ten to the power nine, written as a single float literal.

Which literal writes this value correctly?

A `30E11.9`

B `30.11*10^9`

C `30.11E9.0`

D `30.11E9`

◆ PAUSE AND DECIDE

Which literal writes this value correctly?

A `30E11.9`

The exponent must be a whole number. SyntaxError.

B `30.11*10^9`

Not a literal. ^ is bitwise XOR, so this raises a TypeError on a float.

C `30.11E9.0`

Same problem: a decimal point in the exponent is a SyntaxError.

D `30.11E9`

Mantissa 30.11, exponent 9. Its value is 30110000000.0.

THE SIGNAL

The part after E is always a whole number. And ^ in Python is XOR, never a power.

MODULE 03 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Arithmetic operators and precedence

Seven operators, which ones give floats, what happens with negatives, and the order Python works in.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 03 of 28

admin@sthithapragna.com

Every arithmetic answer turns on three questions: **which operator goes first**, which direction it groups, and whether the result is an int or a float.

Arithmetic operators at a glance

OPERATOR	NAME	7 OP 2	RESULT TYPE WITH TWO INTS
`+ - *`	add, subtract, multiply	9 5 14	int
`/`	true division	3.5	always float
`//`	floor division	3	int
`%`	remainder, or modulo	1	int
`**`	power, or exponent	49	int, unless the exponent is negative

THE TRAP

/ always returns a float, even when the division is exact. 6 / 3 is 2.0.

The rule for the result type

```
1  print(6 / 3)       # 2.0   / always gives a float
2  print(6 // 3)      # 2     int // int gives an int
3  print(4.5 // 2)    # 2.0   one float operand makes a float
4  print(1 + 1.0)     # 2.0
5  print(2 ** -1)     # 0.5   negative exponent gives a float
6  print(True + 2)    # 3     True counts as 1
```

1 Single slash is the one operator that always returns a float.

3-4 For the others: if either operand is a float, the result is a float.

// rounds down, towards minus infinity

```
1  print(7 // 2)      # 3     3.5 rounds down to 3
2  print(-7 // 2)     # -4    -3.5 rounds down to -4
3  print(7 // -2)     # -4
4  print(-7 // -2)    # 3     3.5 rounds down to 3
```

1 Floor means the next whole number to the left on the number line.

2-3 For negatives, left is further from zero. −3.5 floors to −4, not −3.

Division by zero with /, // or % raises ZeroDivisionError.

% takes the sign of the divisor

```
1  print(7 % 3)       # 1
2  print(-7 % 3)      # 2    sign follows the 3
3  print(7 % -3)      # -2   sign follows the -3
4  print(6.5 % 2)     # 0.5  works on floats too
5  print(10 % 2 == 0) # True the classic even-number test
```

2 Check: -7 // 3 is -3, and -3 * 3 + 2 is -7. The remainder fills the gap.

5 n % 2 == 0 is how Python code tests for an even number.

** groups right to left, and beats unary minus

```
1  print(2 ** 3 ** 2)     # 512   means 2 ** (3 ** 2)
2  print((2 ** 3) ** 2)   # 64
3  print(-2 ** 2)         # -4    means -(2 ** 2)
4  print((-2) ** 2)       # 4
5  print(2 ** -2)         # 0.25  minus on the right belongs to 2
6  print(4 ** 0.5)        # 2.0   a fractional power is a root
```

1 Right-sided binding: the rightmost ** is done first.

3 ** is stronger than a minus on its left. The minus is applied last.

Arithmetic precedence, highest first

OPERATORS	GROUPS
`( ) brackets`	innermost first
`**`	right to left
`+x -x unary plus and minus`	right to left
`* / // %`	left to right
`+ - binary add and subtract`	left to right
`= += -= and other assignments`	done last

THE TRAP

* / // and % share one level. Between them, go left to right. Multiplication does not beat division.

When priorities tie, read left to right

```
1  print(12 / 2 * 3)     # 18.0   (12 / 2) * 3, not 12 / 6
2  print(1 // 2 * 3)     # 0      (1 // 2) is 0, times 3 is 0
3  print(8 // 3 % 2)     # 0      (8 // 3) is 2, 2 % 2 is 0
4  print(100 - 10 - 5)   # 85     (100 - 10) - 5
```

1. Twelve divided by two is six, times three is eighteen. And / made it a float.
2. Floor division first gives 0, so the multiplication cannot rescue it.

2 + 3 * 4 ** 2 // 5 - 1, one step at a time

01

Power first

4 ** 2 is 16. Now: 2 + 3 * 16 // 5 - 1

02

* and //, left to right

3 * 16 is 48, then 48 // 5 is 9. Now: 2 + 9 - 1

03

+ and -, left to right

2 + 9 is 11, then 11 - 1 is 10.

04

Result

10, an int. No / anywhere, no float operand.

EXAM TECHNIQUE

Take every step, in order, even when the expression looks easy. Skipping steps is where precedence mistakes happen.

Signs on their own

```
1  x = 1
2  print(+++x)          # 1     three pluses change nothing
3  print(- - -1)        # -1    count the minuses: odd is negative
4  print(2 - -3)        # 5     binary minus, then unary minus
5  print(3 * -2)        # -6
```

2 Unary plus leaves a number unchanged, however many you stack.

3 Each unary minus flips the sign. An odd count is negative.

There is no increment operator. x++ is a SyntaxError, and ++x is just +(+x): x does not change.

Augmented assignment: the right side goes first

```
1  x = 10
2  x += 5          # x = x + 5      -> 15
3  x //= 4         # x = x // 4     -> 3
4  x **= 2         # x = x ** 2     -> 9
5  y = 4
6  y *= 2 + 3      # y = y * (2 + 3) -> 20, not 11
```

2-4 x op= value means x = x op value. Every arithmetic operator has one.

6 The whole right side is worked out first, as if it were in brackets.

+ joins strings, * repeats them

```
1  print("ab" + "cd")       # abcd
2  print("ab" * 3)          # abab
3  print(3 * "ab" + "x")    # abababx   * before +, as usual
4  print("ab" * 0)          # an empty line: ""
5  print("ab" + 3)          # TypeError
```

1-3 Plus joins, times repeats, and precedence works exactly as for numbers.

5 You cannot add a str and an int. Python never converts for you.

Only + and * work on strings. Subtracting or dividing strings is a TypeError. += and *= work too.

Why 0.1 + 0.2 is not 0.3

```
1  print(0.1 + 0.2)                  # 0.30000000000000004
2  print(0.1 + 0.2 == 0.3)           # False
3  print(round(0.1 + 0.2, 2) == 0.3) # True
4  print(1 / 3)                      # 0.3333333333333333
```

1. Floats are stored in binary. Most decimal fractions cannot be stored exactly.
2. So never compare floats for exact equality. Round them, or check they are close.

round() sends exact halves to the even neighbour

```
1  print(round(2.5))        # 2     not 3
2  print(round(3.5))        # 4
3  print(round(7.456, 2))   # 7.46
4  print(round(3.7))        # 4     an int
5  print(round(3.7, 0))     # 4.0   a float, because of the second argument
6  print(round(2.675, 2))   # 2.67  2.675 is stored as slightly less
```

1-2 An exact half rounds to the nearest even number. This avoids bias across many values.

6 Floating-point accuracy again: the stored value is just below the half.

Error messages you may see, and what they mean

- Python 3.12: 5 % 0 says **integer modulo by zero**; 5 // 0 says **integer division or modulo by zero**.
- Python 3.14: all of these now just say **division by zero**.
- The exception is **ZeroDivisionError** in every version. That is what the exam asks about.

> ON THE EXAM
>
> Answer by the exception name, never by the exact message text. Messages change between versions.

What to carry into the exam

- **/ always gives a float.** Otherwise, any float operand gives a float.
- **//** rounds down: -7 // 2 is -4. **%** takes the divisor's sign: -7 % 3 is 2.
- ****** groups right to left, and -2 ** 2 is -4.
- * / // % share a level: left to right. In x *= a + b, a + b goes first.
- 0.1 + 0.2 == 0.3 is False. round(2.5) is 2.

What is the output of this program?

```
1  x = 1 // 5 + 1 / 5
2  print(x)
```

A 0.2

B 0.4

C 0

D 0.0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 1 // 5 + 1 / 5
2  print(x)
```

1 // 5 is 0. 1 / 5 is 0.2. 0 + 0.2 is 0.2

A `0.2`

Floor division gives 0, true division gives 0.2, and 0 + 0.2 is 0.2.

B `0.4`

Treats both divisions as true division.

C `0`

Treats both as floor division. The / makes a float anyway.

D `0.0`

Rounds the 0.2 away. Nothing here discards the fraction of 1 / 5.

THE SIGNAL

Evaluate each division separately, with its own rule, before you add.

What is the output of this program?

```
1  print(2 ** 3 ** 2 ** 1)
```

A 64

B 512.0

C 512

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(2 ** 3 ** 2 ** 1)
```

2 ** 1 is 2. 3 ** 2 is 9. 2 ** 9 is 512.

A 64

*Groups left to right: 8 ** 2 ** 1. Power groups right to left.*

B 512.0

Right value, wrong type. All ints and no negative exponent: an int.

C 512

Right to left: 2 ** 1, then 3 ** 2, then 2 ** 9.

D The code raises a SyntaxError

Chained powers are perfectly legal.

THE SIGNAL

** is the one arithmetic operator that groups right to left. Start at the far right.

What is the output of this program?

```
1  a = -7 // 2
2  b = -7 % 2
3  print(a, b)
```

A -4 1

B -3 -1

C -3 1

D -4 -1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = -7 // 2
2  b = -7 % 2
3  print(a, b)
```

Check: -4 * 2 + 1 is -7.

A `-4 1`

-3.5 floors to -4. The remainder takes the sign of 2.

B `-3 -1`

Chops towards zero, the way C and Java do. Python floors.

C `-3 1`

Right remainder, but -3.5 does not floor to -3.

D `-4 -1`

Right floor, but the remainder follows the divisor, which is positive.

THE SIGNAL

// rounds towards minus infinity, and % takes the sign of the right-hand number.

What is the output of this program?

```
1  x = 4
2  x *= 2 + 3
3  x //= 3
4  print(x)
```

A 3

B 6.666666666666667

C 6.0

D 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 4
2  x *= 2 + 3
3  x //= 3
4  print(x)
```

x = 4 * (2 + 3) = 20, then x = 20 // 3 = 6

A 3

*Reads line 2 as x * 2 + 3, which is 11. The right side goes first.*

B 6.666666666666667

That would be /=. Floor division drops the fraction.

C 6.0

Every value is an int, so // gives an int.

D 6

4 * (2 + 3) is 20, then 20 // 3 is 6.

THE SIGNAL

x op= expression means x = x op (expression). The brackets are invisible, but always there.

What is the output of this program?

```
1  print(-2 ** 2 + 10 // 4 * 2)
```

A 0

B 8

C 5.0

D -8

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(-2 ** 2 + 10 // 4 * 2)
```

-(2 ** 2) is -4. 10 // 4 is 2, times 2 is 4. -4 + 4 is 0.

A `0`

The square happens before the minus: -4. Then 10 // 4 * 2 is 4.

B `8`

*Squares -2 to get 4. But ** beats the minus on its left.*

C `5.0`

*Treats // as / and 4 * 2 as first. Neither is right.*

D `-8`

*Multiplies 4 * 2 before the floor division. They share a level: left to right.*

THE SIGNAL

A minus on the left of ** is applied after the power. A minus on the right belongs to the exponent.

What is the output of this program?

```
1  print(3 * "ab" + "c" * 2)
```

A `abababcababababc`

B `abababcc`

C The code raises a TypeError

D `ab ab ab cc`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(3 * "ab" + "c" * 2)
```

3 * "ab" is "ababab". "c" * 2 is "cc". Joined: "abababcc"

A `abababcabababc`

*Works left to right, ignoring precedence: ((3 * ab) + c) * 2.*

B `abababcc`

Both repetitions happen first, then + joins them.

C The code raises a TypeError

*int * str is legal. Only str + int fails.*

D `ab ab ab cc`

*Repetition adds no separators. That is print's sep, not *.*

THE SIGNAL

Precedence is the same for strings as for numbers: * before +.

What is the output of this program?

```
1  total = 0.1 + 0.2
2  print(total == 0.3, round(total, 1) == 0.3)
```

A True True

B False True

C False False

D True False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  total = 0.1 + 0.2
2  print(total == 0.3, round(total, 1) == 0.3)
```

total is 0.30000000000000004

A `True True`

Assumes floats are exact. 0.1 and 0.2 are stored approximately.

B `False True`

The tiny error breaks ==. Rounding to 1 place removes it.

C `False False`

round(total, 1) is 0.3, the same float as the literal 0.3.

D `True False`

Reverses the effect. Rounding can only help the comparison here.

THE SIGNAL

Never test floats with == directly. Round both sides, or check that they are close.

What is the output of this program?

```
1  x = 7 / 7
2  y = 7 // 7.0
3  z = 7 % 3
4  print(x, y, z)
```

A `1 1 1`

B `1.0 1 1`

C `1.0 1.0 1`

D `1.0 1.0 1.0`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 7 / 7
2  y = 7 // 7.0
3  z = 7 % 3
4  print(x, y, z)
```

A `1 1 1`

Forgets that / always returns a float.

B `1.0 1 1`

Forgets that a float operand makes // return a float.

C **`1.0 1.0 1`**

/ gives a float; 7.0 makes // a float; 7 % 3 is all ints.

D `1.0 1.0 1.0`

Over-applies the float rule. z has no float anywhere.

THE SIGNAL

Decide the type of each result on its own: / always float, others float only if an operand is.

MODULE 04 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Comparison, Boolean and bitwise operators

How Python compares values and strings, what and and or really return, bitwise tricks, and the full precedence table.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 04 of 28

admin@sthithapragna.com

Comparisons always give True or False. **and** and **or** do not: they hand back one of their operands, and stop the moment the answer is known.

Comparisons give a bool

OPERATOR	MEANING	EXAMPLE	RESULT
==	equal to	3 == 3.0	True
!=	not equal to	3 != 3	False
< >	less than, greater than	2 < 10	True
<= >=	less or equal, greater or equal	5 >= 5	True

THE TRAP

= assigns and == compares. There is no <> in Python 3, and =< is a SyntaxError: it must be <=.

Numbers mix. Strings and numbers do not

```
1  print(1 == 1.0)      # True   int and float compare by value
2  print(True == 1)     # True   True is 1 underneath
3  print("1" == 1)      # False  a str never equals an int
4  print(None == 0)     # False
5  print("1" < 1)       # TypeError: '<' not supported
```

1-2 Numbers compare by value, whatever their type. bool counts as a number.

3 Equality between a str and an int is allowed, and always False.

5 Ordering a str against an int is not allowed at all. It raises TypeError.

1 < x < 10 means 1 < x and x < 10

```
1  x = 5
2  print(1 < x < 10)      # True   1 < 5 and 5 < 10
3  print(10 > x > 7)      # False  10 > 5 and 5 > 7
4  print(x == 5 > 4)      # True   x == 5 and 5 > 4
5  print(1 < 3 > 2)       # True   legal, if odd
```

2 A chain is an and of neighbouring pairs. The middle value is shared.

4 == chains too. It does not mean (x == 5) > 4.

Strings compare character by character

```
1  print("apple" < "banana")   # True   a comes before b
2  print("Mike" < "Mikey")     # True   a prefix is smaller
3  print("Zebra" < "apple")    # True   capitals come first
4  print("10" < "9")           # True   '1' comes before '9'
5  print(ord("A"), ord("a"), ord("0"))   # 65 97 48
```

1-2 Compare the first differing character. If one string runs out first, it is smaller.

3-4 The order is by code point, not the alphabet. Digits, then capitals, then lower case.

ASCII, Unicode and code points

- **ASCII** defines 128 characters, codes 0 to 127. Enough for English, not for the world.
- **Unicode** gives every character in every script a number. Its first 128 match ASCII.
- That number is the **code point**. ord() gives it, and chr() turns it back into a character.
- **UTF-8** is an encoding: a way to store code points as bytes. Python source is UTF-8 by default.
- **I18N** is short for internationalisation: 18 letters between the I and the N.

Every value is either truthy or falsy

FALSY: TREATED AS FALSE	TRUTHY: EVERYTHING ELSE, INCLUDING...
`False` `None`	True
`0` `0.0`	-1 0.001
`""` the empty string	" " "0" "False"
`[]` `()` `{}` empty collections	[0] [False] (None,)

THE TRAP

Only empty and zero values are falsy. A string containing a space, a zero or the word False is truthy.

and and or return an operand, not always a bool

```
1  print(not 0)          # True   not always returns a bool
2  print(3 and 5)        # 5      both truthy: the last one
3  print(0 and 5)        # 0      first falsy value
4  print(3 or 5)         # 3      first truthy value
5  print(0 or "x")       # x
6  print("" or 0)        # 0      all falsy: the last one
```

2-3 and returns the first falsy operand, or the last one if all are truthy.

4-6 or returns the first truthy operand, or the last one if all are falsy.

Once the answer is certain, Python stops

```
1  print(0 and 1 / 0)     # 0      and already failed: 1 / 0 never runs
2  print(1 or 1 / 0)      # 1      or already succeeded
3  print(1 and 1 / 0)     # ZeroDivisionError: must check the right side
```

1-2 The right side is skipped entirely, so its error never happens.

3 Here the left side decides nothing, so the right side runs and fails.

Operator precedence, highest first

OPERATORS	NOTES
`**`	right to left
`+x -x ~x`	unary; all three share a level
`* / // %`	
`+ -`	binary
`<< >>`	shifts
`& then ^ then |`	three separate levels, in this order
`== != < <= > >= in is`	all comparisons share one level
`not`	
`and`	
`or`	lowest

Four expressions the table decides

```
1  print(not 1 == 2)                 # True   not (1 == 2)
2  print(True or False and False)    # True   True or (False and False)
3  print(2 & 3 == 2)                 # True   (2 & 3) == 2
4  print(1 + 2 << 1)                 # 6      (1 + 2) << 1
```

1. not is below ==, so the comparison happens first.
2. and binds tighter than or. Evaluate the and, then the or.

Where study material gets the table wrong

- Some banks group **not** with unary plus and minus, near the top. That is wrong.
- The unary operator beside minus is **~**, bitwise not. Logical **not** sits below every comparison.
- Test: not 1 == 2 is True. If not bound tighter, it would be (not 1) == 2, which is False.
- If every option makes this mistake, choose by the order of the other levels.

> REMEMBER IT
>
> Brackets, power, unary, multiply, add, shifts, bitwise, comparisons, not, and, or. Say it once a day.

Operating on individual bits

OPERATOR	NAME	A BIT IN THE RESULT IS 1 WHEN...
&	bitwise and	both bits are 1
|	bitwise or	either bit is 1
^	bitwise xor	the bits differ
~	bitwise not	always: ~x is -x - 1
<< >>	shift left, shift right	x << n is x * 2**n; x >> n is x // 2**n

ON THE EXAM

Bitwise operators need integers. ~1.5 is a TypeError.

6 and 3, bit by bit

```
1  # 6 is 0b110 and 3 is 0b011
2  print(6 & 3)     # 2    0b010  both bits 1
3  print(6 | 3)     # 7    0b111  either bit 1
4  print(6 ^ 3)     # 5    0b101  bits differ
5  print(~6)        # -7   -6 - 1
6  print(6 << 2)    # 24   6 * 4
7  print(6 >> 1)    # 3    6 // 2
```

2-4 Write both numbers in binary, one above the other, and compare column by column.

5 Flip and step back: the answer is the negative, minus one more.

and is not &, and or is not |

LOGICAL

Truthiness, and short-circuits

```
1  print(1 and 2)   # 2
2  print(1 or 2)    # 1
3  print(0 and 1/0) # 0
```

Returns an operand. Skips the right side when it can.

BITWISE

Bits, and always both sides

```
1  print(1 & 2)     # 0
2  print(1 | 2)     # 3
3  print(0 & 1/0)   # ZeroDivisionError
```

Works column by column. Always evaluates both sides.

ON THE EXAM

On True and False, & and | give the same answers as and and or. On other numbers, they usually do not.

== asks the same value. is asks the same object

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)    # True False True
```

1-2 Two separate lists that happen to contain the same values.

3 No copy is made. c is a second sticky note on the same list as a.

Use == to compare values. Use is only to test for None: if x is None.

What to carry into the exam

- **and** returns the first falsy operand, **or** the first truthy one. Both short-circuit.
- Falsy: False, None, 0, 0.0, empty strings and empty collections. " " is truthy.
- Strings compare by code point: digits, capitals, then lower case.
- Order: comparisons, then not, then and, then or. **not** is not a unary arithmetic operator.
- ~x is -x - 1. Shifts multiply or floor divide by powers of two.

What is the output of this program?

```
1  a = 0
2  b = "yes"
3  print(a or b, a and b)
```

A True False

B yes 0

C yes False

D 1 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 0
2  b = "yes"
3  print(a or b, a and b)
```

A `True False`

Assumes and and or return bools. They return an operand.

B `yes 0`

or skips the falsy 0 and returns "yes"; and stops at 0 and returns it.

C `yes False`

Right for or, but and returns the operand 0, not False.

D `1 0`

Converts "yes" to 1. Nothing converts it.

THE SIGNAL

If a question uses and or or outside an if, expect an operand back, not True or False.

What is the output of this program?

```
1  x, y, z = True, False, False
2  print(x or y and z, (x or y) and z)
```

A True True

B False False

C True False

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x, y, z = True, False, False
2  print(x or y and z, (x or y) and z)
```

x or (y and z) is True or False, so True. (x or y) and z is True and False, so False.

A `True True`
Ignores the brackets in the second expression.

B `False False`
Reads the first expression left to right, like the second.

C `True False`
and binds first in the first; the brackets force or first in the second.

D `False True`
Swaps the two results.

THE SIGNAL

and binds tighter than or. Without brackets, group the and first.

What is the output of this program?

```
1  x = 5
2  print(1 < x < 10, 10 > x > 7, x == 5 > 4)
```

A `True False False`

B `True True True`

C The code raises a SyntaxError

D `True False True`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  print(1 < x < 10, 10 > x > 7, x == 5 > 4)
```

x == 5 > 4 means x == 5 and 5 > 4: True and True.

A `True False False`

Reads the third as (x == 5) > 4, that is True > 4. It is a chain.

B `True True True`

5 > 7 is False, so the second chain fails.

C The code raises a SyntaxError

Chaining is legal with any comparison operators, == included.

D `True False True`

Each chain is an and of neighbouring pairs.

THE SIGNAL

a op1 b op2 c always means a op1 b and b op2 c, whatever the operators.

> Each expression is evaluated on its own.

Which two expressions evaluate to True? (Choose two.)

A `'Mike' > 'Mikey'`

B `'mike' > 'Mike'`

C `'10' < '9'`

D `'Zoo' > 'apple'`

◆ PAUSE AND DECIDE

Which two expressions evaluate to True? (Choose two.)

A `'Mike' > 'Mikey'`

Mike runs out first, so it is the smaller string.

B `'mike' > 'Mike'`

m is 109, M is 77. Lower case comes after capitals.

C `'10' < '9'`

Only the first characters matter: '1' is 49, '9' is 57.

D `'Zoo' > 'apple'`

Z is 90, a is 97. Every capital comes before every lower case letter.

THE SIGNAL

Compare code points, not the alphabet: digits, then capitals, then lower case.

What is the output of this program?

```
1  a = 6
2  b = 3
3  print(a & b, a | b, a ^ b)
```

A False True True

B 2 7 7

C 3 7 5

D 2 7 5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 6
2  b = 3
3  print(a & b, a | b, a ^ b)
```

110 & 011 = 010 (2). 110 | 011 = 111 (7). 110 ^ 011 = 101 (5).

A `False True True`

Treats the bitwise operators as logical ones. They return ints.

B `2 7 7`

Treats ^ as or. Exclusive or drops the column where both are 1.

C `3 7 5`

Takes the smaller number for &. It compares bits, not sizes.

D **`2 7 5`**

Column by column: both 1, either 1, bits differ.

THE SIGNAL

Write both numbers in binary, one above the other, and work column by column.

What is the output of this program?

```
1  x = 5
2  print(~x, x << 2, x >> 1)
```

A -6 20 2

B -5 20 2

C -6 10 2.5

D 2 20 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  print(~x, x << 2, x >> 1)
```

~5 is -5 - 1. 5 << 2 is 5 * 4. 5 >> 1 is 5 // 2.

A `-6 20 2`

Flip and step back gives -6. Left 2 is times 4; right 1 floor-halves.

B `-5 20 2`

Treats ~ as a plain minus. It is minus, then one more.

C `-6 10 2.5`

Shifts by one place only, and uses / not //. Shifts stay ints.

D `2 20 2`

Flips only three bits, 101 to 010. Python ints have no fixed width.

THE SIGNAL

~x is -x - 1. x << n is x * 2**n. x >> n is x // 2**n.

What is the output of this program?

```
1 print(bool(0.0), bool(" "), bool([0]), bool("False"))
```

A False False False False

B False True False False

C False True True True

D True True True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1 print(bool(0.0), bool(" "), bool([0]), bool("False"))
```

A `False False False False`

Judges each by what it looks like. Only 0.0 is zero or empty.

B `False True False False`

Looks inside the list and the string. bool only asks if they are empty.

C `False True True True`

0.0 is zero; the rest are non-empty, so truthy.

D `True True True True`

0.0 is a zero, and every zero is falsy.

THE SIGNAL

For strings and collections, ask one question: is it empty? For numbers: is it zero?

What is the output of this program?

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)
```

A True False True

B True True True

C False False True

D True False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)
```

A `True False True`

Equal values, different objects; c is a second name for a's list.

B `True True True`

Treats is like ==. Two list literals make two lists.

C `False False True`

a and b hold the same values, so == is True.

D `True False False`

Assumes c = a copies the list. Assignment never copies.

THE SIGNAL

== compares values. is compares identity. Assignment adds a name; it never copies.

MODULE 05 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Console input, output and type casting

print() with sep and end, why input() always gives text, and exactly which conversions succeed.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 05 of 28

admin@sthithapragna.com

input() **always** returns a string. Nothing the user types becomes a number until your code converts it.

Any number of arguments, any types

```
1  print("A", 1, 2.5, True)     # A 1 2.5 True
2  print()                      # an empty line
3  print("x" + "y", 3 + 4)      # xy 7
```

1. Commas separate arguments. Each is converted to text and joined with one space.
2. With no arguments, print outputs just a newline.
3. Each argument is evaluated first, then printed.

sep goes between arguments; end goes after the last

```
1  print("x", "y", "z", sep="-")      # x-y-z
2  print("x", "y", "z", sep="")       # xyz
3  print("A", end="")
4  print("B", end="!\n")               # AB!
5  print(1, 2, sep="*", end=".\n")     # 1*2.
```

1-2 sep replaces the single space. It goes between arguments, never after the last one.

3-4 end replaces the newline. With end="", the next print continues on the same line.

Three rules, three traps

CODE	WHAT HAPPENS
`print("a", sep="-")`	a (one argument: nothing to separate)
`print(sep="-", "a")`	SyntaxError: positional argument follows keyword argument
`print(print("hi"))`	hi, then None: print returns None

ON THE EXAM

Keyword arguments like sep= and end= must come after all the values to print.

What happens when your program calls input()

01

Pause

The program stops and waits. Nothing below runs yet.

02

Prompt

If you passed a string, it is shown. No newline is added after it.

03

Type and Enter

The user types a line and presses Enter.

04

Return a str

input() returns the typed text as a str, without the newline.

ON THE EXAM

input("Age: ") prints the prompt itself. You do not need a separate print.

The user types 2, then 4

```
1  x = input()
2  y = input()
3  print(x + y)       # 24    joined, not added
4  print(x * 3)       # 222   repeated
5  print(x * y)       # TypeError: can't multiply by a str
```

3 Two strings: + joins them.

4 A string times an int repeats it.

5 A string times a string is not allowed at all.

Convert at the point of input

```
1  age = int(input("Age: "))
2  print("Next year:", age + 1)
```

1. Wrap input() in int() or float() so the variable holds a number from the start.
2. A comma in print mixes text and numbers safely. No str() needed.

If the user types 41, the screen shows: Age: 41, then Next year: 42.

Which conversions work

CALL	RESULT
`int("42") int(" 42 ")`	42. Surrounding spaces and newlines are ignored
`int("4_2") int("-7")`	42 and -7. Underscores and a sign are fine
`int("3.5") int("") int("12a")`	ValueError: not a whole-number string
`float("1e3") float(" 3.5 ")`	1000.0 and 3.5
`float("1,5")`	ValueError: the decimal separator is a point
`int(None)`	TypeError: wrong type, not a bad value

THE TIEBREAKER

A str with the wrong contents raises ValueError. A value of the wrong type raises TypeError.

int() truncates towards zero. // floors

```
print(int(3.9), int(-3.9))      # 3 -3    drop the fraction
print(3.9 // 1, -3.9 // 1)      # 3.0 -4.0  round down
print(round(3.9), round(-3.9))  # 4 -4    nearest
```

1. int() simply throws the fraction away, so negatives move towards zero.
2. Floor division always moves down, so −3.9 becomes −4.

Converting to text and to truth values

```
1  print(str(10) + str(5))      # 105
2  print(str(True), str(0.5))   # True 0.5
3  print(bool("False"))         # True   a non-empty string
4  print(int(True), float(False))   # 1 0.0
5  print(str(bool(1) + 12 / 2)) # 7.0
```

1. str() turns anything into text, so + now joins instead of adding.
3. bool() of a string only asks whether it is empty.
5. Work inside out: True + 6.0 is 7.0, then str gives the text 7.0.

Mixing text and numbers in print

PLUS

Only joins strings

```
1  total = 5
2  print("Total: " + total)
```

TypeError: can only concatenate str (not "int") to str

COMMA OR STR()

Both work

```
1  total = 5
2  print("Total:", total)
3  print("Total: " + str(total))
```

Total: 5, twice. The comma adds the space for you.

ON THE EXAM

Python never converts types silently for +. A comma in print, or an explicit str(), avoids the error.

Older bank favourites: sys.argv and eval()

- **from sys import argv** gives the command-line words as a list of **strings**.
- python index.py 1 2 gives argv == ['index.py', '1', '2']. **argv[0] is the script name.**
- So argv[1] + argv[2] is '12', not 3. The same trap as input().
- **eval(input())** runs the typed text as Python: 3 gives an int, 1, 2 gives a tuple. Never use it on untrusted input.

f-strings: you will see them, the exam will not

- **f"Total: {total}"** puts values straight into text. They arrived in Python 3.6.
- Python 3.12 relaxed the rules: quotes inside the braces can match the outer ones.
- PCEP-30-02 tests print() with commas, sep, end and str(). Answer in those terms.

What to carry into the exam

- **input() returns a str.** "2" + "4" is "24"; "2" * 3 is "222".
- sep goes between arguments only. end replaces the final newline.
- print() returns None. Keyword arguments go after the values.
- Bad contents in a str: **ValueError**. Wrong type: **TypeError**.
- int(-3.9) is -3; -3.9 // 1 is -4.0.

The user types 3 at the first prompt and 2 at the second.

What is the output of this program?

```
1  x = input()
2  y = int(input())
3  print(x * y + x)
```

A 9

B 333

C The code raises a TypeError

D 32

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = input()
2  y = int(input())
3  print(x * y + x)
```

x is "3", y is 2. "3" * 2 is "33", plus "3" is "333".

A 9

Treats x as a number. Only y was converted.

B 333

str * int repeats, then str + str joins.

C The code raises a TypeError

*str * int is legal, and str + str is legal.*

D 32

*Joins x and y. The * happens first, and repeats x.*

THE SIGNAL

Track the type of every variable: only the values wrapped in int() or float() are numbers.

What is the output of this program?

```
1  print("a", "b", sep="", end="-")
2  print("c", "d", sep="+")
```

A ab-c+d

B a b-c + d

C ab-\nc+d

D ab-c+d+

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("a", "b", sep="", end="-")
2  print("c", "d", sep="+")
```

A `ab-c+d`

No gap between a and b, - instead of a newline, + between c and d.

B `a b-c + d`

Keeps the default space. sep replaces it.

C `ab-\nc+d`

end replaces the newline. It does not add to it.

D `ab-c+d+`

sep goes between arguments, never after the last.

THE SIGNAL

sep only between arguments; end only once, at the very end, replacing the newline.

> Each call is evaluated on its own.

Which two calls raise a ValueError? (Choose two.)

A `int(" 7 ")`

B `int("7.0")`

C `float("7e1")`

D `int("seven")`

◆ PAUSE AND DECIDE

Which two calls raise a ValueError? (Choose two.)

A `int(" 7 ")`

Surrounding spaces are stripped. Returns 7.

B `int("7.0")`

int() of a str needs a whole number. A decimal point is not allowed.

C `float("7e1")`

float() accepts E notation. Returns 70.0.

D `int("seven")`

Words are not digits.

THE SIGNAL

int() of a str accepts digits, one sign, underscores and outer spaces. Nothing else.

What is the output of this program?

```
1  x = -7.5
2  print(int(x), x // 1, round(x))
```

A -8 -8.0 -8

B -7 -7.0 -7

C -7 -8 -8

D -7 -8.0 -8

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = -7.5
2  print(int(x), x // 1, round(x))
```

round(-7.5): an exact half goes to the even neighbour, -8.

A `-8 -8.0 -8`
int() does not round down. It drops the fraction: -7.

B `-7 -7.0 -7`
Floor division moves down the number line, to -8.

C `-7 -8 -8`
-7.5 is a float, so // gives a float: -8.0.

D `-7 -8.0 -8`
Truncate, floor as a float, round half to even.

THE SIGNAL

For negatives: int() goes towards zero, // goes down, round() goes to the nearest.

The user types 12 at the first prompt and 3.5 at the second.

What is the output of this program?

```
1  a = input()
2  b = float(input())
3  print(int(a) + int(b), a + str(b))
```

A `15.5 123.5`

B The code raises a ValueError

C `15 123.5`

D `15 15.5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = input()
2  b = float(input())
3  print(int(a) + int(b), a + str(b))
```

int("12") + int(3.5) is 12 + 3. "12" + "3.5" is "123.5".

A `15.5 123.5`
int(b) drops the .5 before the addition.

B The code raises a ValueError
b is already a float, and int() of a float is always fine.

C **`15 123.5`**
12 + 3 is 15; then two strings are joined.

D `15 15.5`
a + str(b) joins two strings. Nothing is added.

THE SIGNAL

int("3.5") would fail. int(3.5) is fine. Check whether the argument is a str or a float.

What is the output of this program?

```
1  x = print("Hi", end=" ")
2  print(x)
```

A Hi None

B Hi

C Hi\nNone

D None Hi

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = print("Hi", end=" ")
2  print(x)
```

A `Hi None`
print returns None; the space from end keeps both on one line.

B `Hi`
x does hold a value: None, which print shows as the word None.

C `Hi\nNone`
end=" " replaced the newline, so no new line starts.

D `None Hi`
Line 1 runs first, so Hi is printed first.

THE SIGNAL

print() shows text and returns None. Assigning its result gives you None, not the text.

MODULE 06 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

Making decisions: if, elif and else

Which branch runs, why order matters, separate ifs versus one chain, and where an else belongs.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 06 of 28

admin@sthithapragna.com

In one if, elif, else chain, **at most one branch runs**: the first whose condition is true. Separate if statements are each checked on their own.

admin@sthithapragna.com

A condition, a colon, an indented block

```
1  temperature = 30
2  if temperature > 25:
3      print("Hot")
4      print("Drink water")
5  print("Done")
```

2 The condition can be any expression. Python tests its truthiness. The colon is required.

3-4 Every line indented under the if runs only when the condition is true.

5 Back at the outer level: this runs either way.

Exactly one of two blocks runs

```
1  age = 15
2  if age >= 18:
3      print("Adult")
4  else:
5      print("Minor")
```

4 else takes no condition, and needs its own colon. It lines up with its if.

2-5 With an else, exactly one of the two blocks always runs.

There is no else if. Python spells it elif. else if x > 3: is a SyntaxError.

Checked top to bottom. The first true branch wins

```
1  score = 85
2  if score >= 90:
3      grade = "A"
4  elif score >= 80:
5      grade = "B"
6  elif score >= 70:
7      grade = "C"
8  else:
9      grade = "F"
10 print(grade)          # B
```

4-5 85 >= 80 is the first true condition. This branch runs, and the chain ends.

6 85 >= 70 is also true, but it is never checked.

8 else is optional. Without it, a chain can run no branch at all.

The widest condition must come last

WRONG ORDER

Everyone gets the first match

```
1  score = 95
2  if score >= 70:
3      grade = "C"
4  elif score >= 90:
5      grade = "A"
6  print(grade)
```

Prints C. 95 >= 70 is true first, so the A branch can never run.

RIGHT ORDER

Most demanding test first

```
1  score = 95
2  if score >= 90:
3      grade = "A"
4  elif score >= 70:
5      grade = "C"
6  print(grade)
```

Prints A. Ranges tested from the top down.

ON THE EXAM

If an earlier condition includes every value a later one would catch, the later branch is dead code.

elif versus separate if statements

ONE CHAIN

At most one branch

```
1  x = 5
2  if x > 0:
3      print("a")
4  elif x > 3:
5      print("b")
```

Prints a. The elif is skipped once a branch has run.

TWO DECISIONS

Each if is checked

```
1  x = 5
2  if x > 0:
3      print("a")
4  if x > 3:
5      print("b")
```

Prints a, then b. Both conditions are true, so both run.

EXAM TECHNIQUE

To count outputs, first mark where each chain starts: every new if starts a new decision.

Any value can be a condition

```
1  name = ""
2  if name:
3      print("Hello", name)
4  else:
5      print("No name given")     # this runs
6  n = 1
7  if n == True:
8      print("equal")             # this runs: 1 == True
```

2 An empty string is falsy, so the else branch runs.

7 True equals 1, so this comparison is true. The same holds for n == 1.

An else belongs to the if it lines up with

ELSE UNDER THE INNER IF

Inner decision has two outcomes

```
x = 7
if x > 5:
    if x > 10:
        print("big")
    else:
        print("medium")
```

Prints medium. The else belongs to if x > 10.

ELSE UNDER THE OUTER IF

Inner decision has no else

```
x = 7
if x > 5:
    if x > 10:
        print("big")
else:
    print("small")
```

Prints nothing. x > 5 is true, x > 10 is false, and the inner if has no else.

THE TIEBREAKER

Read indentation, not intention. An else pairs with the if directly above it at the same level.

pass fills a branch that must be empty

```
1  x = 3
2  if x > 0:
3      pass              # nothing to do here yet
4  else:
5      print("not positive")
```

3 pass is a statement that does nothing. It keeps the block legal.

2-5 Here nothing is printed: the if branch runs, and it does nothing.

A comment alone is not enough: a block with only a comment is an IndentationError.

Conditional code that never starts

CODE	ERROR
`if x = 5:`	SyntaxError: Maybe you meant '==' or ':='?
`if x == 5`	SyntaxError: expected ':'
`else if x > 3:`	SyntaxError: use elif
`elif with no if above it`	SyntaxError: invalid syntax
`If x > 3:`	SyntaxError. Python 3.14 adds: Did you mean 'if'?
`code between an if block and its else`	SyntaxError: the else is orphaned

ON THE EXAM

All of these are syntax errors, so the whole file fails before line 1. Nothing prints.

The user types 101

```
1  room = input()
2  if room == 101:
3      print("found")
4  else:
5      print("not found")      # this runs
```

2 room is the str "101". A str never equals an int, so the test is False.

== between a str and an int is quietly False. <, >, <= and >= between them raise TypeError.

Chained comparisons and logic in conditions

```
1  x = 7
2  late = False
3  if 5 <= x <= 10 and not late:
4      print("in range, on time")
5  if x < 0 or x > 5:
6      print("outside 0 to 5")
7  even = x % 2 == 0          # already a bool: False
```

3 Comparisons first, then not, then and. No brackets needed.

7 A comparison produces True or False itself. No if is needed to build a bool.

Two decision tools you may see, but will not be tested on

- **Conditional expression**: size = "big" if x > 2 else "small". One value chosen by a condition.
- **match and case**, from Python 3.10: structural pattern matching. match and case are soft keywords.
- PCEP-30-02 covers if, if-else, if-elif, if-elif-else, and nesting. Answer in those terms.

What to carry into the exam

- One chain: **first true branch wins**, the rest are skipped, even if also true.
- Every new **if** starts a new decision. Separate ifs can all run.
- Put the most demanding condition first, or later branches become dead code.
- An else pairs with the if directly above it at the same indentation.
- No else if, no = in conditions, no empty blocks: all SyntaxErrors.

How many stars does this program print?

```
1  n = 1
2  if n > 0:
3      print("*")
4  if n == True:
5      print("*")
6  elif n:
7      print("*")
8  else:
9      print("*")
```

A One

B Two

C Three

D Four

◆ PAUSE AND DECIDE

How many stars does this program print?

```
1  n = 1
2  if n > 0:
3      print("*")
4  if n == True:
5      print("*")
6  elif n:
7      print("*")
8  else:
9      print("*")
```

Two decisions. First: 1 > 0. Second chain: 1 == True wins, elif and else skipped.

A. One

Treats line 4 as part of the first chain. A new if starts a new decision.

B. Two

One star from each decision. 1 == True, so the second chain stops there.

C. Three

Runs the elif too. Only the first true branch in a chain runs.

D. Four

Treats every branch as independent, else included.

THE SIGNAL

Split the code into chains first, at every if that is not an elif. Then one branch per chain.

What is the output of this program?

```
1  score = 95
2  if score >= 50:
3      grade = "pass"
4  elif score >= 90:
5      grade = "distinction"
6  else:
7      grade = "fail"
8  print(grade)
```

A `pass`

B `distinction`

C `fail`

D pass, then distinction on the next line

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  score = 95
2  if score >= 50:
3      grade = "pass"
4  elif score >= 90:
5      grade = "distinction"
6  else:
7      grade = "fail"
8  print(grade)
```

95 >= 50 is checked first and is true, so the chain ends there.

A `pass`

The first true condition wins. The distinction branch is dead code.

B `distinction`

What the author meant. Python checks in written order, not best match.

C `fail`

else only runs when every condition above it is false.

D pass, then distinction on the next line

Only one branch of a chain runs, and there is only one print.

THE SIGNAL

Python takes the first true branch, not the best one. Check whether an early test swallows a later one.

What is the output of this program?

```
1  x = 12
2  if x > 5:
3      if x < 10:
4          print("A")
5  else:
6      print("B")
7  print("C")
```

A. B, then C

B. A, then C

C. Only C

D. Nothing is printed

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 12
2  if x > 5:
3      if x < 10:
4          print("A")
5  else:
6      print("B")
7  print("C")
```

12 > 5 is true, so the outer else is skipped. 12 < 10 is false, and the inner if has no else.

A. B, then C
Pairs the else with the inner if. It lines up with the outer one.

B. A, then C
12 < 10 is false, so A is not printed.

C. Only C
Outer test true, inner test false, no inner else. Then C always runs.

D. Nothing is printed
print("C") is outside both ifs, so it always runs.

THE SIGNAL

Before tracing, draw a line from each else up to the if at the same indentation.

The user types 7 and presses Enter.

What happens when this program runs?

```
1  n = input()
2  if n == 7:
3      print("seven")
4  elif n > 5:
5      print("big")
6  else:
7      print("small")
```

A seven is printed

B big is printed

C small is printed

D The code raises a TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  n = input()
2  if n == 7:
3      print("seven")
4  elif n > 5:
5      print("big")
6  else:
7      print("small")
```

n is "7". "7" == 7 is False. "7" > 5 raises TypeError.

A seven is printed

n is the str "7", and a str never equals an int.

B big is printed

Treats n as a number. Comparing it with > is not allowed.

C small is printed

The elif test does not return False. It raises an error.

D The code raises a TypeError

== quietly gives False; > between str and int raises TypeError.

THE SIGNAL

== across types is False. <, >, <= and >= across str and int raise TypeError.

What is the output of this program?

```
1  a, b = 0, "0"
2  if a or not b:
3      print(1)
4  elif not a and b:
5      print(2)
6  else:
7      print(3)
```

A 1

B 2

C 3

D 1 and 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
a, b = 0, "0"
if a or not b:
    print(1)
elif not a and b:
    print(2)
else:
    print(3)
```

a is falsy. b is the non-empty str "0", so it is truthy.

A 1

Treats "0" as falsy. Only the empty string is.

B 2

a or not b is False or False. not a and b is True and "0", which is truthy.

C 3

Would need both conditions false. The elif is truthy.

D 1 and 2

Branches of one chain never both run.

THE SIGNAL

Settle each operand's truthiness first. Then apply not, and, or in that order.

> x has already been set to 5.

Which two lines are syntactically valid? (Choose two.)

A `if x > 3: pass`

B `if x > 3 print(x)`

C `else if x > 3: print(x)`

D `if (x > 3): print(x)`

◆ PAUSE AND DECIDE

Which two lines are syntactically valid? (Choose two.)

A `if x > 3: pass`

A single simple statement may follow the colon on the same line.

B `if x > 3 print(x)`

The colon is missing. SyntaxError: invalid syntax.

C `else if x > 3: print(x)`

There is no else if, and else cannot start a statement alone.

D `if (x > 3): print(x)`

Brackets around a condition are allowed, just unnecessary.

THE SIGNAL

A colon is compulsory; brackets are optional; a one-line body after the colon is legal.

MODULE 07 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

while loops

Counting passes, tracing with a table, loops that never end, break, continue, and the else clause.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 07 of 28

admin@sthithapragna.com

A while loop tests its condition **before every pass**, including the first. If the condition starts false, the body never runs at all.

Test, run the body, go back and test again

```
1  count = 1                # 1. set up
2  while count <= 3:        # 2. test
3      print(count)
4      count += 1           # 3. update
5  print("done")            # after the loop
```

1. Set up the variable the condition depends on.
2. The condition is tested before every pass. The colon and indentation work as for if.
4. Change something the condition depends on, or it can never become false.

Output: 1, 2, 3 on separate lines, then done.

Tracing the loop, one row per test

COUNT	COUNT <= 3	OUTPUT
1	True	1
2	True	2
3	True	3
4	False: loop ends	

EXAM TECHNIQUE

One row for every time the condition is tested. The body ran once for every True: three passes.

How many passes? It depends on the start and the test

START	CONDITION	VALUES SEEN	PASSES
`i = 0`	i < 3	0, 1, 2	3
`i = 0`	i <= 3	0, 1, 2, 3	4
`i = 1`	i < 3	1, 2	2
`i = 1`	i <= 3	1, 2, 3	3

THE SHORTCUT

Counting up by 1: with <, passes = stop − start. With <=, add one.

When the counter does not step by one, trace it

```
1  x = 20
2  while x > 0:
3      print("*")
4      x //= 2            # 20, 10, 5, 2, 1, then 0
5  floor = 1
6  while floor <= 10:
7      floor += floor     # 2, 4, 8, 16: four passes
```

1-4 x takes 20, 10, 5, 2, 1: five stars. Halving 1 gives 0, which ends the loop.

5-7 floor doubles: 1, 2, 4, 8 each pass the test. At 16 the test fails.

Four ways to write an infinite loop by accident

CODE	WHY IT NEVER ENDS
`while i < 5: print(i)`	Nothing in the body changes i.
`while speed < 30: speed *= 2`	speed starts at 0, and 0 doubled is still 0.
`while i != 10: i += 2`	i starts at 1: 1, 3 … 9, 11. It skips 10.
`while i < i + 2: i += 1`	Both sides grow together. Always true.

THE TEST

Ask of every loop: does the body move the variable towards making the test false, and can it land there?

break leaves the loop immediately

```
1  n = 0
2  while True:
3      n += 1
4      if n == 3:
5          break
6      print(n)
7  print("after", n)       # 1, 2, then after 3
```

2 while True never ends by itself. Something inside must break out.

5 break skips the rest of the body, and the rest of the loop, at once.

break and continue are only legal inside a loop. Anywhere else, they are SyntaxErrors.

continue skips to the next test

```
1  x = 0
2  while x < 6:
3      x += 1
4      if x % 2 == 0:
5          continue
6      print(x)              # 1, 3, 5
```

5 continue abandons this pass and jumps straight back to the condition.

3 The update comes before continue, so it always happens.

Update first, or continue can skip it forever

UPDATE AFTER CONTINUE

Never ends

```
1  x = 0
2  while x < 6:
3      if x % 2 == 0:
4          continue
5      x += 1
```

x is 0, which is even. continue skips the update, so x is 0 forever.

UPDATE BEFORE CONTINUE

Ends after six passes

```
1  x = 0
2  while x < 6:
3      x += 1
4      if x % 2 == 0:
5          continue
```

The update always runs, so x reaches 6 and the test fails.

THE TRAP

If continue can fire before the update line, check whether it can fire forever.

else runs when the condition turns false

```
1  i = 1
2  while i <= 3:
3      i += 1
4  else:
5      print("else runs, i is", i)     # i is 4
```

4 The else lines up with while. It runs once, when the test is false.

2-5 A break inside the loop would skip the else completely.

break is the only thing that skips the else

LOOP ENDS NORMALLY

else runs

```
1  n = 0
2  while n < 3:
3      n += 1
4  else:
5      print("no break")
```

The test fails at n == 3, so no break was needed: the else runs.

LOOP ENDS WITH BREAK

else is skipped

```
1  n = 0
2  while n < 3:
3      n += 1
4      if n == 2:
5          break
6  else:
7      print("no break")
```

break fires at n == 2. The else is skipped, and nothing prints.

ON THE EXAM

A loop's else means no break happened. It has nothing to do with whether the body ran.

A loop that never runs still runs its else

```
1  count = 10
2  while count < 5:
3      print("never")
4  else:
5      print("zero passes, else still runs")
```

2 The very first test is false, so the body runs zero times.

4-5 No break happened, so the else runs.

The sentinel pattern: stop on a special value

```
1  total = 0
2  value = int(input())
3  while value != 0:
4      total += value
5      value = int(input())
6  print(total)
```

2 Read once before the loop, so the first test has a value to check.

5 Read again at the end of the body. The special value 0 ends the loop.

If the user types 4, 5, then 0, the program prints 9. The 0 is never added.

while code that never starts

CODE	ERROR
`While True:`	SyntaxError. Python 3.14 adds: Did you mean 'while'?
`while x < 5`	SyntaxError: expected ':'
`break outside a loop`	SyntaxError: 'break' outside loop
`continue outside a loop`	SyntaxError: 'continue' not properly in loop

> ON THE EXAM
>
> Python has no do-while loop. To run a body at least once, use while True with a break.

What to carry into the exam

- The condition is tested **before every pass**. False at the start means zero passes.
- Count passes with a **trace table**: one row per test, passes = number of True rows.
- break leaves the loop; continue jumps back to the test. Update before continue.
- A loop's **else** runs unless a break fired, even after zero passes.
- Infinite: no update, an update that cannot change the value, or one that skips the stop value.

How many stars does this program print?

```
1  x = 20
2  while x > 1:
3      print("*")
4      x //= 3
```

A Two

B Four

C Three

D The loop never ends

◆ PAUSE AND DECIDE

How many stars does this program print?

```
1  x = 20
2  while x > 1:
3      print("*")
4      x //= 3
```

x: 20 (star), 6 (star), 2 (star), 0 (test fails).

A Two

Stops at x == 2, but 2 > 1 is still true.

B Four

Counts the final test as a pass. At 0 the body does not run.

C Three

20, 6 and 2 pass the test. 2 // 3 is 0, which fails it.

D The loop never ends

Floor division by 3 keeps shrinking x until it reaches 0.

THE SIGNAL

Write each value of the variable at the moment of the test. Count only the tests that were True.

What is the output of this program?

```
1  i = 0
2  while i < 5:
3      i += 1
4      if i == 3:
5          continue
6      print(i, end=" ")
```

A `1 2 4 5`

B `1 2`

C `0 1 2 4`

D `1 2 3 4 5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  i = 0
2  while i < 5:
3      i += 1
4      if i == 3:
5          continue
6      print(i, end=" ")
```

i is updated before the print, so the values printed are 1 to 5, minus 3.

A `1 2 4 5`

The update comes first; continue skips only the print for 3.

B `1 2`

Treats continue as break. The loop carries on after 3.

C `0 1 2 4`

Prints before the update. Here the update comes first.

D `1 2 3 4 5`

Ignores the continue. The print is skipped when i is 3.

THE SIGNAL

continue skips the rest of this pass only. break would end the loop.

What is the output of this program?

```
1  n = 10
2  while n > 0:
3      n -= 4
4      if n < 0:
5          break
6  else:
7      print("A")
8  print(n)
```

A. A, then -2

B. 2

C. -2

D. A, then 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 10
2  while n > 0:
3      n -= 4
4      if n < 0:
5          break
6  else:
7      print("A")
8  print(n)
```

n: 10 -> 6 -> 2 -> -2, and -2 < 0 fires the break.

A. A, then -2
The loop ended with break, so the else is skipped.

B. 2
Stops tracing at 2. 2 > 0 is true, so there is another pass.

C. -2
n reaches -2 inside the loop, break fires, the else is skipped.

D. A, then 2
Assumes the test failed at 2. It passed, and break ended the loop.

THE SIGNAL

Ask how the loop ended. Test failed: else runs. break: else skipped.

What is the output of this program?

```
1  i = 1
2  while i != 10:
3      i += 2
4  print(i)
```

A 10

B 11

C 9

D Nothing: the loop never ends

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  i = 1
2  while i != 10:
3      i += 2
4  print(i)
```

i: 1, 3, 5, 7, 9, 11, 13 … always odd, never 10.

A **10**

i is always odd, so it can never equal 10.

B **11**

i passes 10 without stopping; != 10 stays true for 11.

C **9**

9 != 10 is true, so the loop continues past 9.

D **Nothing: the loop never ends**

Starting odd and adding 2 skips 10 for ever.

THE SIGNAL

With != as the test, check the stop value can actually be reached, not just passed.

What is the output of this program?

```
1  count = 5
2  while count < 5:
3      print("loop")
4      count += 1
5  else:
6      print("else")
7  print(count)
```

A else, then 5

B loop, else, then 6

C 5 only

D loop, then 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 5
2  while count < 5:
3      print("loop")
4      count += 1
5  else:
6      print("else")
7  print(count)
```

5 < 5 is False at the very first test: zero passes, no break.

A else, then 5

Zero passes, no break: the else runs, and count is unchanged.

B loop, else, then 6

Runs the body once. The test is checked before the first pass.

C 5 only

Skips the else because the body never ran. That is not the rule.

D loop, then 6

Runs the body, and drops the else. Both are wrong.

THE SIGNAL

A loop's else runs whenever no break fired, including after zero passes.

The user types 3, then 4, then -1, pressing Enter after each.

What is the output of this program?

```
1  total = 0
2  count = 0
3  n = int(input())
4  while n >= 0:
5      total += n
6      count += 1
7      n = int(input())
8  print(total, count)
```

A 6 3

B 7 2

C 7 3

D 0 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  total = 0
2  count = 0
3  n = int(input())
4  while n >= 0:
5      total += n
6      count += 1
7      n = int(input())
8  print(total, count)
```

n = 3: pass. n = 4: pass. n = -1: test fails, never added or counted.

A `6 3`

Adds and counts the -1. The test stops the loop before that.

B `7 2`

Two passes: 3 and 4. The sentinel ends the loop at the test.

C `7 3`

Counts the sentinel as a pass. It only reaches the test.

D `0 0`

The first value, 3, passes the test, so the body runs.

THE SIGNAL

In a sentinel loop, the stop value is read and tested, but never processed.

MODULE 08 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

for loops and range()

Every form of range(), counting passes, looping over strings, lists and dictionaries, and the traps in changing what you loop over.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 08 of 28

admin@sthithapragna.com

A for loop runs once for each item in a sequence. range() builds a sequence of whole numbers that **includes start and stops just before stop**.

admin@sthithapragna.com

for name in sequence:

```
for i in range(3):
    print(i)            # 0, then 1, then 2
print("done")
```

1. Each pass, the next item from the sequence is assigned to i. Then the body runs.
2. No update line and no test to forget: the sequence decides how many passes.

admin@sthithapragna.com

range(stop), range(start, stop), range(start, stop, step)

CALL	VALUES	RULE
`range(5)`	0, 1, 2, 3, 4	One argument is the stop. Start is 0.
`range(2, 5)`	2, 3, 4	Two arguments are start and stop.
`range(0, 10, 3)`	0, 3, 6, 9	The third argument is the step.
`range(5, 0, -1)`	5, 4, 3, 2, 1	A negative step counts down. Stop is still excluded.

THE RULE

The stop value is never included, whichever direction you count.

admin@sthithapragna.com

list() shows the values. print() alone does not

```
1  print(list(range(2, 5)))       # [2, 3, 4]
2  print(list(range(0, 10, 3)))   # [0, 3, 6, 9]
3  print(list(range(-1, 1)))      # [-1, 0]
4  print(list(range(10, 0, -3)))  # [10, 7, 4, 1]
5  print(range(3))                # range(0, 3)
```

3 Negative starts are fine. From -1 up to, but not including, 1.

5 A range is not a list. Printing it shows the range itself, not its values.

admin@sthithapragna.com

When the step points away from stop, nothing happens

```
1  print(list(range(5, 0)))       # []  step is +1, cannot go down
2  print(list(range(0, -2)))      # []
3  print(list(range(2, 2)))       # []  start equals stop
4  print(list(range(0, 5, -1)))   # []  step is -1, cannot go up
```

1-2 Without a step, range only counts up. A lower stop gives an empty range.

3-4 An empty range is not an error. The loop body simply runs zero times.

admin@sthithapragna.com

How many times does the body run?

RANGE	VALUES	PASSES
`range(10)`	0 to 9	10
`range(1, 10)`	1 to 9	9: stop minus start
`range(0, 10, 2)`	0, 2, 4, 6, 8	5
`range(1, 10, 3)`	1, 4, 7	3
`range(10, 0, -3)`	10, 7, 4, 1	4

EXAM TECHNIQUE

With a step of 1, passes are stop minus start. With any other step, write the values out.

The calls that fail

CALL	ERROR
`range(1.5)`	TypeError: 'float' object cannot be interpreted as an integer
`range("5")`	TypeError: 'str' object cannot be interpreted as an integer
`range(0, 10, 0)`	ValueError: range() arg 3 must not be zero
`for i in 5:`	TypeError: 'int' object is not iterable

ON THE EXAM

range() takes whole numbers only. A step of zero is the one ValueError.

admin@sthithapragna.com

A for loop can walk any sequence

```
1  for ch in "Hi!":
2      print(ch, end=" ")        # H i !
3  scores = {"Ann": 7, "Bob": 9}
4  for name in scores:
5      print(name)               # Ann, then Bob: the keys
6  for name, s in scores.items():
7      print(name, s)            # Ann 7, then Bob 9
```

1-2 A string is a sequence of characters, one per pass.

4-5 Looping over a dictionary gives its keys, in insertion order.

6-7 items() gives key and value pairs, unpacked into two names.

admin@sthithapragna.com

for keeps the last value. while goes one further

FOR

Last value used

```
for i in range(3):
    pass
print(i)
```

Prints 2, the last value assigned. range(3) never produced 3.

WHILE

The value that failed the test

```
i = 0
while i < 3:
    i += 1
print(i)
```

Prints 3: the loop only stops once i has reached 3.

ON THE EXAM

If a for loop runs zero times, its variable is never assigned: print(i) would be a NameError.

admin@sthithapragna.com

Changing i inside the body does not steer the loop

```
1  for i in range(3):
2      print(i, end=" ")
3      i = 10                    # overwritten on the next pass
4  print()                       # output so far: 0 1 2
```

3 The next pass assigns the next value from range, whatever i was set to.

The loop variable is a copy of the name, not a window into the list

LOOP OVER THE ITEMS

The list does not change

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n *= 2
4  print(nums)
```

[1, 2, 3]. n is rebound to a new number; the list is untouched.

LOOP OVER THE POSITIONS

The list changes

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= 2
4  print(nums)
```

[2, 4, 6]. nums[i] names a slot in the list itself.

ON THE EXAM

To change a list's elements, loop over range(len(the_list)) and assign through the index.

admin@sthithapragna.com

Removing items as you go skips some

```
1  data = [1, 2, 3, 4]
2  for x in data:
3      data.remove(x)
4  print(data)                  # [2, 4]
5  li = [1, 2, 3]
6  for v in range(len(li)):     # range(3), fixed now
7      li.insert(1, li[v])
8  print(li)                    # [1, 1, 1, 1, 2, 3]
```

2-3 The loop tracks a position. Removing an item shifts the rest left, so the next one is jumped over.

6 range(len(li)) is worked out once, before the first pass. Growing li does not add passes.

admin@sthithapragna.com

else runs when the loop was not broken

```
1  for n in [3, 5, 7]:
2      if n % 2 == 0:
3          print("found even")
4          break
5  else:
6      print("no even numbers")   # this runs
```

4 A break here would skip the else.

5-6 Exactly as with while: no break, so the else runs. Even after zero passes.

admin@sthithapragna.com

Building a result pass by pass

```
1  total = 0
2  for i in range(1, 5):
3      total += i                  # 1 + 2 + 3 + 4
4  print(total)                    # 10
5  word = ""
6  for ch in "abc":
7      word = ch + word            # new character goes in front
8  print(word)                     # cba
```

1-3 Start with an empty result, and add to it on every pass.

7 Order matters: ch + word builds backwards, word + ch builds forwards.

for versus while

FOR

Walk a known sequence

- The items or count are known up front
- No condition and no update to forget
- Cannot run forever over a range
- Loop variable keeps its last value

WHILE

Repeat until something changes

- The number of passes is not known
- You write the test and the update
- Can run forever if you get it wrong
- Ideal for reading input to a sentinel

ON THE EXAM

Both support break, continue and else, with exactly the same rules.

What to carry into the exam

- range includes start and **excludes stop**, in both directions.
- No step means +1. range(5, 0) is empty, not an error.
- After a for loop, the variable holds its **last** value: range(3) leaves 2.
- Reassigning the loop variable does not change the list or the passes.
- range(len(x)) is fixed at the start. Removing items while looping skips some.

How many stars does this program print?

```
1  for i in range(-2, 3, 2):
2      print("*")
```

A Two

B Four

C Three

D Five

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How many stars does this program print?

```
1  for i in range(-2, 3, 2):
2      print("*")
```

Values: -2, 0, 2. The next would be 4, which is past the stop.

A Two

Stops at 0. 2 is still below the stop of 3.

B Four

Includes 4. It is beyond the stop.

C Three

-2, 0 and 2 are all below 3.

D Five

Counts every integer from -2 to 2. The step is 2, not 1.

THE SIGNAL

With a step other than 1, list the values. Stop when the next would reach or pass stop.

What is the output of this program?

```
1  for i in range(10, 0, -3):
2      print(i, end=" ")
```

A 10 7 4

B 10 7 4 1

C 9 6 3 0

D 7 4 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(10, 0, -3):
2      print(i, end=" ")
```

10, 7, 4, 1. The next would be -2, which is past the stop of 0.

A `10 7 4`

Stops early. 1 is still above the stop of 0.

B `10 7 4 1`

Start is included; keep stepping while above 0.

C `9 6 3 0`

Starts at 9 and includes 0. Start is included; stop is not.

D `7 4 1`

Skips the start. The first value is always start itself.

THE SIGNAL

Start is always the first value. Stop is never reached, in either direction.

admin@sthithapragna.com

What is the output of this program?

```
1  total = 0
2  for k in range(1, 6, 2):
3      total += k
4  print(k, total)
```

A 6 9

B 7 9

C 5 9

D 5 15

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the output of this program?

```
1  total = 0
2  for k in range(1, 6, 2):
3      total += k
4  print(k, total)
```

k takes 1, 3, 5. total is 1 + 3 + 5 = 9. k keeps its last value, 5.

A. 6 9

Thinks k ends at the stop. range never produces 6.

B. 7 9

Adds one more step, as a while counter would.

C. 5 9

k keeps the last value it was given; 1 + 3 + 5 is 9.

D. 5 15

Adds every number from 1 to 5. The step is 2.

THE SIGNAL

After a for loop, the variable holds the last value the sequence produced, never the stop.

admin@sthithapragna.com

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n = n * 10
4  print(nums)
```

A `[10, 20, 30]`

B `[1, 2, 3]`

C `[10, 2, 3]`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n = n * 10
4  print(nums)
```

n is moved to a new number each pass. The list's slots are never touched.

A `[10, 20, 30]`
Would need nums[i] = … inside a range(len(nums)) loop.

B `[1, 2, 3]`
Assigning to n rebinds the name only. The list is unchanged.

C `[10, 2, 3]`
No pass changes the list, including the first.

D The code raises a TypeError
Multiplying an int by 10 is fine.

THE SIGNAL

Assigning to the loop variable never changes the list. Assign through an index to do that.

What is the output of this program?

```
for i in range(5, 1):
    print("A")
else:
    print("B")
```

A. A four times, then B

B. Nothing is printed

C. A, then B

D. B

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(5, 1):
2      print("A")
3  else:
4      print("B")
```

range(5, 1) counts up by 1 from 5, and 5 is already past 1: it is empty.

A. A four times, then B
Counts down without being told to. No step means +1.

B. Nothing is printed
The body never runs, but no break fired, so the else does.

C. A, then B
An empty range gives zero passes, not one.

D. B
Empty range: zero passes. No break: the else runs.

THE SIGNAL

Two rules together: no step means counting up, and a loop's else runs whenever no break fired.

What is the output of this program?

```
1  word = ""
2  for ch in "code":
3      word = ch + word
4  print(word)
```

A code

B cccc

C e

D edoc

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the output of this program?

```
1  word = ""
2  for ch in "code":
3      word = ch + word
4  print(word)
```

"c", then "oc", then "doc", then "edoc".

A `code`

That is word + ch. Here each new character goes in front.

B `cccc`

ch takes a different character on each pass.

C `e`

word is built up, not replaced. It keeps the earlier characters.

D **`edoc`**

Each character is placed in front of what came before.

THE SIGNAL

ch + word builds in reverse; word + ch builds in order. Trace the first two passes to be sure.

What is the output of this program?

```
1  prices = {"tea": 2, "cake": 3}
2  total = 0
3  for item in prices:
4      total += prices[item]
5  print(total, item)
```

A 5 tea

B 5 cake

C The code raises a TypeError

D 2 tea

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  prices = {"tea": 2, "cake": 3}
2  total = 0
3  for item in prices:
4      total += prices[item]
5  print(total, item)
```

item takes the keys "tea", then "cake". prices[item] looks up each value.

A `5 tea`

item keeps the last key, not the first.

B `5 cake`

Keys in insertion order; the last one is "cake". 2 + 3 is 5.

C The code raises a TypeError

item is a key, so prices[item] is a valid lookup.

D `2 tea`

The loop makes two passes, not one.

THE SIGNAL

Looping over a dictionary gives its keys, in insertion order. Look values up with d[key].

> Each range is converted to a list on its own.

Which two ranges produce exactly three values? (Choose two.)

A `range(3)`

B `range(1, 3)`

C `range(0, 9, 3)`

D `range(3, 0)`

◆ PAUSE AND DECIDE

Which two ranges produce exactly three values? (Choose two.)

A `range(3)`

0, 1, 2.

B `range(1, 3)`

1, 2: stop minus start is 2.

C `range(0, 9, 3)`

0, 3, 6. The next would be 9, the excluded stop.

D `range(3, 0)`

No step, so it counts up from 3: empty.

THE SIGNAL

For every range question: write the values out, then count them.

admin@sthithapragna.com

MODULE 09 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

Nested loops, break, continue and else

Loops inside loops, counting their passes, and exactly which loop break, continue and else affect.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 09 of 28

admin@sthithapragna.com

The inner loop runs **all the way through** on every pass of the outer loop. break, continue and else only ever affect the **innermost** loop they belong to.

The inner loop restarts on every outer pass

```
1  for i in range(1, 3):
2      for j in range(1, 4):
3          print(i, j, end="  ")
4      print()
```

2-3 For each value of i, j runs through 1, 2 and 3 from the beginning.

4 This print is inside the outer loop only: one new line per value of i.

Output: 1 1 1 2 1 3, then on the next line 2 1 2 2 2 3.

The inner loop is the minute hand

OUTER PASS	INNER PASSES	PRINTED
`i = 1`	j = 1, 2, 3	1 1 1 2 1 3
`i = 2`	j = 1, 2, 3	2 1 2 2 2 3
`finished`		

THE PICTURE

The inner variable changes fastest. The outer variable only moves when the inner loop has finished.

How many times does the inner body run?

OUTER	INNER	INNER BODY RUNS
`range(3)`	range(4)	3 x 4 = 12
`range(4)`	range(i)	0 + 1 + 2 + 3 = 6
`range(1, 4)`	range(i)	1 + 2 + 3 = 6
`range(3)`	range(i, 3)	3 + 2 + 1 = 6

EXAM TECHNIQUE

Fixed inner range: multiply. Inner range that uses the outer variable: add up each outer pass.

When the inner range depends on the outer variable

```
1  for row in range(1, 4):
2      for col in range(row):
3          print("*", end="")
4      print()
```

2 Row 1 gets one star, row 2 gets two, row 3 gets three.

3-4 end="" keeps the stars on one line; the outer print() ends the row.

Output: *, then **, then ***, on three lines. print("*" * row) would do the same job.

break ends the inner loop only

```
1  for i in range(3):
2      for j in range(3):
3          if j == 1:
4              break
5          print(i, j)
```

4 break ends the j loop. The i loop carries on with its next value.

Output: 0 0, then 1 0, then 2 0. The outer loop still makes all three passes.

continue skips to the next pass of the inner loop

```
1  for i in range(2):
2      for j in range(3):
3          if j == 1:
4              continue
5          print(i, j, end="  ")
```

4 continue skips the rest of this inner pass. j moves on to 2.

Output: 0 0 0 2 1 0 1 2. Only the j == 1 pairs are missing.

In the inner loop, or in the outer loop

BREAK INSIDE THE INNER LOOP

Ends one inner loop

```
1  for i in range(3):
2      for j in range(2):
3          if i == 1:
4              break
5          print(i, j, end="  ")
```

0 0 0 1 2 0 2 1. When i is 1, the inner loop is cut short, then i moves on to 2.

BREAK IN THE OUTER LOOP

Ends everything

```
1  for i in range(3):
2      for j in range(2):
3          print(i, j, end="  ")
4      if i == 1:
5          break
```

0 0 0 1 1 0 1 1. After the inner loop finishes for i == 1, the outer loop ends.

THE TIEBREAKER

Look at the indentation of the break. It ends whichever loop it is directly inside.

Python has no break 2. Use a flag

```
found = False
for i in range(3):
    for j in range(3):
        if i * j == 2:
            found = True
            break
    if found:
        break
print(i, j)                 # 1 2
```

5-6 Record the discovery, then leave the inner loop.

7-8 Back in the outer loop, check the flag and leave that loop too.

The inner else can run on every outer pass

```
1  for i in range(3):
2      for j in range(2):
3          if i == 1:
4              break
5      else:
6          print("no break at", i)
```

5 This else lines up with the inner for, so it belongs to the inner loop.

5-6 It runs each time the inner loop ends without break: for i = 0 and i = 2.

What each keyword affects

KEYWORD	EFFECT	WHICH LOOP
`break`	Ends the loop at once; its else is skipped	The innermost loop containing it
`continue`	Skips to the next pass	The innermost loop containing it
`else`	Runs if the loop ended without break	The loop it lines up with
`pass`	Does nothing	Not a loop keyword at all

ON THE EXAM

None of these reaches past the innermost loop. To leave more than one, use a flag.

Reusing the loop variable name

```
1  for i in range(2):
2      for i in range(3):
3          print(i, end=" ")
4  print("|", i)
```

2 The inner loop overwrites i, but the outer loop still gets its next value from its own range.

Output: 0 1 2 0 1 2 | 2. The outer loop still makes two passes.

A while loop inside a for loop

```
1  for n in [3, 1]:
2      while n > 0:
3          print(n, end=" ")
4          n -= 1
5  print()                          # output: 3 2 1 1
```

1 The for loop hands the while loop a starting value each time.

2-4 The while loop counts that value down to zero, then the for loop moves on.

What to carry into the exam

- The inner loop runs **completely** on every outer pass.
- Fixed inner range: multiply the counts. Inner range using the outer variable: add them.
- break and continue act on the **innermost** loop only. Use a flag to leave both.
- An else belongs to the loop it lines up with; an inner else can run on every outer pass.
- Reusing a loop variable name does not change how many passes the outer loop makes.

How many stars does this program print?

```
for i in range(3):
    for j in range(i, 4):
        print("*")
```

A 9

B 12

C 6

D 7

◆ PAUSE AND DECIDE

How many stars does this program print?

```
for i in range(3):
    for j in range(i, 4):
        print("*")
```

i = 0: 0 to 3, four stars. i = 1: three. i = 2: two. 4 + 3 + 2 = 9.

A 9

The inner range shrinks: 4 + 3 + 2.

B 12

Multiplies 3 by 4. The inner range depends on i, so add instead.

C 6

Uses range(i, 3). The inner stop is 4.

D 7

Starts the inner loop at i + 1. range(i, 4) includes i.

THE SIGNAL

If the inner range uses the outer variable, count each outer pass separately and add.

What is the output of this program?

```
for i in range(1, 4):
    for j in range(1, 4):
        if j > i:
            break
        print(j, end="")
```

A 123123123

B 1

C 111222333

D 112123

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(1, 4):
    for j in range(1, 4):
        if j > i:
            break
        print(j, end="")
```

i = 1 prints 1. i = 2 prints 12. i = 3 prints 123.

A 123123123

Ignores the break. It cuts each row short.

B 1

Thinks break ends both loops. The outer loop carries on.

C 111222333

Prints i, not j.

D **112123**

Each row prints j up to i, then break ends that row only.

THE SIGNAL

A break in the inner loop shortens that pass of the inner loop. The outer loop is unaffected.

What is the output of this program?

```
1  count = 0
2  for i in range(3):
3      for j in range(3):
4          if i == j:
5              continue
6          count += 1
7  print(count)
```

A 6

B 3

C 9

D 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 0
2  for i in range(3):
3      for j in range(3):
4          if i == j:
5              continue
6          count += 1
7  print(count)
```

9 pairs in total. 3 have i == j: (0,0), (1,1), (2,2). 9 - 3 = 6.

A 6

All 9 pairs, minus the 3 where i equals j.

B 3

Counts the skipped pairs instead of the counted ones.

C 9

Ignores the continue.

D 0

Treats continue as ending the loops. It skips one pass only.

THE SIGNAL

Count the total passes, then subtract the ones continue skips.

What is the output of this program?

```
for i in range(2):
    for j in range(2):
        if j == 2 * i:
            break
    else:
        print("X", end="")
    print(i, end="")
```

A X0X1

B 0X1

C 01

D X01

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(2):
    for j in range(2):
        if j == 2 * i:
            break
    else:
        print("X", end="")
    print(i, end="")
```

i = 0: j == 0 breaks at once, no X, print 0. i = 1: j never equals 2, so X, then print 1.

A. X0X1

Runs the else on both passes. For i = 0 the inner loop breaks.

B. 0X1

Break on the first pass skips the else; no break on the second.

C. 01

Never runs the else. For i = 1 the inner loop is not broken.

D. X01

Runs the else on the wrong pass.

THE SIGNAL

An inner loop's else is decided afresh on every outer pass: was this inner loop broken?

What is the output of this program?

```
found = False
for i in range(1, 4):
    for j in range(1, 4):
        if i + j == 4:
            found = True
            break
    if found:
        break
print(i, j)
```

A 3 1

B 2 2

C 1 3

D 3 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
found = False
for i in range(1, 4):
    for j in range(1, 4):
        if i + j == 4:
            found = True
            break
    if found:
        break
print(i, j)
```

A 3 1

The last pair that sums to 4. The flag stops at the first.

B 2 2

Also sums to 4, but i = 1 comes first.

C 1 3

The first pair found, in trace order. The flag ends both loops.

D 3 3

Assumes both loops ran to the end. The flag stopped them.

i = 1: j = 1, 2, 3. 1 + 3 == 4 at j = 3. Both loops end there.

THE SIGNAL

With a flag, both loops stop at the first match in trace order: outer value first, then inner.

How many stars does this program print?

```
for i in range(4):
    if i % 2 == 1:
        continue
    for j in range(i):
        print("*", end="")
```

A Four

B Six

C None

D Two

◆ PAUSE AND DECIDE

How many stars does this program print?

```
for i in range(4):
    if i % 2 == 1:
        continue
    for j in range(i):
        print("*", end="")
```

i = 0: range(0), no stars. i = 1: skipped. i = 2: two stars. i = 3: skipped.

A **Four**
Counts i = 3 as well. Odd values are skipped.

B **Six**
Ignores the continue: 0 + 1 + 2 + 3.

C **None**
i = 2 is even, so its inner loop runs twice.

D **Two**
Only even i reach the inner loop: 0 stars, then 2.

THE SIGNAL

A continue in the outer loop, before the inner loop, skips the whole inner loop for that pass.

What is the output of this program?

```
for i in range(2):
    for i in range(3):
        pass
    print(i, end=" ")
```

A 0 1

B 2 2

C 2

D 1 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(2):
    for i in range(3):
        pass
    print(i, end=" ")
```

The inner loop always leaves i at 2. The outer loop still makes two passes.

A `0 1`

Forgets that the inner loop overwrites i before the print.

B `2 2`

Two outer passes; each print follows an inner loop that ends at 2.

C `2`

Thinks the overwritten i ends the outer loop. It takes its next value from range(2).

D `1 2`

Mixes the outer and inner values.

THE SIGNAL

A for loop takes its next value from its own sequence, never from the current value of its variable.

> A break statement sits inside the inner loop of two nested loops. Both loops have an else clause.

Which two statements are true when the break fires? (Choose two.)

A. Both the inner and the outer loop end immediately

B. The else clauses of both loops are skipped

C. Only the innermost loop containing the break ends

D. The else clause of the loop that was broken is skipped

◆ PAUSE AND DECIDE

Which two statements are true when the break fires? (Choose two.)

A Both the inner and the outer loop end immediately

break reaches only the innermost loop. Use a flag to leave both.

B The else clauses of both loops are skipped

The outer loop was not broken, so its else can still run.

C Only the innermost loop containing the break ends

The outer loop carries on with its next pass.

D The else clause of the loop that was broken is skipped

That is exactly what else means: no break happened.

THE SIGNAL

break touches exactly one loop, and exactly one else: the innermost loop and its own else.

MODULE 10 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

Loop patterns the exam loves

Accumulating, counting, finding the largest, searching, working with digits, and bubble sort.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 10 of 28

admin@sthithapragna.com

Most loop questions are one of six patterns. **Check the starting value first**: it is where the traps are hidden.

Recognise the pattern, then check its start

PATTERN	STARTS WITH	EACH PASS
Accumulate	total = 0, or product = 1	total += n product *= n
Count	count = 0	if condition: count += 1
Largest or smallest	best = data[0]	if n > best: best = n
Search	found = False	if match: found = True; break
Digits	a whole number n	n % 10 is the last digit; n //= 10 drops it
Bubble sort	an unsorted list	swap neighbours that are out of order

ON THE EXAM

The starting value is the most common trap in every one of these patterns.

A sum starts at 0. A product starts at 1

```
1  nums = [2, 3, 4]
2  total = 0
3  product = 1
4  for n in nums:
5      total += n
6      product *= n
7  print(total, product)       # 9 24
```

2 0 is the neutral value for adding: 0 + n is n.

3 1 is the neutral value for multiplying. Start at 0 and the product is 0 for ever.

Add one only when the condition holds

```
1  words = ["apple", "kiwi", "banana", "fig"]
2  count = 0
3  for w in words:
4      if len(w) > 3:
5          count += 1
6  print(count)                # 3
```

4-5 count goes up by one, not by the value. apple, kiwi and banana qualify; fig does not.

Start with the first element, not with 0

STARTS AT 0

Wrong for all-negative data

```
1  data = [-5, -2, -9]
2  largest = 0
3  for v in data:
4      if v > largest:
5          largest = v
6  print(largest)
```

Prints 0. No value beats 0, so the starting value is never replaced.

STARTS AT DATA[0]

Always a real value

```
1  data = [-5, -2, -9]
2  largest = data[0]
3  for v in data:
4      if v > largest:
5          largest = v
6  print(largest)
```

Prints -2. The best so far is always a value from the list.

THE TRAP

If the answer is 0 and 0 is not in the data, suspect the starting value.

The same pattern with nested loops

```
1  grid = [[4, 9], [7, 2], [5, 8]]
2  smallest = grid[0][0]
3  for row in grid:
4      for value in row:
5          if value < smallest:
6              smallest = value
7  print(smallest)              # 2
```

2 grid[0][0] is the first number of the first row: 4.

3-6 The outer loop takes each row, the inner loop each number in it.

Python also has min(), max() and sum(). The exam uses both the loops and the built-ins.

Two ways to report whether something was found

A FLAG

Remember the result

```
found = False
for n in [4, 7, 1]:
    if n == 7:
        found = True
        break
print(found)
```

Prints True. The flag keeps the answer for later code.

FOR WITH ELSE

Report straight away

```
for n in [4, 7, 1]:
    if n == 7:
        print("found")
        break
else:
    print("not found")
```

Prints found. The else would run only if no break fired.

ON THE EXAM

Both patterns break on the first match, so later matches are never seen.

Testing for a prime with for and else

```
1  n = 13
2  for d in range(2, n):
3      if n % d == 0:
4          print(n, "is not prime")
5          break
6  else:
7      print(n, "is prime")        # 13 is prime
```

2-5 Try every possible divisor from 2 up to n - 1. Any exact division means not prime.

6-7 No break means no divisor was found.

Trap: for n = 1, range(2, 1) is empty, so the else runs and 1 is reported as prime.

% 10 takes the last digit; // 10 removes it

```
1  n = 472
2  total = 0
3  while n > 0:
4      total += n % 10          # 2, then 7, then 4
5      n //= 10                 # 47, then 4, then 0
6  print(total)                 # 13
```

4 472 % 10 is 2: the remainder after dividing by 10 is the last digit.

5 472 // 10 is 47: floor division by 10 chops the last digit off.

Build the reverse one digit at a time

```
1  n = 472
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10    # 2, then 27, then 274
5      n //= 10
6  print(rev)                     # 274
```

4 Times 10 shifts the digits built so far one place left, making room for the new last digit.

Zeros on the end disappear: 120 reverses to 21, because the reverse would start with 0.

Bubble sort, from Python Essentials 1

```
1  data = [8, 10, 6, 2, 4]
2  swapped = True
3  while swapped:
4      swapped = False
5      for i in range(len(data) - 1):
6          if data[i] > data[i + 1]:
7              data[i], data[i + 1] = data[i + 1], data[i]
8              swapped = True
9  print(data)                    # [2, 4, 6, 8, 10]
```

5 len(data) - 1, because each step compares i with i + 1. Going one further would be an IndexError.

7 The swap from module two: both right-hand values are read before either slot changes.

3-4 Repeat whole passes until one pass makes no swaps at all.

Each pass carries the largest remaining value to the end

AFTER PASS	LIST	SWAPS MADE?
1	[8, 6, 2, 4, 10]	yes: 10 has bubbled to the end
2	[6, 2, 4, 8, 10]	yes: 8 is in place
3	[2, 4, 6, 8, 10]	yes: sorted, but the loop does not know yet
4	[2, 4, 6, 8, 10]	no: the loop ends

ON THE EXAM

The last pass changes nothing. It is needed to prove the list is sorted.

Functions that replace these loops

- **sum(nums)**, **min(nums)**, **max(nums)** and **len(nums)** do the accumulating and searching.
- **sorted(nums)** returns a new sorted list; the original is unchanged.
- **nums.sort()** sorts in place and returns **None**. print(nums.sort()) prints None.
- The exam uses the loops to test tracing, and the built-ins to test what they return.

What to carry into the exam

- Sums start at 0; **products start at 1**.
- Largest and smallest start at **the first element**, never 0.
- A search with break stops at the **first** match. for-else reports no match.
- n % 10 is the last digit; n // 10 removes it. 120 reversed is 21.
- Bubble sort compares neighbours, uses range(len(data) - 1), and stops after a pass with no swaps.

What is the output of this program?

```
1  nums = [3, 1, 4]
2  result = 0
3  for n in nums:
4      result = result * 10 + n
5  print(result)
```

A 314

B 8

C 413

D 3140

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [3, 1, 4]
2  result = 0
3  for n in nums:
4      result = result * 10 + n
5  print(result)
```

0 * 10 + 3 = 3. 3 * 10 + 1 = 31. 31 * 10 + 4 = 314.

A 314

Each pass shifts left one place and adds the next digit.

B 8

That is the plain sum. The times 10 changes it.

C 413

The reversing pattern. Here digits are added in list order.

D 3140

One shift too many. The last digit is added, not shifted.

THE SIGNAL

result * 10 + digit builds a number left to right, in the order the digits arrive.

What is the output of this program?

```
1  temps = [-4, -9, -2, -7]
2  highest = 0
3  for t in temps:
4      if t > highest:
5          highest = t
6  print(highest)
```

A -2

B -9

C 0

D -4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  temps = [-4, -9, -2, -7]
2  highest = 0
3  for t in temps:
4      if t > highest:
5          highest = t
6  print(highest)
```

No temperature is greater than 0, so highest is never replaced.

A `-2`

The right answer to the question the code was meant to ask. The start of 0 prevents it.

B `-9`

That is the smallest, and the test uses >.

C `0`

Every value is below the starting 0, so it is never replaced.

D `-4`

Only if highest started at temps[0] and nothing beat it. -2 would.

THE SIGNAL

If the answer is a value that is not in the data, the starting value leaked through.

What is the output of this program?

```
1  n = 1203
2  count = 0
3  while n > 0:
4      if n % 10 == 0:
5          count += 1
6      n //= 10
7  print(count)
```

A 0

B 1

C 2

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 1203
2  count = 0
3  while n > 0:
4      if n % 10 == 0:
5          count += 1
6      n //= 10
7  print(count)
```

Last digits in turn: 3, 0, 2, 1. One of them is 0.

A 0

Stops too early. The 0 is the second digit examined.

B 1

Counts zero digits: 3, 0, 2, 1 contains one zero.

C 2

Counts the final n == 0 too. The loop ends before testing it.

D 4

Counts every digit, ignoring the if.

THE SIGNAL

The loop sees digits from right to left, and stops as soon as n reaches 0.

What is the output of this program?

```
1  n = 9
2  for d in range(2, n):
3      if n % d == 0:
4          print("not prime", d)
5          break
6  else:
7      print("prime")
```

A prime

B not prime 9

C not prime 2

D not prime 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 9
2  for d in range(2, n):
3      if n % d == 0:
4          print("not prime", d)
5          break
6  else:
7      print("prime")
```

d = 2: 9 % 2 is 1. d = 3: 9 % 3 is 0, so print and break.

A `prime`
9 has a divisor, 3, so break fires and the else is skipped.

B `not prime 9`
range(2, 9) stops at 8. 9 is never tried.

C `not prime 2`
9 % 2 is 1, not 0. 9 is odd.

D `not prime 3`
3 is the first divisor found, then break stops the search.

THE SIGNAL

The search reports the first divisor it finds, in the order range produces them.

What is the output of this program?

```
1  data = [5, 1, 4, 2]
2  for i in range(len(data) - 1):
3      if data[i] > data[i + 1]:
4          data[i], data[i + 1] = data[i + 1], data[i]
5  print(data)
```

A [1, 4, 2, 5]

B [1, 2, 4, 5]

C [1, 5, 4, 2]

D [5, 4, 2, 1]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [5, 1, 4, 2]
2  for i in range(len(data) - 1):
3      if data[i] > data[i + 1]:
4          data[i], data[i + 1] = data[i + 1], data[i]
5  print(data)
```

i = 0: [1, 5, 4, 2]. i = 1: [1, 4, 5, 2]. i = 2: [1, 4, 2, 5].

A `[1, 4, 2, 5]`

One pass carries 5 to the end; the rest is not yet sorted.

B `[1, 2, 4, 5]`

Fully sorted needs another pass. There is no outer loop.

C `[1, 5, 4, 2]`

Stops after the first comparison. The loop makes three.

D `[5, 4, 2, 1]`

That is descending order. The test swaps when the left is bigger.

THE SIGNAL

One pass of bubble sort only guarantees the largest value reaches the end.

What is the output of this program?

```
1  words = ["a", "bb", "ccc", "dd"]
2  total = 0
3  for w in words:
4      if len(w) == 2:
5          total += len(w)
6  print(total)
```

A 2

B 4

C 8

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["a", "bb", "ccc", "dd"]
2  total = 0
3  for w in words:
4      if len(w) == 2:
5          total += len(w)
6  print(total)
```

"bb" and "dd" qualify, and each adds its length, 2.

A 2

Counts the matching words. The body adds their lengths.

B 4

Two words of length 2 each add 2.

C 8

Adds every length, ignoring the condition.

D 3

Picks up "ccc". Its length is 3, not 2.

THE SIGNAL

Read what is added: += 1 counts, += value sums.

What is the output of this program?

```
1  target = 7
2  for n in [2, 7, 7, 3]:
3      if n == target:
4          print("found", end=" ")
5          break
6  else:
7      print("missing", end=" ")
8  print("done")
```

A found found done

B found missing done

C found done

D missing done

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  target = 7
2  for n in [2, 7, 7, 3]:
3      if n == target:
4          print("found", end=" ")
5          break
6  else:
7      print("missing", end=" ")
8  print("done")
```

The first 7 triggers the break. The second 7 is never examined.

A `found found done`
break stops the loop at the first 7.

B `found missing done`
break skips the else.

C `found done`
First match, break, else skipped, then the line after the loop.

D `missing done`
7 is in the list, so break fires.

THE SIGNAL

A search with break finds the first match only, and the else runs only when there was no match.

What is the output of this program?

```
1  n = 120
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10
5      n //= 10
6  print(rev)
```

A 021

B 12

C 210

D 21

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 120
2  rev = 0
3  while n > 0:
4      rev = rev * 10 + n % 10
5      n //= 10
6  print(rev)
```

rev: 0, then 2, then 21. n: 12, then 1, then 0.

A `021`

*An int never prints a leading zero. 0 * 10 + 0 is just 0.*

B `12`

Drops the 0 but does not reverse the rest.

C `210`

The trailing 0 of 120 becomes a leading 0, which vanishes.

D **`21`**

The first digit taken is 0, so rev stays 0; then 2, then 21.

THE SIGNAL

Trailing zeros vanish when a number is reversed as an int.

MODULE 11 · BLOCK 3 - DATA COLLECTIONS

Lists I: indexing, slicing and methods

Positions from the front and the back, slices with steps, and every list method the exam uses.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 11 of 28

admin@sthithapragna.com

A list is an ordered sequence you can change. Positions start at 0, negative positions count back from the end, and a slice stops before its end.

Square brackets, commas, any types

```
1  empty = []
2  mixed = [1, "two", 3.0, True, None]
3  nested = [[1, 2], [3]]
4  letters = list("hi")
5  print(len(mixed), len(nested), letters)   # 5 2 ['h', 'i']
```

2 A list can hold any mixture of types, None included.

3 A list inside a list counts as one element. len(nested) is 2.

4 list() turns any sequence into a list: a string becomes its characters.

Every element has a positive and a negative index

ELEMENT	"A"	"B"	"C"	"D"	"E"
Positive index	0	1	2	3	4
Negative index	-5	-4	-3	-2	-1

THE RULE

The last element is always [-1], and the first is [0]. For a list of length n, the valid indexes are -n to n - 1.

Reading and changing single elements

```
1  data = [10, 20, 30, 40]
2  print(data[0], data[-1], data[len(data) - 1])   # 10 40 40
3  data[1] = 25                                     # lists can change
4  print(data)                                      # [10, 25, 30, 40]
5  print(data[data[0] // 10])                       # data[1], so 25
```

3 Assigning to an index replaces that element in place.

5 The index can be any expression. Work out the inner one first.

What goes wrong, and what does not

EXPRESSION, WITH DATA = [1, 2, 3]	RESULT
`data[3]`	IndexError: list index out of range
`data[-4]`	IndexError: list index out of range
`data[1.0]`	TypeError: list indices must be integers or slices
`data[int(-1 / 2)]`	1: int(-0.5) is 0, so this is data[0]

ON THE EXAM

Positions outside -n to n - 1 raise IndexError. An index must be an int, even 1.0 is refused.

list[start:stop] gives a new list

```
1  a = [1, 2, 3, 4, 5]
2  print(a[1:3])      # [2, 3]        start included, stop excluded
3  print(a[:2])       # [1, 2]        from the beginning
4  print(a[2:])       # [3, 4, 5]     to the end
5  print(a[-2:])      # [4, 5]        the last two
6  print(a[:])        # [1, 2, 3, 4, 5]  a full copy
7  print(a[1:100])    # [2, 3, 4, 5]  no error
```

2 Exactly like range: from index 1 up to, but not including, index 3.

5 a[-n:] is the last n elements, whatever the list's length.

7 Slices never raise IndexError. Out-of-range ends are cut back to fit.

list[start:stop:step]

```
1  a = [1, 2, 3, 4, 5]
2  print(a[::2])      # [1, 3, 5]        every second element
3  print(a[1::2])     # [2, 4]
4  print(a[::-1])     # [5, 4, 3, 2, 1]  reversed
5  print(a[3:0:-1])   # [4, 3, 2]        stop 0 is excluded
6  print(a[3:1])      # []               step +1 cannot go down
```

4 A step of -1 with no start or stop walks the whole list backwards.

5 Counting down from index 3, stopping before index 0: indexes 3, 2, 1.

6 As with range, a step pointing away from stop gives an empty result, not an error.

Slice assignment and del

```
1  a = [1, 2, 3, 4, 5]
2  a[1:3] = [9]         # [1, 9, 4, 5]        two out, one in
3  a[1:1] = [7, 8]      # [1, 7, 8, 9, 4, 5]  empty slice: insert
4  del a[0]             # [7, 8, 9, 4, 5]
5  del a[1:3]           # [7, 4, 5]
6  print(a)
```

2 The replaced slice and the new values can have different lengths.

4-5 del removes one element, or a whole slice. del a[:] empties the list.

With a step, sizes must match: x[::2] = [1, 2] on 5 elements raises ValueError. del a removes the name a.

append, extend, insert, + and *

CODE, WITH A = [1, 2]	RESULT	CHANGES A?
`a.append(3)`	[1, 2, 3]	yes; returns None
`a.extend([3, 4])`	[1, 2, 3, 4]	yes; returns None
`a.insert(0, 9)`	[9, 1, 2]	yes; returns None
`a + [3]`	[1, 2, 3]	no: a new list
`a * 2`	[1, 2, 1, 2]	no: a new list

THE RULE

Every list method that changes the list returns None. The operators + and * build a new list instead.

One element, or each element

APPEND

Adds one element

```
a = [1, 2]
a.append([3, 4])
print(a, len(a))
```

[1, 2, [3, 4]] 3. The whole list goes in as a single element.

EXTEND

Adds each element

```
a = [1, 2]
a.extend([3, 4])
print(a, len(a))
```

[1, 2, 3, 4] 4. extend("hi") would add 'h' and 'i'.

ON THE EXAM

append always adds exactly one element, whatever you give it. extend adds as many as its argument holds.

insert(i, x) puts x in front of position i

```
1  x = [0, 1, 2]
2  x.insert(0, "a")      # ['a', 0, 1, 2]
3  x.insert(-1, "b")     # ['a', 0, 1, 'b', 2]   before the last element
4  x.insert(99, "c")     # ['a', 0, 1, 'b', 2, 'c']  past the end: appends
5  print(x)
```

3 insert(-1, x) does not add at the end. It goes in front of the current last element.

4 A position beyond the end is not an error. The item is simply appended.

pop returns what it removes. remove does not

```
1  a = [3, 1, 3, 2]
2  last = a.pop()          # 2      removes and returns the last
3  first = a.pop(0)        # 3      removes and returns index 0
4  a.remove(3)             #        removes the first 3 it finds
5  print(last, first, a)   # 2 3 [1]
6  print(a.remove(1))      # None
```

2-3 pop() takes the last element; pop(i) takes the one at index i. Both return it.

4 remove(x) searches by value, deletes the first match, and returns None.

clear() empties the list. del a[i] removes by position, like pop, but returns nothing.

When list methods fail

CODE	ERROR
`[].pop()`	IndexError: pop from empty list
`[1].remove(5)`	ValueError: list.remove(x): x not in list
`[1].index(5)`	ValueError. Python 3.14 message: list.index(x): x not in list
`[1, 2] + 3`	TypeError: can only concatenate list (not "int") to list

THE TIEBREAKER

Missing position: IndexError. Missing value: ValueError. Wrong type: TypeError.

index, count and in

```
1  a = [5, 3, 5, 1]
2  print(a.index(5))      # 0     the first position only
3  print(a.count(5))      # 2
4  print(3 in a, 9 not in a)   # True True
5  print(0 in [False])    # True: in compares with ==
```

2 index returns the position of the first match, even when there are several.

5 0 == False is True, so in finds it. Module four's rule again.

sort() and reverse() change the list and return None

```
1  a = [3, 1, 2]
2  a.sort()                # [1, 2, 3]
3  a.sort(reverse=True)    # [3, 2, 1]
4  a.reverse()             # [1, 2, 3]: just flips the current order
5  print(a.sort())         # None
```

3 reverse=True sorts from largest to smallest.

4 reverse() does not sort. It turns the list back to front, as it is now.

5 Printing the result of sort() prints None. The sorted list is in a itself.

What to carry into the exam

- Index 0 is first, -1 is last. Outside -n to n - 1: **IndexError**.
- Slices include start, exclude stop, and **never raise IndexError**.
- append adds one element; extend adds each. insert(-1, x) goes before the last.
- pop returns what it removes; remove, append, extend, insert, sort and reverse return None.
- Missing position: IndexError. Missing value: ValueError.

What is the output of this program?

```
1  data = [4, 0, 2, 1]
2  print(data[data[-1]] + data[data[1]])
```

A 4

B 1

C 5

D 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [4, 0, 2, 1]
2  print(data[data[-1]] + data[data[1]])
```

data[-1] is 1, so data[1] is 0. data[1] is 0, so data[0] is 4. 0 + 4 = 4.

A 4

Inner indexes first: data[1] is 0, data[0] is 4.

B 1

Adds the inner values, data[-1] + data[1]. Each is used as an index.

C 5

Reads data[-1] as 1 and adds data[0]. The first term is data[1].

D 2

Treats -1 as the second-last element. -1 is the last.

THE SIGNAL

For nested indexing, evaluate the innermost brackets first, then use the result as the index.

What is the output of this program?

```
1  a = ["p", "y", "t", "h", "o", "n"]
2  print(a[4:1:-1])
```

A `[]`

B `['h', 't', 'y']`

C `['o', 'h', 't', 'y']`

D `['o', 'h', 't']`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["p", "y", "t", "h", "o", "n"]
2  print(a[4:1:-1])
```

Indexes 4, 3, 2. Index 1 is the stop, so it is excluded.

A `[]`
The step is -1, so counting down from 4 to 1 is possible.

B `['h', 't', 'y']`
Starts at index 3. Start is included: index 4 is 'o'.

C `['o', 'h', 't', 'y']`
Includes index 1. Stop is excluded, even counting down.

D `['o', 'h', 't']`
Start 4 included, stop 1 excluded: indexes 4, 3, 2.

THE SIGNAL

With a negative step, start is still included and stop is still excluded.

What is the output of this program?

```
1  a = [1, 2]
2  a.append([3, 4])
3  a.extend([5, 6])
4  print(len(a))
```

A 6

B 5

C 4

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  a.append([3, 4])
3  a.extend([5, 6])
4  print(len(a))
```

a is [1, 2, [3, 4], 5, 6].

A 6

Treats append like extend. The appended list is one element.

B 5

2, plus 1 for the appended list, plus 2 from extend.

C 4

Treats extend like append.

D 3

Ignores the extend.

THE SIGNAL

append adds exactly one element. extend adds one per item of its argument.

What is the output of this program?

```
1  x = [1, 2, 3]
2  x.insert(-1, 9)
3  x.insert(0, x.pop())
4  print(x)
```

A `[3, 1, 2, 9]`

B `[9, 1, 2, 3]`

C `[1, 2, 3, 9]`

D `[3, 1, 2, 9, 3]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = [1, 2, 3]
2  x.insert(-1, 9)
3  x.insert(0, x.pop())
4  print(x)
```

After line 2: [1, 2, 9, 3]. pop() takes 3, leaving [1, 2, 9]. Insert 3 at the front.

A `[3, 1, 2, 9]`
9 goes before the last; pop moves the 3 to the front.

B `[9, 1, 2, 3]`
Assumes insert(-1) appends, so pop moves the 9.

C `[1, 2, 3, 9]`
Ignores line 3.

D `[3, 1, 2, 9, 3]`
pop removes the element it returns. The 3 is moved, not copied.

THE SIGNAL

insert(-1, x) lands before the last element. pop() removes and returns, so the element moves.

What is the output of this program?

```
1  a = [2, 4, 2, 6]
2  b = a.remove(2)
3  c = a.pop(1)
4  print(a, b, c)
```

A `[4, 6] 2 2`

B `[2, 6] None 4`

C `[4, 6] None 2`

D `[4, 2] None 6`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [2, 4, 2, 6]
2  b = a.remove(2)
3  c = a.pop(1)
4  print(a, b, c)
```

remove(2) deletes the first 2: [4, 2, 6]. pop(1) takes index 1, which is 2: [4, 6].

A `[4, 6] 2 2`
remove returns None, not the value removed.

B `[2, 6] None 4`
Treats remove(2) as removing index 2. It removes the value 2.

C `[4, 6] None 2`
remove deletes the first 2 and returns None; pop(1) returns the other 2.

D `[4, 2] None 6`
Treats pop(1) as popping the last element.

THE SIGNAL

remove works by value and returns None. pop works by position and returns the element.

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:4] = [0]
3  del a[-1]
4  print(a)
```

A `[1, 0, 4, 5]`

B `[1, 0]`

C `[1, 0, 0, 0]`

D `[0, 5]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:4] = [0]
3  del a[-1]
4  print(a)
```

a[1:4] is [2, 3, 4]. Replaced by [0]: [1, 0, 5]. Then del a[-1]: [1, 0].

A `[1, 0, 4, 5]`

Replaces only indexes 1 and 2. Stop 4 means indexes 1, 2 and 3.

B `[1, 0]`

Three elements replaced by one, then the last one deleted.

C `[1, 0, 0, 0]`

Assumes the 0 fills every slot. The slice shrinks to one element.

D `[0, 5]`

Starts the slice at index 0. Index 1 is the 2.

THE SIGNAL

Slice assignment without a step can change the list's length.

> my_list = [5, 4, 3, 2] has been assigned. Each expression is evaluated on its own.

Which two expressions raise an exception? (Choose two.)

A `my_list[-4]`

B `my_list[4]`

C `my_list[1:10]`

D `my_list.index(1)`

◆ PAUSE AND DECIDE

Which two expressions raise an exception? (Choose two.)

A `my_list[-4]`

-4 is the first element of four. 5.

B `my_list[4]`

Valid indexes stop at 3. IndexError.

C `my_list[1:10]`

Slices never raise IndexError. [4, 3, 2].

D `my_list.index(1)`

1 is not in the list. ValueError.

THE SIGNAL

Indexing checks bounds; slicing does not. index() of a missing value is a ValueError.

What is the output of this program?

```
1  data = [0, None, [], "", [[]]]
2  print(len(data), [] in data, 0 in data[4])
```

A 4 True False

B 5 False False

C 5 True False

D 5 True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [0, None, [], "", [[]]]
2  print(len(data), [] in data, 0 in data[4])
```

data[4] is [[]]: a list holding one empty list. 0 == [] is False.

A `4 True False`
Every element counts, even None and empty ones.

B `5 False False`
data[2] is an empty list, and [] == [] is True.

C `5 True False`
Five elements; [] is data[2]; 0 does not equal [].

D `5 True True`
0 is not in [[]]. Its only element is [], and 0 == [] is False.

THE SIGNAL

len counts every top-level element, empty or not. in compares each element with ==.

MODULE 12 · BLOCK 3 - DATA COLLECTIONS

Lists II: references, copies and comprehensions

Two names on one list, three ways to copy, sort versus sorted, comprehensions, and lists inside lists.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 12 of 28

admin@sthithapragna.com

Assignment **never copies** a list. It sticks another name on the same list. To get a separate list, you must copy it.

b = a sticks a second label on the same list

```
1  a = [1, 2, 3]
2  b = a
3  b[0] = 9
4  print(a, b is a)     # [9, 2, 3] True
```

2 No new list is made. b and a are two names for one list.

3 Changing the list through b changes what a sees, because it is the same list.

Three ways to copy a list

```
1  a = [1, 2, 3]
2  c1 = a[:]            # slice of everything
3  c2 = list(a)         # build a new list from a
4  c3 = a.copy()        # the copy method
5  c1[0] = 0
6  print(a, c1)         # [1, 2, 3] [0, 2, 3]
7  print(c2 == a, c2 is a)   # True False
```

2-4 All three build a new list holding the same elements.

5-6 Changing the copy leaves the original untouched.

7 Equal values, but a different object.

Change the list, or move the label?

CHANGE THE LIST

Seen through every name

```
1  a = [1, 2]
2  b = a
3  b.append(3)
4  print(a)
```

[1, 2, 3]. append changes the one list both names are stuck on.

MOVE THE LABEL

Only that name changes

```
1  a = [1, 2]
2  b = a
3  b = [7]
4  print(a)
```

[1, 2]. b is peeled off and stuck on a new list. a is untouched.

THE TIEBREAKER

Methods, index assignment and del change the list. A plain b = … only moves the name b.

a += [2] versus a = a + [2]

+=

Changes the list in place

```
1  a = [1]
2  b = a
3  a += [2]
4  print(b)
```

[1, 2]. For lists, += extends the existing list, so b sees it.

= +

Builds a new list

```
1  a = [1]
2  b = a
3  a = a + [2]
4  print(b)
```

[1]. + makes a new list, and a is moved onto it. b keeps the old one.

THE TRAP

For lists, x += y is not the same as x = x + y when another name shares the list.

Deleting through one name empties it for both

```
1  nums = [1, 2, 3]
2  vals = nums
3  del vals[1:2]
4  print(nums, len(nums))    # [1, 3] 2
5  del vals[:]
6  print(nums)               # []
```

3 del on a slice changes the list itself, so nums shrinks too.

5 del vals[:] empties the shared list. del vals would only remove the name.

A copy is shallow: inner lists are still shared

```
1  m = [[1, 2], [3]]
2  s = m[:]
3  s[0][0] = 99       # changes an inner list both share
4  s[1] = [7]         # replaces a slot in s only
5  print(m)           # [[99, 2], [3]]
6  print(s)           # [[99, 2], [7]]
```

2 The copy is a new outer list, but its elements are the same inner lists.

3 Reaching into a shared inner list changes it for both.

4 Replacing a whole slot only affects the copy.

[[0] * 3] * 3 repeats one row three times

MULTIPLYING A LIST OF LISTS

Three names for one row

```
1  grid = [[0] * 3] * 3
2  grid[0][0] = 1
3  print(grid)
```

[[1, 0, 0], [1, 0, 0], [1, 0, 0]]. Every row is the same list.

A COMPREHENSION

Three separate rows

```
1  grid = [[0] * 3 for i in range(3)]
2  grid[0][0] = 1
3  print(grid)
```

[[1, 0, 0], [0, 0, 0], [0, 0, 0]]. A new row is built each pass.

THE TRAP

* repeats references, not copies. Build rows with a comprehension or a loop.

sort() changes the list; sorted() returns a new one

LIST.SORT()

In place, returns None

```
1  a = [3, 1, 2]
2  b = a.sort()
3  print(a, b)
```

[1, 2, 3] None. a is sorted; b is None.

SORTED(LIST)

New list, original kept

```
1  a = [3, 1, 2]
2  b = sorted(a)
3  print(a, b)
```

[3, 1, 2] [1, 2, 3]. a is unchanged; b is sorted.

THE TIEBREAKER

Assigning the result of sort() is always a mistake: you get None.

min, max, sum, len, and more on sorted

```
1  nums = [4, -2, 7]
2  print(min(nums), max(nums), sum(nums), len(nums))   # -2 7 9 3
3  print(max(["apple", "Banana"]))    # apple
4  print(sorted("cab"))                # ['a', 'b', 'c']
5  print(sorted(nums, reverse=True))   # [7, 4, -2]
```

3 Strings compare by code point: lower-case a beats capital B.

4 sorted() always returns a list, even when given a string.

max([]) raises ValueError. sum([]) is 0. Sorting a mix of str and int raises TypeError.

[expression for item in sequence if condition]

```
squares = [x * x for x in range(4)]              # [0, 1, 4, 9]
threes = [x for x in range(10) if x % 3 == 0]    # [0, 3, 6, 9]
caps = [c.upper() for c in "abc"]                # ['A', 'B', 'C']
nothing = [i for i in range(-1, -2)]            # []
```

1. For each x, the expression on the left becomes one element.
2. The if filters: only items that pass it produce an element.
4. An empty range gives an empty list. range(-1, -2) counts up, so it is empty.

Every comprehension is a loop with append

THE LOOP

Four lines

```
evens = []
for n in range(6):
    if n % 2 == 0:
        evens.append(n)
```

[0, 2, 4]

THE COMPREHENSION

One line, same result

```
evens = [n for n in range(6) if n % 2 == 0]
```

[0, 2, 4]. Unlike the loop, n does not exist afterwards.

EXAM TECHNIQUE

If a comprehension confuses you, rewrite it as the loop. The result is always the same list.

A matrix: grid[row][column]

```
1  board = [[1, 2, 3],
2           [4, 5, 6]]
3  print(board[1][2])                  # 6    row 1, then column 2
4  print(len(board), len(board[0]))    # 2 3  rows, then columns
5  print([row[1] for row in board])    # [2, 5]  a whole column
6  print(board[2][0])                  # IndexError: there is no row 2
```

3 The first index picks a row, the second picks an element within it.

5 A column is not stored anywhere. Build it by taking one element from each row.

6 Two rows means valid row indexes 0 and 1.

A cube, from Python Essentials 1

```
1  rooms = [[[False for r in range(3)] for f in range(2)]
2           for b in range(2)]
3  rooms[1][0][2] = True       # building 1, floor 0, room 2
4  print(len(rooms), len(rooms[0]), len(rooms[0][0]))   # 2 2 3
5  print(rooms[1][0])          # [False, False, True]
```

1-2 Nested comprehensions: 2 buildings, each with 2 floors, each with 3 rooms.

3 Three indexes, outermost first: building, then floor, then room.

A stack is last in, first out

```
1  stack = []
2  stack.append(1)
3  stack.append(2)
4  stack.append(3)
5  print(stack.pop(), stack.pop(), stack)   # 3 2 [1]
6  queue = [1, 2, 3]
7  print(queue.pop(0), queue)               # 1 [2, 3]
```

2-5 append pushes onto the top; pop takes from the top. LIFO: last in, first out.

6-7 pop(0) takes from the front instead: first in, first out, or FIFO, like a queue.

What to carry into the exam

- b = a shares one list. [:], list() and copy() make a new one.
- Methods, index assignment, del and += change the shared list. b = [...] only moves b.
- Copies are **shallow**: inner lists are shared. [[0]*3]*3 repeats one row.
- **sort() returns None**; sorted() returns a new list.
- A comprehension is a loop with append; its variable does not survive it.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  c = a[:]
4  b[0] = 7
5  c[1] = 8
6  print(a)
```

A [7, 2, 3]

B [7, 8, 3]

C [1, 2, 3]

D [1, 8, 3]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  c = a[:]
4  b[0] = 7
5  c[1] = 8
6  print(a)
```

b is the same list as a. c is a separate copy.

A `[7, 2, 3]`

b shares a's list, so the 7 shows. c is a copy, so the 8 does not.

B `[7, 8, 3]`

Treats c as another alias. [:] makes a new list.

C `[1, 2, 3]`

Treats b as a copy. Assignment never copies.

D `[1, 8, 3]`

Swaps the two: b is the alias, c is the copy.

THE SIGNAL

Label each name: alias or copy. Only changes through aliases reach the original.

What is the output of this program?

```
1  x = [1]
2  y = x
3  x += [2]
4  x = x + [3]
5  print(y, x)
```

A `[1, 2, 3] [1, 2, 3]`

B `[1] [1, 2, 3]`

C `[1, 2] [1, 2, 3]`

D `[1, 2] [1, 2]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = [1]
2  y = x
3  x += [2]
4  x = x + [3]
5  print(y, x)
```

Line 3 extends the shared list: y sees [1, 2]. Line 4 builds a new list for x only.

A `[1, 2, 3] [1, 2, 3]`
Treats x = x + [3] as changing the shared list. It makes a new one.

B `[1] [1, 2, 3]`
Treats x += [2] as making a new list. For lists it extends in place.

C `[1, 2] [1, 2, 3]`
+= changes the shared list; = + moves x to a new list.

D `[1, 2] [1, 2]`
Ignores line 4. x itself does get the 3.

THE SIGNAL

For lists: += changes the list in place; x = x + … builds a new list and moves x.

What is the output of this program?

```
1  m = [[0, 0], [0, 0]]
2  n = m.copy()
3  n[0][0] = 5
4  n[1] = [9, 9]
5  print(m)
```

A `[[0, 0], [0, 0]]`

B `[[5, 0], [9, 9]]`

C `[[5, 0], [0, 0]]`

D `[[0, 0], [9, 9]]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  m = [[0, 0], [0, 0]]
2  n = m.copy()
3  n[0][0] = 5
4  n[1] = [9, 9]
5  print(m)
```

copy() is shallow: m and n hold the same inner lists until one is replaced.

A `[[0, 0], [0, 0]]`
Assumes a deep copy. Inner lists are shared.

B `[[5, 0], [9, 9]]`
Treats n as an alias. Replacing n[1] only changes n.

C `[[5, 0], [0, 0]]`
Line 3 changes a shared inner list; line 4 changes n's outer list only.

D `[[0, 0], [9, 9]]`
Reverses which change reaches m.

THE SIGNAL

In a shallow copy, reaching into an inner list is shared; replacing a whole slot is not.

What is the output of this program?

```
1  nums = [3, 1, 2]
2  result = nums.sort()
3  print(result, sorted(nums, reverse=True))
```

A `[1, 2, 3] [3, 2, 1]`

B `None [3, 2, 1]`

C `None [1, 2, 3]`

D `[1, 2, 3] None`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [3, 1, 2]
2  result = nums.sort()
3  print(result, sorted(nums, reverse=True))
```

sort() sorts nums in place and returns None. sorted() returns a new list.

A `[1, 2, 3] [3, 2, 1]`
Assumes sort() returns the sorted list. It returns None.

B `None [3, 2, 1]`
sort() gives None; sorted with reverse=True gives a new descending list.

C `None [1, 2, 3]`
Ignores reverse=True.

D `[1, 2, 3] None`
Swaps which function returns None.

THE SIGNAL

sort() returns None, always. sorted() returns a list, always.

What is the output of this program?

```
1  data = [x * 2 for x in range(5) if x % 2 == 1]
2  print(data)
```

A [0, 4, 8]

B [1, 3]

C [2, 6, 10]

D [2, 6]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [x * 2 for x in range(5) if x % 2 == 1]
2  print(data)
```

range(5) is 0 to 4. The odd ones are 1 and 3. Doubled: 2 and 6.

A `[0, 4, 8]`
Keeps the even values. The test keeps the odd ones.

B `[1, 3]`
Filters correctly but forgets to double.

C `[2, 6, 10]`
Includes 5. range(5) stops at 4.

D `[2, 6]`
Filter first: 1 and 3. Then apply the expression: 2 and 6.

THE SIGNAL

Read a comprehension in order: sequence, then filter, then the expression on the left.

What is the output of this program?

```
1  grid = [[0] * 2] * 2
2  grid[1][0] = 5
3  print(grid)
```

A `[[5, 0], [5, 0]]`

B `[[0, 0], [5, 0]]`

C `[[0, 5], [0, 5]]`

D `[[0, 0], [0, 0]]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  grid = [[0] * 2] * 2
2  grid[1][0] = 5
3  print(grid)
```

The outer * 2 repeats a reference to one inner list. Both rows are that list.

A `[[5, 0], [5, 0]]`
Both rows are the same list, so the change appears twice.

B `[[0, 0], [5, 0]]`
*Assumes two separate rows. * repeats one row.*

C `[[0, 5], [0, 5]]`
Mixes up row and column. grid[1][0] is the first element of a row.

D `[[0, 0], [0, 0]]`
The assignment does change the shared row.

THE SIGNAL

[[…]] * n makes n references to one inner list. Changing one row changes them all.

> board = [[1, 2, 3], [4, 5, 6]] has been assigned. Each expression is evaluated on its own.

Which two expressions evaluate to 6? (Choose two.)

A `board[1][2]`

B `board[2][1]`

C `board[-1][-1]`

D `max(board[0])`

◆ PAUSE AND DECIDE

Which two expressions evaluate to 6? (Choose two.)

A `board[1][2]`
Row 1 is [4, 5, 6]; element 2 is 6.

B `board[2][1]`
There is no row 2. IndexError.

C `board[-1][-1]`
Last row, last element: 6.

D `max(board[0])`
board[0] is [1, 2, 3], whose maximum is 3.

THE SIGNAL

Row first, then column. Negative indexes work at every level.

What is the output of this program?

```
1  stack = []
2  for n in [4, 5, 6]:
3      stack.append(n)
4  stack.pop()
5  stack.append(stack.pop() * 10)
6  print(stack)
```

A [4, 5, 60]

B [4, 50]

C [50, 4]

D [40, 5]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  stack = []
2  for n in [4, 5, 6]:
3      stack.append(n)
4  stack.pop()
5  stack.append(stack.pop() * 10)
6  print(stack)
```

[4, 5, 6] -> pop removes 6 -> [4, 5] -> pop removes 5 -> append 50 -> [4, 50].

A `[4, 5, 60]`

Forgets the first pop removed the 6.

B `[4, 50]`

Two pops take 6 then 5; 5 * 10 goes back on top.

C `[50, 4]`

append adds to the end, not the front.

D `[40, 5]`

Pops from the front. pop() takes from the top: last in, first out.

THE SIGNAL

A stack only ever changes at its top: append pushes there, pop takes from there.

MODULE 13 · BLOCK 3 - DATA COLLECTIONS

Tuples

The comma that makes a tuple, reading tuples like lists, what immutable really means, and packing and unpacking.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 13 of 28

admin@sthithapragna.com

A tuple is a sequence you **cannot change**. And it is the **comma** that makes a tuple, not the brackets.

Commas make tuples

```
1  t1 = (1, 2, 3)
2  t2 = 1, 2, 3            # packing: the brackets are optional
3  t3 = (5,)               # one element needs a trailing comma
4  t4 = (5)                # no comma: this is just the int 5
5  t5 = ()                 # the empty tuple
6  print(t1 == t2, type(t3), type(t4), len(t5))
```

2 Values separated by commas become a tuple, with or without brackets.

3-4 Without the comma, brackets only group. (5) is the int 5; (5,) is a tuple.

Output: True <class 'tuple'> <class 'int'> 0

A stray comma, a stray point

```
1  y = 2 + 3 * 5,           # the trailing comma makes a tuple
2  print(y)                 # (17,)
3  print(type((2.)), type((2,)))      # float, then tuple
4  print(tuple("ab"), tuple(range(3)))  # ('a', 'b') (0, 1, 2)
```

1-2 The expression is worked out first, then the comma packs it into a one-element tuple.

3 (2.) is a float in brackets. (2,) is a tuple. One character apart.

4 tuple() converts any sequence, just as list() does.

Everything that reads a list reads a tuple

```
1  data = (1, 2, 4, 8)
2  print(data[1], data[-1])          # 2 8
3  print(data[1:3], data[-2:-1])     # (2, 4) (4,)
4  print(len(data), 4 in data)       # 4 True
5  print(data.count(2), data.index(8))   # 1 3
```

2 Indexing gives one element: an int here.

3 Slicing gives a tuple, even when only one element is left.

5 count and index are the only two tuple methods.

What you cannot do to a tuple

CODE, WITH T = (1, 2, 3)	RESULT
`t[0] = 9`	TypeError: 'tuple' object does not support item assignment
`del t[0]`	TypeError: 'tuple' object doesn't support item deletion
`t.append(4) t.sort()`	AttributeError: tuples have no such methods
`t.index(9)`	ValueError: tuple.index(x): x not in tuple
`t[3]`	IndexError: tuple index out of range
`del t`	Works: it removes the name t, not an element

THE TIEBREAKER

Changing an element: TypeError. Calling a list-only method: AttributeError.

+ and * work, because they make a new tuple

```
1  t = (1, 2)
2  t = t + (3,)              # a new tuple: (1, 2, 3)
3  print(t * 2)              # (1, 2, 3, 1, 2, 3)
4  t += (4,)                 # also a new tuple, rebound to t
5  print(t, sorted((3, 1, 2)))   # (1, 2, 3, 4) [1, 2, 3]
6  print(t + [5])            # TypeError: tuple + list
```

2-4 Nothing is changed. Each line builds a new tuple and moves the name onto it.

5 sorted() of a tuple returns a list.

6 You can only join a tuple to another tuple.

For a tuple, += always makes a new object

TUPLE

b keeps the old tuple

```
1  a = (1,)
2  b = a
3  a += (2,)
4  print(b)
```

(1,). A tuple cannot grow, so += builds a new one for a.

LIST

b sees the change

```
1  a = [1]
2  b = a
3  a += [2]
4  print(b)
```

[1, 2]. For a list, += extends the shared list in place.

ON THE EXAM

Same operator, different behaviour: immutable types rebuild, mutable types change in place.

A tuple can hold a list, and the list can still change

```
1  t = ([1, 2], "x")
2  t[0].append(3)          # fine: changes the list inside
3  print(t)                # ([1, 2, 3], 'x')
4  t[0] = [9]              # TypeError: replaces a tuple slot
```

2 The tuple still holds the same list. Only the list's contents changed.

4 Replacing the element itself is what a tuple forbids.

Several values out of one tuple

```
1  point = 3, 4              # packing
2  x, y = point              # unpacking: x is 3, y is 4
3  a, b = 1, 2
4  a, b = b, a               # the swap is pack, then unpack
5  print(x, y, a, b)         # 3 4 2 1
6  for name, score in [("Ann", 7), ("Bob", 9)]:
7      print(name, score)    # Ann 7, then Bob 9
```

2 Unpacking assigns each element to a name, in order.

4 The right side is packed into a tuple first, then unpacked into a and b.

6-7 A loop over pairs can unpack each pair into two names.

The counts must match: x, y = (1, 2, 3) raises ValueError: too many values to unpack.

Tuple versus list

TUPLE

Fixed after creation

- Written with commas; () is empty
- Only count() and index()
- Can be a dictionary key
- Suits fixed records: a point, a date

LIST

Changes as you go

- Written with []
- append, insert, sort, and more
- Cannot be a dictionary key
- Suits collections that grow

ON THE EXAM

Both support indexing, slicing, len, in, for, + and *. Only the list can be changed in place.

Tuples as keys, and comparing tuples

```
1  grid = {(0, 0): "start", (2, 3): "end"}
2  print(grid[2, 3])                    # end
3  print((1, 2) < (1, 3), (2,) > (1, 9))   # True True
```

2 Inside the square brackets, 2, 3 is packed into the tuple (2, 3).

3 Tuples compare element by element, like strings compare character by character.

What to carry into the exam

- The **comma** makes a tuple: (5,) is a tuple, (5) is an int, 1, 2 is a tuple.
- Indexing gives an element; **slicing gives a tuple**, even (4,).
- Assigning or deleting an element: **TypeError**. append or sort: **AttributeError**.
- + and * and += make a new tuple. A list inside a tuple can still change.
- Unpacking needs matching counts; tuples can be dictionary keys.

What is the output of this program?

```
1  data = (5, 10, 15, 20)
2  data = data[-3:-1]
3  data = data[-1]
4  print(data)
```

A (15,)

B (10, 15)

C The code raises a TypeError

D 15

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = (5, 10, 15, 20)
2  data = data[-3:-1]
3  data = data[-1]
4  print(data)
```

data[-3:-1] is (10, 15), a tuple. Its [-1] is the int 15.

A `(15,)`
Line 3 indexes, which gives an element, not a slice.

B `(10, 15)`
Stops after line 2. Line 3 takes one element.

C The code raises a TypeError
Nothing changes the tuple; the name is just rebound.

D **15**
The slice is (10, 15); indexing it gives the int 15.

THE SIGNAL

Slicing a tuple gives a tuple. Indexing gives an element.

What is the output of this program?

```
1  t = (1,) * 2 + (2,)
2  t += t[1:]
3  print(len(t), t)
```

A `5 (1, 1, 2, 1, 2)`

B `4 (1, 1, 2, 2)`

C `3 (1, 1, 2)`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (1,) * 2 + (2,)
2  t += t[1:]
3  print(len(t), t)
```

(1,) * 2 is (1, 1); plus (2,) is (1, 1, 2). t[1:] is (1, 2). Joined: (1, 1, 2, 1, 2).

A `5 (1, 1, 2, 1, 2)`
+= builds a new tuple from t and its slice.

B `4 (1, 1, 2, 2)`
Takes t[1:] as just the last element.

C `3 (1, 1, 2)`
Assumes += cannot work on a tuple and changes nothing.

D The code raises a TypeError
+= on a tuple is legal: it makes a new tuple.

THE SIGNAL

+= on a tuple is allowed. It never changes the old tuple; it builds a new one.

What is the output of this program?

```
1  t = ([0], 5)
2  t[0][0] += 1
3  u = t
4  u[0].append(9)
5  print(t)
```

A `([0], 5)`

B The code raises a TypeError

C `([1, 9], 5)`

D `([1], 5)`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = ([0], 5)
2  t[0][0] += 1
3  u = t
4  u[0].append(9)
5  print(t)
```

Both changes happen inside the list. No tuple slot is ever replaced.

A `([0], 5)`
Assumes the tuple protects the list's contents. It does not.

B The code raises a TypeError
t[0][0] changes the list, not a tuple slot. That is allowed.

C `([1, 9], 5)`
The list inside changes twice; u is the same tuple as t.

D `([1], 5)`
Forgets that u and t name the same tuple, holding the same list.

THE SIGNAL

A tuple fixes which objects it holds, not what happens inside them.

> t = (1, 2, 3) has been assigned. Each line is run on its own.

Which two lines raise an exception? (Choose two.)

A `print(t.count(4))`

B `print(t[3])`

C `print(t[1:9])`

D `t.append(4)`

◆ PAUSE AND DECIDE

Which two lines raise an exception? (Choose two.)

A `print(t.count(4))`

Counting a missing value is not an error. It prints 0.

B `print(t[3])`

Valid indexes are 0 to 2. IndexError.

C `print(t[1:9])`

Slices never raise IndexError. Prints (2, 3).

D `t.append(4)`

Tuples have no append method. AttributeError.

THE SIGNAL

count() of a missing value is 0, not an error. index() of a missing value would raise ValueError.

MODULE 14 · BLOCK 3 - DATA COLLECTIONS

Dictionaries

Keys and values, which keys are allowed, KeyError versus get(), changing a dictionary, and looping over it.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 14 of 28

admin@sthithapragna.com

A dictionary maps **keys** to values. You look up by key, never by position, and each key appears **only once**.

Braces, colons, commas

```
ages = {"Ann": 31, "Bob": 27}
empty = {}
pairs = dict([("x", 1), ("y", 2)])
print(ages["Ann"], len(ages), type(empty))   # 31 2 <class 'dict'>
print(pairs)                                 # {'x': 1, 'y': 2}
```

1. Each entry is key colon value. len counts the pairs.
2. Empty braces make an empty dictionary, not a set.
3. dict() turns a sequence of two-element pairs into a dictionary.

Keys must be hashable. Values can be anything

TYPE	AS A KEY?	WHY
`str, int, float, bool`	yes	immutable values
`tuple, e.g. (1, 2)`	yes	immutable, as long as its contents are
`list, e.g. [1, 2]`	no: TypeError	a list can change, so it cannot be hashed
`dict`	no: TypeError	also changeable

ON THE EXAM

Python 3.14 words the error as: cannot use 'list' as a dict key (unhashable type: 'list'). Still a TypeError.

1, 1.0 and True are the same key

```
1  d = {1: "int", 1.0: "float", True: "bool"}
2  print(d, len(d))            # {1: 'bool'} 1
3  d2 = {"a": 1, "a": 2}
4  print(d2)                   # {'a': 2}
```

1-2 1 == 1.0 == True, so all three land in one entry. The first key stays; the last value wins.

3-4 Repeating a key in a literal is legal. The later value replaces the earlier one.

Square brackets fail. get() does not

D[KEY]

KeyError if missing

```
1  d = {"a": 1}
2  print(d["z"])
```

KeyError: 'z'. The error names the missing key.

D.GET(KEY, DEFAULT)

None, or your default

```
1  d = {"a": 1}
2  print(d.get("z"), d.get("z", 0))
```

None 0. get never raises; it returns None unless you give a default.

THE TRAP

d[0] on a dictionary is a key lookup, not a position. With no key 0, it is a KeyError.

Assigning to a key adds it, or replaces its value

```
1  stock = {"tea": 5}
2  stock["cake"] = 2          # new key: added
3  stock["tea"] = 7           # existing key: value replaced
4  stock["tea"] += 1          # read the value, then write it back
5  stock.update({"cake": 3, "jam": 1})
6  print(stock)               # {'tea': 8, 'cake': 3, 'jam': 1}
```

2-3 The same syntax adds a new key or replaces an existing value.

4 += must read first, so the key must already exist. stock["milk"] += 1 is a KeyError.

5 update() adds or replaces several entries at once.

del, pop, popitem and clear

```
1  d = {"a": 1, "b": 2, "c": 3}
2  del d["a"]                  # removes the pair
3  print(d.pop("b"))           # 2: removes and returns the value
4  print(d.popitem())          # ('c', 3): the most recently added pair
5  d.clear()
6  print(d)                    # {}
```

3 pop(key) returns the value. pop(key, default) avoids a KeyError.

4 popitem() removes and returns the last pair added, as a tuple.

del d["q"] and d.pop("q") on a missing key raise KeyError. del d removes the whole name.

in checks the keys, not the values

```
1  d = {"Ann": 7, "Bob": 9}
2  print("Ann" in d, 7 in d)          # True False
3  print(7 in d.values())             # True
4  print(("Bob", 9) in d.items())     # True
```

2 in on a dictionary only ever looks at keys. 7 is a value, so it is not found.

3-4 To search values or pairs, ask values() or items() instead.

Keys by default; values() and items() on request

```
1  d = {"Ann": 7, "Bob": 9}
2  for k in d:                    # keys: Ann, Bob
3      print(k, d[k])
4  for k, v in d.items():         # pairs, unpacked
5      print(k, v)
6  print(list(d.keys()), list(d.values()))   # ['Ann', 'Bob'] [7, 9]
7  print(sorted({"z": 1, "x": 2}))           # ['x', 'z']
```

2-3 Looping over the dictionary itself gives the keys. Look values up with d[k].

4-5 items() gives each pair as a tuple, unpacked into two names.

7 sorted() of a dictionary returns its keys, sorted, as a list.

Printed directly, the views look like dict_keys(['Ann', 'Bob']). Wrap them in list() to get a plain list.

Dictionaries remember insertion order

```
1  d = {"b": 1, "a": 2}
2  d["c"] = 3
3  d["b"] = 9                  # replacing keeps b's place
4  print(list(d))              # ['b', 'a', 'c']
5  del d["b"]
6  d["b"] = 0                  # re-added: goes to the end
7  print(list(d))              # ['a', 'c', 'b']
```

3 Changing a value does not move its key.

5-6 A deleted key that is added again counts as new, and goes to the end.

Chained lookups: the value becomes the next key

```
1  data = {"one": "two", "two": "three", "three": "one"}
2  res = data["one"]                  # "two"
3  for _ in range(2):
4      res = data[res]                # "three", then "one"
5  print(res)                         # one
```

4 Each pass looks up the current value as a key. Write down res after every pass.

3 The underscore is an ordinary name, used when the loop variable is not needed.

Same rules as lists

D2 = D

Two names, one dictionary

```
1  d = {"a": 1}
2  d2 = d
3  d2["a"] = 5
4  print(d, d2 is d)
```

{'a': 5} True. The change shows through both names.

D3 = D.COPY()

A separate dictionary

```
1  d = {"a": 1}
2  d3 = d.copy()
3  d3["a"] = 5
4  print(d, d3 is d)
```

{'a': 1} False. The copy changes alone.

ON THE EXAM

copy() is shallow, as for lists: a list stored as a value is still shared.

What goes wrong

CODE, WITH D = {"A": 1}	ERROR
`d["z"] d[0] del d["z"]`	KeyError
`d["a", "b"]`	KeyError: ('a', 'b'): the commas make one tuple key
`{[1, 2]: "x"}`	TypeError: a list is unhashable
`d.vals()`	AttributeError. Did you mean: 'values'?
`deleting keys inside for k in d`	RuntimeError: dictionary changed size during iteration

THE TIEBREAKER

Missing key: KeyError. Unhashable key: TypeError. Misspelled method: AttributeError.

Related things you may see

- **{}** is an empty dictionary. **{1, 2}**, with no colons, is a **set**: not on PCEP-30-02.
- Dictionary comprehensions: {k: k * 2 for k in range(3)}.
- Python 3.9 added | to merge dictionaries: a | b.
- For the exam: literals, dict(), [], get, update, del, pop, popitem, clear, keys, values, items, copy.

What to carry into the exam

- No positions: d[0] looks for the **key** 0. Missing key: **KeyError**.
- 1, 1.0 and True are **one key**: first spelling kept, last value wins.
- get() returns None or a default; pop() returns the value; popitem() the last pair.
- in and for look at **keys**. Use values() or items() for the rest.
- Insertion order is kept. d2 = d is an alias; copy() is a shallow copy.

What is the output of this program?

```
1  data = {1: 3, 2: 1, 3: 2}
2  x = 1
3  for _ in range(len(data)):
4      x = data[x]
5  print(x)
```

A 1

B 2

C 3

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = {1: 3, 2: 1, 3: 2}
2  x = 1
3  for _ in range(len(data)):
4      x = data[x]
5  print(x)
```

Three passes: x = 1 -> 3 -> 2 -> 1.

A. 1
1 goes to 3, 3 goes to 2, 2 goes back to 1.

B. 2
Stops one pass early. len(data) is 3.

C. 3
Stops after the first pass.

D. The code raises a KeyError
Every value is also a key, so every lookup succeeds.

THE SIGNAL

For chained lookups, write the value after every pass. Count passes from len().

What is the output of this program?

```
1  d = {True: "yes", 1: "one", 1.0: "float"}
2  print(len(d), d[1])
```

A 3 one

B 1 float

C 1 yes

D 3 float

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {True: "yes", 1: "one", 1.0: "float"}
2  print(len(d), d[1])
```

True == 1 == 1.0, so one entry. d is {True: 'float'}.

A `3 one`

Treats the three keys as different. They are equal.

B `1 float`

One entry; each assignment replaced the value, so the last one stays.

C `1 yes`

Keeps the first value. The first key spelling stays, but the last value wins.

D `3 float`

Only one entry exists, so len is 1.

THE SIGNAL

Equal keys collide: one entry, the first key's spelling, the last value.

What is the output of this program?

```
1  stock = {"tea": 0, "jam": 4}
2  print(stock.get("tea", 9), stock.get("cake", 9),
3        "jam" in stock, 4 in stock)
```

A 9 9 True False

B 0 9 True True

C 0 None True False

D 0 9 True False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  stock = {"tea": 0, "jam": 4}
2  print(stock.get("tea", 9), stock.get("cake", 9),
3        "jam" in stock, 4 in stock)
```

A `9 9 True False`

tea exists, so get returns its value, 0, even though 0 is falsy.

B `0 9 True True`

4 is a value, and in only checks keys.

C `0 None True False`

A default was given, so a missing key returns 9, not None.

D `0 9 True False`

Existing key: its value. Missing key: the default. in checks keys.

THE SIGNAL

get uses the default only when the key is missing, never because the value is falsy.

What is the output of this program?

```
1  d = {"a": 1, "b": 2}
2  d["c"] = d.pop("a") + d["b"]
3  d["b"] = 5
4  print(d)
```

A `{'a': 1, 'b': 5, 'c': 3}`

B `{'c': 3, 'b': 5}`

C `{'b': 5, 'c': 3}`

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {"a": 1, "b": 2}
2  d["c"] = d.pop("a") + d["b"]
3  d["b"] = 5
4  print(d)
```

pop removes a and returns 1. 1 + 2 is 3, added as c. Replacing b keeps its place.

A `{'a': 1, 'b': 5, 'c': 3}`
pop removes the key as well as returning its value.

B `{'c': 3, 'b': 5}`
Replacing b's value does not move b to the end.

C **`{'b': 5, 'c': 3}`**
a removed, c added at the end, b updated in place.

D The code raises a KeyError
a and b both exist when they are read.

THE SIGNAL

pop removes as it returns. Updating a value keeps its key's position.

What is the output of this program?

```
menu = {"tea": 2.5, "cake": 3.75}
for item, price in menu.items():
    print(item[0] + str(price)[0], end=" ")
```

A `t2 c3`

B `tea2 cake3`

C `2t 3c`

D `t2.5 c3.75`

◆ PAUSE AND DECIDE

What is the output of this program?

```
menu = {"tea": 2.5, "cake": 3.75}
for item, price in menu.items():
    print(item[0] + str(price)[0], end=" ")
```

"tea"[0] is "t"; str(2.5)[0] is "2". Then "c" and "3".

A `t2 c3`

items() gives each pair; [0] takes a first character of each part.

B `tea2 cake3`

Forgets the [0] on the item.

C `2t 3c`

Reverses the order of the join.

D `t2.5 c3.75`

Forgets the [0] on the price.

THE SIGNAL

items() unpacks into key and value. Convert to str before indexing a number.

> d = {"a": 1} has been assigned. Each line is run on its own.

Which two lines raise an exception? (Choose two.)

A `print(d["b"])`

B `print(d.get("b"))`

C `print(d.pop("a", 0))`

D `e = {[1, 2]: "x"}`

◆ PAUSE AND DECIDE

Which two lines raise an exception? (Choose two.)

A `print(d["b"])`

b is not a key. KeyError.

B `print(d.get("b"))`

get never raises. Prints None.

C `print(d.pop("a", 0))`

a exists, so it prints 1. The default would only matter if it did not.

D `e = {[1, 2]: "x"}`

A list cannot be a key. TypeError.

THE SIGNAL

Square brackets on a missing key raise; get does not. Keys must be hashable.

MODULE 15 · BLOCK 3 - DATA COLLECTIONS

Strings

Quotes and escapes, indexing and slicing, why strings never change, and the string methods the exam uses.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 15 of 28

admin@sthithapragna.com

A string is an **immutable** sequence of characters. Every method that seems to change a string **returns a new one**.

Single or double quotes: the same string

```
1  a = 'single'
2  b = "double"
3  c = "It's fine"            # apostrophe inside double quotes
4  d = 'She said "hi"'        # double quotes inside single quotes
5  e = 'It\'s escaped'        # or escape the quote with a backslash
6  print(c, d, e)
```

3-4 To include one kind of quote, surround the string with the other kind.

5 Or put a backslash in front of the quote, so it no longer ends the string.

Output: It's fine She said "hi" It's escaped

A backslash gives the next character a special meaning

```
1  print("line1\nline2")       # \n is a newline: two lines of output
2  print("a\tb")               # \t is a tab
3  print("back\\slash")        # \\ is one backslash
4  print(len("\n"), len("a\\b"))   # 1 3
```

1-3 The backslash escapes from the normal meaning of the next character.

4 An escape sequence is written with two characters, but it is one character in the string.

A backslash at the very end of a string escapes the closing quote: "back\" is a SyntaxError.

Triple quotes keep the line breaks

```
1  text = """Line one
2  Line two"""
3  print(text)
4  print(len("""a
5  b"""))                  # 3: a, a newline, b
```

1-2 Three quotes, single or double, let a string run over several lines.

4-5 Each line break inside becomes a newline character, and counts towards len.

Indexing and slicing work exactly as for lists

```
1  s = "Python"
2  print(s[0], s[-1], s[1:4])     # P n yth
3  print(s[::-1], s[::2])         # nohtyP Pto
4  print(len(s), "th" in s, "TH" in s)   # 6 True False
5  print(s[6])                    # IndexError: string index out of range
```

2-3 Positive and negative indexes, slices and steps: all the list rules apply.

4 in finds a substring, not just one character. It is case-sensitive.

You cannot change a string. You build a new one

CHANGING A CHARACTER

Not allowed

```
1  s = "cat"
2  s[0] = "b"
```

TypeError: 'str' object does not support item assignment

BUILDING A NEW STRING

The way to do it

```
1  s = "cat"
2  s = "b" + s[1:]
3  print(s)
```

bat. A new string is built, and the name s is moved onto it.

ON THE EXAM

Assigning or deleting a character: TypeError. List methods like append: AttributeError.

Methods return a new string. Capture it

RESULT THROWN AWAY

s is unchanged

```
1  s = "Hello"
2  s.upper()
3  print(s)
```

Hello. upper() built HELLO, but nothing kept it.

RESULT CAPTURED

s now names the new string

```
1  s = "Hello"
2  s = s.upper()
3  print(s)
```

HELLO. The new string is assigned back to s.

THE TRAP

A string method called on a line by itself has no lasting effect.

Five case methods, all returning a new string

METHOD, ON "PYTHON IS FUN"	RESULT
`upper()`	PYTHON IS FUN
`lower()`	python is fun
`capitalize()`	Python is fun: first letter up, all the rest down
`title()`	Python Is Fun: every word starts with a capital
`swapcase()`	PyTHON IS FUN: every letter flipped

THE TRAP

capitalize() lowers everything after the first character, not just the first word.

Break a string into a list, and join a list into a string

```
1  print("a b  c".split())         # ['a', 'b', 'c']   any run of spaces
2  print("a,,b".split(","))       # ['a', '', 'b']    empty piece kept
3  print("-".join(["x", "y", "z"]))   # x-y-z
4  print("".join(["a", "b"]))       # ab
5  print("-".join([1, 2]))         # TypeError: expected str
```

1. With no argument, split cuts on any whitespace, and ignores runs of it.
2. With a separator, every separator cuts, so two in a row leave an empty string.
3-5. join is called on the separator, and every item joined must be a string.

find returns -1. index raises ValueError

FIND

-1 when missing

```
print("banana".find("na"))
print("banana".find("x"))
```

2, then -1. find never raises.

INDEX

ValueError when missing

```
print("banana".index("na"))
print("banana".index("x"))
```

2, then ValueError: substring not found.

THE TRAP

-1 is a valid index, the last character. Test find's result with == -1, never use it as an index.

count, replace, strip, startswith and endswith

```
1  s = "  banana  "
2  print(s.strip(), s.strip().count("a"))    # banana 3
3  print("banana".count("an", 2))           # 1   counting from index 2
4  print("banana".replace("a", "o"))        # bonono
5  print("banana".replace("a", "o", 1))     # bonana   first one only
6  print("xxhixx".strip("x"), "hi".endswith("I"))   # hi False
```

2 strip removes whitespace from both ends only. lstrip and rstrip do one end.

3 A second argument to count, find or index sets where the search starts.

5 A third argument to replace limits how many are replaced.

Tests that return True or False

CALL	RESULT	WHY
`"12".isdigit()`	True	every character is a digit
`"-12".isdigit()` `"1.5".isdigit()`	False	a sign or a point is not a digit
`"".isdigit()`	False	an empty string passes no test
`"abc".isalpha()` `"ab c".isalpha()`	True, then False	a space is not a letter
`"abc123".isalnum()`	True	letters and digits only
`"ABC1".isupper()`	True	only the letters are checked

> ON THE EXAM
>
> isdigit() cannot tell you whether int() will succeed: "-12" converts but fails isdigit().

ord() and chr()

```
1  print(ord("a"), ord("A"), ord("0"))     # 97 65 48
2  print(chr(97), chr(ord("p") + 3))       # a s
3  print(ord("c") - ord("a"))              # 2
4  print(chr(ord("g") - 32))               # G
```

2 Code point of p, plus 3, back to a character: p, q, r, s.

4 Each capital's code point is exactly 32 below its lower-case letter.

Which operators work

OPERATOR	EXAMPLE	RESULT
`+ +=`	"ab" + "cd"	abcd: concatenation
`*`	"ab" * 3	ababab: needs an int
`in not in`	"b" in "abc"	True: substring test
`== < >`	"a" < "b"	True: by code point, module 4
`- /`	"abc" - "c"	TypeError: unsupported operand

ON THE EXAM

len(s) gives the length. There is no s.size() or s.length(): those are AttributeErrors.

Is a slice of a string a copy?

- For a **list**, s[:] always builds a new, equal list. That is the model the exam uses.
- For a **string**, CPython skips the copy: str1[:] is str1 gives True, on 3.12 and 3.14.
- A string cannot change, so nothing except **is** can ever tell the difference.
- On the exam, answer with the model: str2 = str1[:] makes a different but equal string.

> ON THE EXAM
>
> The exam tests the idea that slicing copies. The shortcut is a CPython optimisation, not a language rule.

What to carry into the exam

- Strings are **immutable**. A method on a line by itself changes nothing.
- \n is one character. \\ is one backslash. Use the other quote, or escape it.
- split() cuts on whitespace; split(",") keeps empty pieces; join needs strings.
- **find gives -1; index raises ValueError.**
- capitalize lowers the rest. isdigit fails on "-1". chr(ord(c) + n) shifts letters.

What is the output of this program?

```
1  s = "abcdefg"
2  print(s[-3:], s[1:-1:2], s[::-3])
```

A efg bdf gda

B efg bdf gd

C efg ace gda

D efg bdf adg

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "abcdefg"
2  print(s[-3:], s[1:-1:2], s[::-3])
```

[1:-1:2]: indexes 1, 3, 5. [::-3]: indexes 6, 3, 0.

A `efg bdf gda`

Last three; indexes 1, 3, 5; indexes 6, 3, 0.

B `efg bdf gd`

Stops the backward slice at 3. With no stop given, it runs to the start.

C `efg ace gda`

Starts the second slice at 0. Its start is 1.

D `efg bdf adg`

Reverses the result of the third slice. A negative step already reads backwards.

THE SIGNAL

A negative step with no start begins at the last character, and with no stop runs to the first.

What is the output of this program?

```
1  word = "python"
2  word.upper()
3  word = word.replace("p", "j").capitalize()
4  print(word)
```

A PYTHON

B jython

C Jython

D JYTHON

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  word = "python"
2  word.upper()
3  word = word.replace("p", "j").capitalize()
4  print(word)
```

Line 2 is thrown away. "python" -> "jython" -> "Jython".

A PYTHON

Assumes line 2 changed word. Its result was never kept.

B jython

Forgets the capitalize() at the end of the chain.

C Jython

Line 2 is lost; replace then capitalize are chained and kept.

D JYTHON

Applies line 2 as well. It had no lasting effect.

THE SIGNAL

A string method on a line by itself changes nothing. Chained methods apply left to right.

What is the output of this program?

```
1  parts = "2024-09-26".split("-")
2  print("/".join(parts[::-1]), len(parts))
```

A 2024/09/26 3

B 26/09/2024 3

C 26/09/2024 10

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  parts = "2024-09-26".split("-")
2  print("/".join(parts[::-1]), len(parts))
```

parts is ['2024', '09', '26']. Reversed: ['26', '09', '2024'].

A `2024/09/26 3`
Ignores the [::-1], which reverses the list of parts.

B `26/09/2024 3`
Split into 3 strings, reverse the list, join with /.

C `26/09/2024 10`
Counts characters. parts is a list of 3 pieces.

D The code raises a TypeError
The pieces are strings, so join accepts them.

THE SIGNAL

split gives a list of strings, so join can always put them back together.

What is the output of this program?

```
s = "mississippi"
print(s.find("ss"), s.index("ss", 3),
      s.count("s"), s.find("x"))
```

A 2 5 2 -1

B 2 2 4 -1

C 2 5 4 -1

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
s = "mississippi"
print(s.find("ss"), s.index("ss", 3),
      s.count("s"), s.find("x"))
```

m-i-s-s-i-s-s-i-p-p-i: "ss" starts at 2 and at 5. Four single s characters.

A `2 5 2 -1`

Counts "ss". The count is of a single "s": 4.

B `2 2 4 -1`

Ignores the start of 3, so finds the first "ss" again.

C `2 5 4 -1`

First ss at 2; from 3 the next is at 5; four s; x missing gives -1.

D The code raises a ValueError

find returns -1 for a missing substring. Only index raises.

THE SIGNAL

find and index agree when there is a match. Only a missing substring separates them.

What is the output of this program?

```
1  code = ""
2  for ch in "abc":
3      code += chr(ord(ch) + 1)
4  print(code.upper())
```

A bcd

B ABC

C The code raises a TypeError

D BCD

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  code = ""
2  for ch in "abc":
3      code += chr(ord(ch) + 1)
4  print(code.upper())
```

a -> b, b -> c, c -> d. code is "bcd", printed upper-cased.

A **bcd**
Forgets the upper() in the print.

B **ABC**
Forgets the + 1 shift.

C **The code raises a TypeError**
ord gives an int, + 1 is int arithmetic, and chr turns it back.

D **BCD**
Each letter shifts one on, then upper() capitalises the result.

THE SIGNAL

ord to a number, arithmetic, chr back to a character: the types line up at each step.

> The goal is a string whose text is: It's "odd"

Which two literals produce exactly that text? (Choose two.)

A `'It's "odd"'`

B `"It's \"odd\""`

C `'It\'s "odd"'`

D `"It's "odd""`

◆ PAUSE AND DECIDE

Which two literals produce exactly that text? (Choose two.)

A `'It's "odd"'`

The apostrophe ends the string after It. SyntaxError.

B `"It's \"odd\""`

Double quotes outside; the apostrophe is fine; inner quotes escaped.

C `'It\'s "odd"'`

Single quotes outside; the apostrophe escaped; double quotes are fine.

D `"It's "odd""`

The second double quote ends the string before odd. SyntaxError.

THE SIGNAL

Only the quote character that surrounds the string needs escaping inside it.

What is the output of this program?

```
for s in ["12", "-3", "4.5", "x9"]:
    print(s.isdigit(), end=" ")
```

A True True True False

B True False False False

C True True False False

D False False False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for s in ["12", "-3", "4.5", "x9"]:
2      print(s.isdigit(), end=" ")
```

isdigit checks every character. -, . and x are not digits.

A `True True True False`

Treats anything number-like as digits. isdigit checks characters.

B `True False False False`

Only "12" is all digits. The -, the . and the x each fail.

C `True True False False`

Accepts the minus sign. It is not a digit.

D `False False False False`

"12" is made entirely of digits.

THE SIGNAL

isdigit asks whether every character is a digit, not whether the text is a number.

What is the output of this program?

```
1  s = "a\tb\\n"
2  print(len(s))
```

A 7

B 6

C 4

D 5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "a\tb\\n"
2  print(len(s))
```

Characters: a, tab, b, one backslash, n.

A 7

Counts every typed character. Each escape pair is one character.

B 6

Treats \\n as a backslash plus a newline. \\ ends the escape; n is a letter.

C 4

Treats \\n as one newline. The \\ is used up first.

D 5

a, \t, b, \\ and n: five characters.

THE SIGNAL

Read escapes left to right: each backslash takes exactly the next character with it.

MODULE 16 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Functions I: def, return and None

Defining and calling functions, return versus print, the None every function can give back, and two useful modules.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 16 of 28

admin@sthithapragna.com

A function gives back exactly what its **return** statement hands it. With no return, it gives back **None**. Printing is not returning.

def creates the function. Calling it runs the body

```
1  def greet(name):              # header: def, name, parameters, colon
2      return "Hello, " + name    # body: indented
3
4  message = greet("Ann")        # the call runs the body
5  print(message)                # Hello, Ann
```

1 def, a name, brackets holding the parameters, and a colon. The brackets are required even when empty.

4 Calling the function runs its body, and the call is replaced by the returned value.

def fun: is a SyntaxError, expected '('. A parameterless function is written def fun():

The def must run before the call

```
1  def a():
2      return b() + 1          # b is only looked up when a runs
3
4  def b():
5      return 5
6
7  print(a())                  # 6: b exists by the time a is called
```

2 A name inside a body is looked up when the body runs, not when the def runs.

7 By this line, both defs have run, so both functions exist.

Calling a function above its def is a NameError: print(hello()) followed by def hello() fails.

return hands a value back. print only shows it

PRINT

Shows text; gives back None

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

6, then None. show prints 6, but returns nothing, so y is None.

RETURN

Gives back the value

```
1  def calc(x):
2      return x * 2
3
4  y = calc(3)
5  print(y)
```

6, once. calc prints nothing; the value comes back and is stored in y.

THE TRAP

print(func()) prints whatever func prints, then func's return value. For a printing function, that ends in None.

Three ways to return None

```
1  def nothing():
2      pass                    # no return at all
3  def bare():
4      return                  # return with no value
5  def maybe(n):
6      if n > 10:
7          return "big"        # only some paths return
8  print(nothing(), bare(), maybe(5))    # None None None
```

2 Falling off the end of the body returns None.

4 A bare return ends the function and returns None.

5-7 If a path through the function never reaches a return, that call gives None.

Nothing after a return runs

```
1  def f():
2      return 1
3      print("never")          # unreachable
4  def first_even(nums):
5      for n in nums:
6          if n % 2 == 0:
7              return n        # leaves the loop and the function
8      return -1
9  print(f(), first_even([3, 8, 6]), first_even([1, 5]))   # 1 8 -1
```

3 Unreachable code is legal, but it never runs.

7 return inside a loop ends the loop and the function together.

An expression on its own line changes nothing

```
1  def grow(n):
2      n * 10                    # worked out, then thrown away
3      return n + 1
4
5  print(grow(grow(2)))          # grow(2) is 3, grow(3) is 4
```

2 Nothing assigns the result, so n is unchanged. It would need n = n * 10.

5 Nested calls run from the inside out.

Functions calling functions, and returning several values

```
1  def double(x): return x * 2
2  def add3(x): return x + 3
3  print(double(add3(1)), add3(double(1)))   # 8 5
4  def pair():
5      return 1, 2              # packed into one tuple
6  print(pair(), type(pair()))  # (1, 2) <class 'tuple'>
```

1-2 A one-line body can follow the colon directly, just as with if.

3 Inside out: add3(1) is 4, doubled is 8. double(1) is 2, plus 3 is 5.

5 return 1, 2 returns one tuple. The caller can unpack it: a, b = pair().

What goes wrong with functions

CODE	ERROR
`print(hello()) above def hello():`	NameError: name 'hello' is not defined
`print(func4(1)) when only func1 exists`	NameError. Did you mean: 'func1'?
`def fun:`	SyntaxError: expected '('
`def 1function():`	SyntaxError: a name cannot start with a digit
`def f(a, b): ... f(1)`	TypeError: missing 1 required positional argument: 'b'
`def f(): ... f(5)`	TypeError: f() takes 0 positional arguments but 1 was given

THE TIEBREAKER

A bad definition fails before anything runs. A bad call fails only when that line is reached.

Shadowing a built-in function

SHADOWED

list now names your list

```
1  list = [1, 2]
2  print(list("ab"))
```

TypeError: 'list' object is not callable. The built-in is hidden.

NOT SHADOWED

list is still the built-in

```
1  items = [1, 2]
2  print(list("ab"))
```

['a', 'b']. Choose a name that does not hide a built-in.

ON THE EXAM

Built-in functions such as print, len, list and input are names, not keywords. Assigning to them hides them.

import math

```
1  import math
2  print(math.sqrt(16), math.pi)          # 4.0 3.141592653589793
3  print(math.floor(-2.5), math.ceil(2.1))   # -3 3
4  print(math.pow(2, 3))                  # 8.0: always a float
5  from math import sqrt
6  print(sqrt(25))                        # 5.0: no math. prefix needed
```

1-4 import math brings in the module. Its contents are used as math.name.

3 floor rounds down, ceil rounds up. Both return int.

5-6 from math import sqrt copies just that name in, so it is used bare.

sqrt(4) without from math import sqrt is a NameError. math.sqrt(-1) is a ValueError.

import random

```
1  import random
2  n = random.randint(1, 6)       # 1 to 6, both ends included
3  x = random.random()            # a float from 0.0 up to, not including, 1.0
4  c = random.choice(["a", "b"])  # one element of the sequence
5  import math as m
6  print(m.sqrt(9))               # 3.0: an alias for the module
```

2 randint includes both ends, unlike range. randint(1, 6) can return 6.

5 import ... as gives the module a shorter name.

Python 3.14 reworded one maths error

- Python 3.12: math.sqrt(-1) says **ValueError: math domain error**.
- Python 3.14: **ValueError: expected a nonnegative input, got -1.0**.
- The exception is ValueError in both. Answer by the exception name.

What to carry into the exam

- A call is replaced by the **returned value**. No return, or a bare return: **None**.
- **print is not return.** print(func()) shows func's printing, then its return value.
- return ends the function at once, loops included.
- The def must run before the call; names in a body are looked up when it runs.
- n * 10 on its own line changes nothing. randint includes both ends.

What is the output of this program?

```
1  def check(n):
2      if n > 10:
3          return "big"
4
5  result = check(5)
6  print(result, check(20))
```

A `None big`

B `big big`

C The code raises a TypeError

D `small big`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def check(n):
2      if n > 10:
3          return "big"
4
5  result = check(5)
6  print(result, check(20))
```

check(5) never reaches a return, so it gives None.

A `None big`

5 skips the only return, so None; 20 returns "big".

B `big big`

5 > 10 is False, so the return is never reached for 5.

C The code raises a TypeError

Missing a return path is legal. It just gives None.

D `small big`

There is no "small" anywhere. Nothing makes one up.

THE SIGNAL

If some path through a function has no return, that call gives None.

What is the output of this program?

```
1  def first_even(nums):
2      for n in nums:
3          if n % 2 == 0:
4              return n
5      return -1
6
7  print(first_even([3, 8, 6]), first_even([1, 5]))
```

A. 6 -1

B. 8 None

C. 8 -1

D. [8, 6] -1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def first_even(nums):
2      for n in nums:
3          if n % 2 == 0:
4              return n
5      return -1
6
7  print(first_even([3, 8, 6]), first_even([1, 5]))
```

8 is the first even number, and return ends the loop there. [1, 5] has none.

A `6 -1`
Keeps looping after 8. return ends the function at once.

B `8 None`
Misses the final return -1 after the loop.

C `8 -1`
First even is returned; no even at all reaches return -1.

D `[8, 6] -1`
return hands back one value and stops. It does not collect.

THE SIGNAL

return inside a loop ends the loop and the function together, at the first match.

What is the output of this program?

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

A 6, then 6

B None, then 6

C 6 only

D 6, then None

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(x):
2      print(x * 2)
3
4  y = show(3)
5  print(y)
```

Line 4 runs show, which prints 6. show returns None, so y is None.

A 6, then 6

Treats print as return. show gives back nothing.

B None, then 6

Reverses the order. The call on line 4 prints first.

C 6 only

Forgets that print(y) runs, and shows None.

D 6, then None

show prints 6 during the call, then returns None into y.

THE SIGNAL

Write down what a function prints, then what it returns, separately.

What is the output of this program?

```
1  def grow(n):
2      n * 10
3      return n + 1
4
5  print(grow(grow(2)))
```

A 211

B 31

C The code raises a SyntaxError

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grow(n):
2      n * 10
3      return n + 1
4
5  print(grow(grow(2)))
```

n * 10 is thrown away. grow(2) is 3, then grow(3) is 4.

A **211**

*Treats n * 10 as updating n twice. Nothing is assigned.*

B **31**

*Applies the * 10 once. It never applies.*

C **The code raises a SyntaxError**

An expression on its own line is legal; it is simply discarded.

D **4**

Only return n + 1 has an effect: 2 to 3, then 3 to 4.

THE SIGNAL

A calculation that is not assigned, returned or printed has no effect.

What is the output of this program?

```
1  def a():
2      return b() + 1
3
4  def b():
5      return 5
6
7  print(a())
```

A The code raises a NameError

B 6

C 5

D None

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def a():
2      return b() + 1
3
4  def b():
5      return 5
6
7  print(a())
```

Both defs have run by line 7. b is looked up only when a's body runs.

A The code raises a NameError

b is defined before a is called. The body's names are looked up then.

B 6

a calls b, which returns 5, and adds 1.

C 5

Forgets the + 1 in a.

D None

Both functions return values.

THE SIGNAL

A def must run before a call to it runs. The order of defs among themselves does not matter.

> Consider Python functions in general.

Which two statements about functions are true? (Choose two.)

A. A function with no return statement returns None

B. return must be the last line of a function body

C. A function must be defined before it is called

D. print() inside a function sends a value back to the caller

◆ PAUSE AND DECIDE

Which two statements about functions are true? (Choose two.)

A A function with no return statement returns None

Falling off the end gives None.

B return must be the last line of a function body

return can appear anywhere, several times; later lines are just unreachable.

C A function must be defined before it is called

The def must have run before the call runs, or it is a NameError.

D print() inside a function sends a value back to the caller

print only shows text. return sends a value back.

THE SIGNAL

None when nothing is returned; def before the call; print shows, return gives back.

MODULE 17 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Functions II: passing arguments

Parameters and arguments, positional and keyword passing, default values, and what a function can change in the caller's data.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 17 of 28

admin@sthithapragna.com

Passing an argument works exactly like **parameter = argument**. The function gets another name for the same object, so it can change a list, but never the caller's variable.

Parameters are in the def. Arguments are in the call

```
1  def subtract(a, b):          # a and b are parameters
2      return a - b
3
4  print(subtract(5, 2))        # 5 and 2 are arguments: 3
```

1 Parameters are names in the header. They exist only while the function runs.

4 Arguments are the values supplied in the call.

By position, by name, or both

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(5, 2))          # positional: a=5, b=2 gives 3
4  print(subtract(b=5, a=2))      # keyword: a=2, b=5 gives -3
5  print(subtract(5, b=2))        # mixed: 3
```

3 Positional: the order of the arguments decides which parameter each one fills.

4 Keyword: the name decides. The order of keyword arguments does not matter.

5 Mixed: positional arguments first, then keyword arguments.

Built-ins use keyword arguments too: print(1, 2, sep="-") and sorted(x, reverse=True).

Positional arguments must come before keyword arguments

KEYWORD FIRST

Never runs

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(a=5, 2))
```

SyntaxError: positional argument follows keyword argument.

POSITIONAL FIRST

Works

```
1  def subtract(a, b):
2      return a - b
3  print(subtract(5, b=2))
```

3. The positional 5 fills a; the keyword fills b.

THE TRAP

This is a SyntaxError, so the whole file fails before line 1 runs, not just this call.

Argument errors are TypeErrors

CALL, WITH DEF SUBTRACT(A, B)	TYPEERROR MESSAGE
`subtract(5)`	missing 1 required positional argument: 'b'
`subtract(1, 2, 3)`	takes 2 positional arguments but 3 were given
`subtract(5, a=2)`	got multiple values for argument 'a'
`subtract(5, c=1)`	got an unexpected keyword argument 'c'

THE TIEBREAKER

Wrong count or wrong name: TypeError when the call runs. Wrong order: SyntaxError before anything runs.

A default fills in when the argument is left out

```
1  def power(base, exp=2):
2      return base ** exp
3  print(power(3))                 # 9:  exp uses its default
4  print(power(3, 3))              # 27: the default is replaced
5  print(power(exp=0, base=7))     # 1:  keywords, any order
```

1 exp=2 in the header gives exp a default value. base has none, so it is required.

3-4 Leave the argument out and the default is used; supply one and it wins.

Parameters with defaults must come last

DEFAULT FIRST

Never runs

```
1  def f(a=1, b):
2      return a + b
```

SyntaxError: parameter without a default follows parameter with a default.

DEFAULT LAST

Works

```
1  def f(b, a=1):
2      return a + b
3  print(f(5))
```

6. b is required; a is optional.

ON THE EXAM

The same shape as the call rule: required things first, optional things after.

Defaults are evaluated once, when def runs

```
1  n = 5
2  def show(x=n):          # the default is taken now: 5
3      return x
4  n = 10
5  print(show())           # 5, not 10
```

2 The default expression runs once, when the def statement runs, and the value is stored.

4-5 Changing n later does not change the stored default.

A list as a default is shared between calls

```
1  def add_item(item, box=[]):
2      box.append(item)
3      return box
4  print(add_item(1))      # [1]
5  print(add_item(2))      # [1, 2]: the same list again
```

1 One empty list is created, once, when def runs. Every call without box gets that same list.

5 The second call appends to the list the first call already changed.

The usual fix: box=None, then inside: if box is None: box = [].

Changing the object versus rebinding the parameter

CHANGE THE LIST

The caller sees it

```
1  def fill(items):
2      items.append(9)
3
4  data = [1]
5  fill(data)
6  print(data)
```

[1, 9]. items and data are two names for one list.

REBIND THE PARAMETER

The caller does not

```
1  def replace(items):
2      items = [9]
3
4  data = [1]
5  replace(data)
6  print(data)
```

[1]. items is moved to a new list; data is untouched.

THE TIEBREAKER

Methods, index assignment, del and += change the caller's list. Plain assignment to the parameter does not.

An immutable argument can never be changed from inside

```
1  def inc(n):
2      n += 1                  # makes a new int; rebinds n
3      return n
4  x = 5
5  print(inc(x), x)            # 6 5
6  def shout(s):
7      return s.upper()        # a new string
8  word = "hi"
9  print(shout(word), word)    # HI hi
```

2 An int cannot change, so += builds a new int and moves n. x keeps 5.

5 To get the new value out, the function must return it, and the caller must use it.

+= and del on a list parameter

```
1  def grow(items):
2      items += [0]              # extends the caller's list
3      del items[0]              # removes from the caller's list
4  data = [1, 2]
5  grow(data)
6  print(data)                  # [2, 0]
```

2 For a list, += changes the list in place, so the caller sees it. Module 12's rule.

3 del on an element also changes the shared list.

On a string argument, del text[2] is a TypeError: strings do not support item deletion.

What to carry into the exam

- Positional arguments first, then keyword. Otherwise: **SyntaxError**.
- Wrong count, a repeated parameter, or an unknown keyword: **TypeError** at run time.
- Defaults go last in the def, and are evaluated **once**, when def runs.
- A default list is **shared** between calls.
- A function can change a list it is given, but never the caller's variable, number or string.

What is the output of this program?

```
1  def calc(x, y, z):
2      return x * 10 + y - z
3
4  print(calc(1, z=2, y=5))
```

A 13

B 10

C 11

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def calc(x, y, z):
2      return x * 10 + y - z
3
4  print(calc(1, z=2, y=5))
```

x = 1 by position; z = 2 and y = 5 by name. 10 + 5 - 2 = 13.

A. 13
Keywords fill by name, whatever their order.

B. 10
Fills y and z in the order written: y = 2, z = 5.

C. 11
Drops the y, or adds z instead of subtracting it.

D. The code raises a SyntaxError
The positional argument comes first, so the order is legal.

THE SIGNAL

Keyword arguments are matched by name. The order in which they are written does not matter.

What happens when this program runs?

```
1  def area(w, h):
2      return w * h
3
4  print("start")
5  print(area(w=3, 4))
```

A start, then 12

B start, then 7

C start, then a TypeError

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def area(w, h):
2      return w * h
3
4  print("start")
5  print(area(w=3, 4))
```

SyntaxError: positional argument follows keyword argument. Found before line 1 runs.

A start, then 12
The call is never reached. The file does not compile.

B start, then 7
** is multiplication, and the call never runs anyway.*

C start, then a TypeError
Wrong order is a SyntaxError, found before anything runs.

D Nothing is printed; the program raises a SyntaxError
A positional after a keyword is a grammar error, so start never prints.

THE SIGNAL

Argument order errors are SyntaxErrors. Argument count errors are TypeErrors.

What is the output of this program?

```
1  def price(cost, tax=20, discount=0):
2      return cost + cost * tax // 100 - discount
3
4  print(price(100, discount=5), price(100, 10))
```

A 105 110

B 115 120

C 115 110

D 95 110

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def price(cost, tax=20, discount=0):
2      return cost + cost * tax // 100 - discount
3
4  print(price(100, discount=5), price(100, 10))
```

First: tax 20 (default), discount 5: 100 + 20 - 5 = 115. Second: tax 10, discount 0: 110.

A `105 110`

Puts the 5 into tax. discount=5 names its parameter.

B `115 120`

Keeps tax at 20 in the second call. The positional 10 replaces it.

C `115 110`

Defaults fill the gaps; the positional 10 goes to tax, the second parameter.

D `95 110`

Forgets the default tax of 20 in the first call.

THE SIGNAL

A keyword argument skips over earlier defaults. A second positional argument fills the second parameter.

What is the output of this program?

```
1  def change(a, b):
2      a.append(3)
3      b = [0]
4
5  x = [1]
6  y = [2]
7  change(x, y)
8  print(x, y)
```

A `[1, 3] [0]`

B `[1, 3] [2]`

C `[1] [2]`

D `[1] [0]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def change(a, b):
2      a.append(3)
3      b = [0]
4
5  x = [1]
6  y = [2]
7  change(x, y)
8  print(x, y)
```

a shares x's list and changes it. b is rebound; y is untouched.

A `[1, 3] [0]`
Treats b = [0] as changing y. It only moves the name b.

B `[1, 3] [2]`
append changes the shared list; assignment rebinds only b.

C `[1] [2]`
Assumes a function works on copies. It gets the same objects.

D `[1] [0]`
Reverses both effects.

THE SIGNAL

Inside a function: change the object and the caller sees it; rebind the parameter and the caller does not.

What is the output of this program?

```
1  def bump(n):
2      n += 5
3      return n
4
5  n = 1
6  bump(n)
7  print(n, bump(n))
```

A 6 6

B 1 1

C 6 11

D 1 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def bump(n):
2      n += 5
3      return n
4
5  n = 1
6  bump(n)
7  print(n, bump(n))
```

The parameter n is not the outer n. Line 6's result is thrown away.

A 6 6

Assumes line 6 changed the outer n. Ints cannot be changed from inside.

B 1 1

bump does return 6; the second value is that return.

C 6 11

Assumes both calls changed the outer n.

D 1 6

The outer n stays 1; bump(1) returns 6 again.

THE SIGNAL

Same name inside and outside does not mean the same variable. An int argument cannot be changed.

What is the output of this program?

```
1  limit = 3
2  def cap(x, top=limit):
3      return min(x, top)
4
5  limit = 100
6  print(cap(50))
```

A 3

B 50

C 100

D The code raises a NameError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  limit = 3
2  def cap(x, top=limit):
3      return min(x, top)
4
5  limit = 100
6  print(cap(50))
```

top's default was stored as 3 when def ran. min(50, 3) is 3.

A **3**

The default was taken when def ran, while limit was 3.

B **50**

Would need top to be 100 or more.

C **100**

Assumes the default is looked up on each call.

D **The code raises a NameError**

limit exists when def runs, so the default is fine.

THE SIGNAL

A default is evaluated once, when def runs. Later changes to the names it used have no effect.

What is the output of this program?

```
1  def log(msg, history=[]):
2      history.append(msg)
3      return len(history)
4
5  log("a")
6  print(log("b"), log("c", []))
```

A 1 1

B 2 1

C 2 3

D 3 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def log(msg, history=[]):
2      history.append(msg)
3      return len(history)
4
5  log("a")
6  print(log("b"), log("c", []))
```

The default list holds "a", then "a", "b". The call with [] gets its own fresh list.

A 1 1

Assumes a fresh default list on every call. It is created once.

B 2 1

a and b share the default list; c gets the list passed in.

C 2 3

Adds c to the shared list. An explicit argument replaces the default.

D 3 1

Counts c in the shared list too.

THE SIGNAL

Calls that rely on a default list share it. A call that passes its own list does not touch it.

> def f(a, b=1): pass has been defined. Each call is run on its own.

Which two calls raise a TypeError? (Choose two.)

A `f(1)`

B `f(b=2, a=1)`

C `f(1, 2, 3)`

D `f(1, a=2)`

◆ PAUSE AND DECIDE

Which two calls raise a TypeError? (Choose two.)

A `f(1)`

a is 1 and b uses its default. Fine.

B `f(b=2, a=1)`

Both by name, any order. Fine.

C `f(1, 2, 3)`

Takes from 1 to 2 positional arguments but 3 were given.

D `f(1, a=2)`

1 fills a by position, then a=2 fills it again: multiple values.

THE SIGNAL

Count the arguments against the parameters, and check no parameter is filled twice.

MODULE 18 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Scope and the global keyword

Where a name exists, why assigning inside a function makes a new local, the UnboundLocalError trap, and what global changes.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 18 of 28

admin@sthithapragna.com

If a function **assigns** to a name anywhere in its body, that name is local for the **whole** function. Reading an outside name is fine. Assigning to one needs global.

A function can read a global variable

```
1  x = 1
2  def show():
3      print(x + 4)          # x is only read, so the global x is used
4  show()                    # 5
5  print(x)                  # 1
```

1 A name assigned outside every function is global to the file.

3 show never assigns x, so it looks x up in the global scope.

Assignment creates a new local that hides the global

```
1  num = 1
2  def f():
3      num = 3                # a new local num
4      print(num, end=" ")
5  f()
6  print(num)                 # output: 3 1
```

3 Assigning inside f creates a local num. The global num is not touched.

6 Outside, the global num is still 1.

Locals and parameters exist only during the call

```
1  def f(a):
2      y = 10
3  f(1)
4  print(y)                    # NameError: name 'y' is not defined
```

1-2 Both the parameter a and the variable y are local to f.

4 Once f returns, its local names are gone. print(a) would fail the same way.

Reading a name before its local assignment

```
1  num = 1
2  def f():
3      num = num + 3          # UnboundLocalError
4  f()
```

3 num is assigned in f, so num is local for all of f. The right-hand side reads the local, which has no value yet.

Python 3.11 onwards: cannot access local variable 'num' where it is not associated with a value. Older: local variable 'num' referenced before assignment.

The assignment does not have to come first

READ ONLY

Uses the global

```
1  x = 5
2  def f():
3      print(x)
4  f()
```

5. f never assigns x, so x is global.

READ, THEN ASSIGN

Fails at the print

```
1  x = 5
2  def f():
3      print(x)
4      x = 2
5  f()
```

UnboundLocalError, raised at print(x). The later x = 2 makes x local everywhere in f.

THE TIEBREAKER

Python decides local or global by scanning the whole body when def runs, not line by line.

global lets a function assign to the outside name

```
1  count = 0
2  def tick():
3      global count            # count here means the global count
4      count += 1
5  tick()
6  tick()
7  print(count)               # 2
```

3 Declares that count in this function refers to the global name. No local is created.

4 Now reading and assigning both use the global.

A global declared in a function can be created there

```
1  def setup():
2      global mode
3      mode = "fast"           # creates the global mode
4  setup()
5  print(mode)               # fast
```

2-3 mode did not exist before. The assignment creates it in the global scope.

5 print(mode) before setup() runs would be a NameError.

Changing a global list needs no global keyword

CHANGE THE OBJECT

Works without global

```
1  data = [1]
2  def add():
3      data.append(2)
4  add()
5  print(data)
```

[1, 2]. data is only read, to find the list; then the list is changed.

ASSIGN THE NAME

Creates a local

```
1  data = [1]
2  def reset():
3      data = []
4  reset()
5  print(data)
```

[1]. data = [] makes a new local. The global list is untouched.

THE TIEBREAKER

global is about assigning to a name. Calling a method, or setting data[0], is not assigning to data.

The test: is the name itself assigned?

INSIDE THE FUNCTION	NEEDS GLOBAL?	WHY
`print(data)`	no	only reads the name
`data.append(2)`	no	reads the name, changes the object
`data[0] = 9`	no	assigns to an element, not to the name
`data = [9]`	yes	assigns to the name: otherwise a local
`data += [2]`	yes	+= is an assignment: UnboundLocalError

THE TRAP

The last row surprises people: for a list, += changes it in place, but it still counts as assigning the name.

The lookup order: local, then global, then built-in

SCOPE	WHAT LIVES THERE	EXAMPLE
Local	names assigned in the current function, and its parameters	y = 10 inside f
Enclosing	names in an outer function, for nested functions	beyond the syllabus
Global	names assigned at the top level of the file	count = 0
Built-in	names Python provides	len, print, range

ON THE EXAM

The first scope that has the name wins. That is why a local or global named len hides the built-in len.

if and for blocks do not create a scope

```
1  if True:
2      inside = 1
3  for i in range(3):
4      pass
5  print(inside, i)          # 1 2: both are global names
```

1-4 Names assigned in if, while and for blocks belong to the surrounding scope.

5 At the top level of a file, that scope is global.

What goes wrong

CODE	ERROR
`reading a local, like y, after the function has returned`	NameError
`x = x + 1 inside a function, with no global`	UnboundLocalError
`print(x) then global x, in the same function`	SyntaxError: used prior to global declaration
`def f(x): global x`	SyntaxError: name 'x' is parameter and global

ON THE EXAM

UnboundLocalError is a kind of NameError. Module 20 places both in the exception hierarchy.

What to carry into the exam

- Assigned anywhere in a function: **local for the whole function**.
- Only read: the global is used. Locals and parameters vanish after the call.
- Read before the local assignment: **UnboundLocalError**. data += [2] counts as assigning.
- global lets a function assign, or even create, the outside name.
- append, data[0] = … and other changes to a global object need no global.

What is the output of this program?

```
1  level = 1
2  def up():
3      level = 5
4      print(level, end=" ")
5
6  up()
7  print(level)
```

A 5 1

B 5 5

C 1 1

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  level = 1
2  def up():
3      level = 5
4      print(level, end=" ")
5
6  up()
7  print(level)
```

level = 5 creates a local inside up. The global level stays 1.

A. 5 1

The local prints 5; the global, untouched, prints 1.

B. 5 5

Treats the assignment as changing the global. That needs global.

C. 1 1

Inside up, the local level hides the global.

D. The code raises an UnboundLocalError

level is assigned before it is read, so the local has a value.

THE SIGNAL

Assigning inside a function without global makes a separate local variable.

What happens when this program runs?

```
1  total = 10
2  def add(n):
3      total = total + n
4      return total
5
6  print(add(5))
```

A 15 is printed

B The code raises an UnboundLocalError

C 10 is printed

D 5 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  total = 10
2  def add(n):
3      total = total + n
4      return total
5
6  print(add(5))
```

total is assigned in add, so it is local throughout add. The right side reads it before it has a value.

A. 15 is printed
What the author meant. It would need global total.

B. The code raises an UnboundLocalError
The local total is read on the right before it is assigned.

C. 10 is printed
The function never gets as far as returning.

D. 5 is printed
Treats the local total as starting at 0. It has no value at all.

THE SIGNAL

x = x + something inside a function, with no global, is always an UnboundLocalError.

What is the output of this program?

```
1  score = 0
2  def win():
3      global score
4      score += 3
5  def lose():
6      score = -1
7
8  win()
9  lose()
10 win()
11 print(score)
```

A 6

B -1

C 5

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  score = 0
2  def win():
3      global score
4      score += 3
5  def lose():
6      score = -1
7
8  win()
9  lose()
10 win()
11 print(score)
```

win changes the global twice: 0, 3, 6. lose only sets its own local.

A 6

lose assigns a local, so only the two wins count.

B -1

Treats lose as changing the global. It has no global line.

C 5

Applies lose's -1 to the global between the wins.

D 3

Counts only one win.

THE SIGNAL

global applies only in the function that declares it. Every other function follows the normal rule.

What is the output of this program?

```
1  items = []
2  def add(x):
3      items.append(x)
4      return len(items)
5
6  add("a")
7  print(add("b"), items)
```

A `1 ['b']`

B `2 ['a', 'b']`

C The code raises an UnboundLocalError

D `2 []`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  items = []
2  def add(x):
3      items.append(x)
4      return len(items)
5
6  add("a")
7  print(add("b"), items)
```

items is never assigned inside add, so it is the global list, and append changes it.

A `1 ['b']`
Assumes each call starts with a fresh list. The global list persists.

B `2 ['a', 'b']`
append changes the one global list; no assignment, so no global needed.

C The code raises an UnboundLocalError
There is no assignment to items, so it is not local.

D `2 []`
The changes are made to the global list itself.

THE SIGNAL

Calling a method on a global object is reading the name, not assigning it.

What happens when this program runs?

```
1  data = [1]
2  def grow():
3      data += [2]
4
5  grow()
6  print(data)
```

A [1, 2] is printed

B [1] is printed

C The code raises an UnboundLocalError

D [2] is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  data = [1]
2  def grow():
3      data += [2]
4
5  grow()
6  print(data)
```

+= is an assignment to data, so data is local in grow, and it is read before it has a value.

A [1, 2] is printed
Would be right with global data, or with data.append(2).

B [1] is printed
The function does not quietly fail. It raises.

C The code raises an UnboundLocalError
+= assigns the name, making it local; reading it first fails.

D [2] is printed
Treats the local data as an empty list. It has no value.

THE SIGNAL

For scope, += is an assignment, even when it would change a list in place.

> Consider scope in Python functions in general.

Which two statements are true? (Choose two.)

A. A name assigned inside a function is local unless declared global

B. A function cannot read a global variable without the global keyword

C. A function's parameters can still be used after it returns

D. Changing an element of a global list inside a function needs no global keyword

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A A name assigned inside a function is local unless declared global

The rule of this module.

B A function cannot read a global variable without the global keyword

Reading needs no keyword; only assigning does.

C A function's parameters can still be used after it returns

Parameters are local, and vanish with the call.

D Changing an element of a global list inside a function needs no global keyword

data[0] = x assigns an element, not the name.

THE SIGNAL

global is needed only to assign the name itself. Reading, and changing the object, never need it.

MODULE 19 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Recursion and generators

Functions that call themselves, how to trace them, when they never stop, and functions that yield values one at a time.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 19 of 28

admin@sthithapragna.com

A recursive function calls itself on a smaller problem, and must reach a **base case** that stops. A generator **yields** values one at a time, only when asked.

A base case, and a call on a smaller problem

```
1  def fact(n):
2      if n <= 1:              # base case: answer directly
3          return 1
4      return n * fact(n - 1) # recursive case: a smaller problem
5
6  print(fact(4))             # 24
```

2-3 The base case needs no further call. Without it, the calls never end.

4 Each call passes a smaller n, moving towards the base case.

Calls stack up like plates, then unwind

CALL	IS WAITING TO WORK OUT	RETURNS
`fact(4)`	4 * fact(3)	4 * 6 = 24
`fact(3)`	3 * fact(2)	3 * 2 = 6
`fact(2)`	2 * fact(1)	2 * 1 = 2
`fact(1)`	base case	1

EXAM TECHNIQUE

Read the middle column downwards as the calls stack up, then the right column upwards as they return.

A wrong base case poisons every result

```
1  def total(x):
2      if x == 0:
3          return 0            # right for adding
4      return x + total(x - 1)
5  def product(x):
6      if x == 0:
7          return 0            # wrong for multiplying
8      return x * product(x - 1)
9  print(total(3), product(3))   # 6 0
```

3 3 + 2 + 1 + 0 is 6. Adding 0 at the end changes nothing.

7 3 * 2 * 1 * 0 is 0. Multiplying by 0 at the bottom wipes out everything.

Before the call, or after it

PRINT, THEN RECURSE

Counts down

```
1  def down(n):
2      if n == 0:
3          return
4      print(n, end=" ")
5      down(n - 1)
6  down(3)
```

3 2 1. Each call prints before going deeper.

RECURSE, THEN PRINT

Counts up

```
1  def up(n):
2      if n == 0:
3          return
4      up(n - 1)
5      print(n, end=" ")
6  up(3)
```

1 2 3. Each call prints only after the deeper calls finish.

THE TIEBREAKER

Work done before the recursive call happens on the way down. Work after it happens on the way back up.

No base case, or one that is never reached

```
1  def forever(n):
2      return forever(n + 1)      # no base case at all
3
4  forever(0)
5  # RecursionError: maximum recursion depth exceeded
```

2 Every call makes another. Python stops the program at about 1000 calls deep.

A base case that is skipped does the same: if n == 0 with n going 5, 3, 1, -1 … is never reached.

Recursion with more than one branch

```
1  def fib(n):
2      if n < 2:
3          return n
4      return fib(n - 1) + fib(n - 2)
5
6  print([fib(i) for i in range(7)])   # [0, 1, 1, 2, 3, 5, 8]
```

2-3 Two base cases in one: fib(0) is 0, fib(1) is 1.

4 Each call makes two more. Exam questions keep n small enough to trace.

What to know about recursion

- Every recursive function could be written as a loop, and the reverse.
- Recursion suits problems made of smaller copies of themselves.
- Each waiting call uses memory. Deep recursion hits the limit: RecursionError.
- To trace: list the calls going down, then the returns coming up.

yield hands out one value, then pauses

```
1  def count_up(n):
2      i = 0
3      while i < n:
4          yield i             # hand out i, then pause here
5          i += 1
6  print(list(count_up(3)))     # [0, 1, 2]
7  for x in count_up(2):
8      print(x, end=" ")        # 0 1
```

4 Any function containing yield is a generator function.

6-8 list() and for loops ask for values one after another until there are none left.

Calling a generator function runs nothing yet

```
1  def gen():
2      print("started")
3      yield 1
4      yield 2
5  g = gen()                  # nothing is printed yet
6  print(type(g))             # <class 'generator'>
7  print(next(g))             # started, then 1
8  print(list(g))             # [2]: only what is left
9  print(list(g))             # []: used up
```

5 The call just creates a generator object. The body has not started.

7 next() asks for one value: the body runs up to the first yield.

8-9 A generator can be used up only once. After that it gives nothing.

next() on a used-up generator raises StopIteration.

Two ways a function can hand back values

	RETURN	YIELD
What the call gives	the returned value	a generator object
Values handed back	one	as many as the yields reached
After handing back	the function is finished	paused, resumes on request
Reused	every call runs it again	used up after one pass

ON THE EXAM

A function containing yield always returns a generator, even if the yield is never reached. A return inside it simply ends it.

Tracing a generator is tracing a loop

```
 1  def every_other(data):
 2      for d in data[::2]:
 3          yield d
 4  for x in every_other("abcdef"):
 5      print(x, end="")          # ace
 6  print()
 7  def stars(n):
 8      s = "*"
 9      for i in range(n):
10          yield s
11          s += s
12  print(list(stars(3)))        # ['*', '**', '****']
```

2-3 The slice takes every second character: a, c, e. One yield per character.

10-11 Each value is yielded before s doubles, so the first value is a single star.

What to carry into the exam

- Recursion needs a **base case that is reached**. Otherwise: RecursionError.
- Trace calls going down, then returns coming up. A base value of 0 zeroes a product.
- Work before the recursive call happens on the way down; after it, on the way up.
- A function with **yield** returns a generator; its body runs only when values are requested.
- A generator can be used up only once. The order of yield and update decides every value.

What is the output of this program?

```
1  def walk(n):
2      if n == 1:
3          return 1
4      return n + walk(n - 2)
5
6  print(walk(5))
```

A 15

B 9

C 8

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def walk(n):
2      if n == 1:
3          return 1
4      return n + walk(n - 2)
5
6  print(walk(5))
```

walk(5) = 5 + walk(3) = 5 + 3 + walk(1) = 5 + 3 + 1 = 9.

A 15

Adds every number from 5 down to 1. The step is 2.

B 9

5, then 3, then the base case 1: 5 + 3 + 1.

C 8

Leaves out the base case's 1.

D The code raises a RecursionError

From 5, stepping by 2, n does land on 1.

THE SIGNAL

Check the base case is actually reached by the steps the recursion takes.

What happens when this program runs?

```
1  def walk(n):
2      if n == 0:
3          return 0
4      return n + walk(n - 2)
5
6  print(walk(5))
```

A 9 is printed

B 0 is printed

C 6 is printed

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def walk(n):
2      if n == 0:
3          return 0
4      return n + walk(n - 2)
5
6  print(walk(5))
```

n goes 5, 3, 1, -1, -3 … It is always odd, so it never equals 0.

A 9 is printed

Assumes the recursion stops at 1. The base case is 0.

B 0 is printed

The base case is never reached, so 0 is never returned.

C 6 is printed

Treats the step as 1. It is 2.

D The code raises a RecursionError

Odd n minus 2 is always odd: n skips 0 for ever.

THE SIGNAL

A base case tested with == can be jumped over. Check the steps can land on it exactly.

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          show(n - 1)
4          print(n, end="")
5
6  show(3)
```

A 321

B 0123

C 123

D 3210

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          show(n - 1)
4          print(n, end="")
5
6  show(3)
```

The print comes after the recursive call, so it happens on the way back up: 1, then 2, then 3.

A 321

That order needs the print before the recursive call.

B 0123

show(0) prints nothing: its if is false.

C 123

Each call prints after the deeper call returns, so 1 comes first.

D 3210

Wrong order, and includes 0, which is never printed.

THE SIGNAL

Work after the recursive call happens as the calls unwind, deepest first.

What is the output of this program?

```
1  def mul(n):
2      if n == 0:
3          return 0
4      return n * mul(n - 1)
5
6  print(mul(4))
```

A 24

B 0

C 10

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def mul(n):
2      if n == 0:
3          return 0
4      return n * mul(n - 1)
5
6  print(mul(4))
```

4 * 3 * 2 * 1 * 0 = 0.

A 24

What a base case of 1 would give. This one returns 0.

B 0

The base case multiplies everything by 0.

C 10

Adds instead of multiplying.

D The code raises a RecursionError

n steps down by 1, so it does reach 0.

THE SIGNAL

A product's base case must return 1. Returning 0 makes every result 0.

What is the output of this program?

```
1  def gen():
2      yield 1
3      yield 2
4      yield 3
5
6  g = gen()
7  print(next(g), list(g), list(g))
```

A `1 [2, 3] []`

B `1 [1, 2, 3] [1, 2, 3]`

C `1 [2, 3] [2, 3]`

D `[1] [2] [3]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def gen():
2      yield 1
3      yield 2
4      yield 3
5
6  g = gen()
7  print(next(g), list(g), list(g))
```

One generator, used from left to right: next takes 1, list takes the rest, then nothing is left.

A `1 [2, 3] []`
next takes 1; the first list takes the rest; the generator is then used up.

B `1 [1, 2, 3] [1, 2, 3]`
Treats each list(g) as a fresh start. g is one generator.

C `1 [2, 3] [2, 3]`
Forgets that a generator can be used up only once.

D `[1] [2] [3]`
next gives a value, not a list.

THE SIGNAL

A generator remembers its position and is used up after one pass. Calling gen() again would start a new one.

What is the output of this program?

```
1  def grow(n):
2      s = "a"
3      for _ in range(n):
4          yield s
5          s = s + "b"
6
7  for x in grow(3):
8      print(x, end=" ")
```

A abb

B a ab abb abbb

C ab abb abbb

D a ab abb

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grow(n):
2      s = "a"
3      for _ in range(n):
4          yield s
5          s = s + "b"
6
7  for x in grow(3):
8      print(x, end=" ")
```

Three passes. Each yields s first, then adds a b: a, ab, abb.

A `abb`

Only the last value. The loop prints every value yielded.

B `a ab abb abbb`

Four values from three passes. range(3) gives three yields.

C `ab abb abbb`

Updates before yielding. Here the yield comes first.

D `a ab abb`

Three yields, each before its update.

THE SIGNAL

Count the yields, then check whether each comes before or after the update.

MODULE 20 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

The exception hierarchy

The family tree of built-in exceptions, what abstract exceptions are, and which operation raises which error.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 20 of 28

admin@sthithapragna.com

Exceptions form a **family tree**. Every exception is also an instance of its parents, so catching a parent **catches every child** below it.

BaseException, and its three children you must know

```
1  BaseException
2   +-- SystemExit          raised by exit() or sys.exit()
3   +-- KeyboardInterrupt   the user presses Ctrl+C
4   +-- Exception           everything a program normally handles
```

1 BaseException is the root. Every exception, built-in or not, descends from it.

2-3 These two are not below Exception, so handling Exception does not stop them.

4 All the everyday errors live under Exception.

BaseException also has GeneratorExit and, since 3.11, BaseExceptionGroup. Both are beyond the syllabus.

The part of the tree the exam asks about

```
Exception
 +-- ArithmeticError
 |    +-- ZeroDivisionError
 |    +-- OverflowError
 +-- LookupError
 |    +-- IndexError
 |    +-- KeyError
 +-- TypeError
 +-- ValueError
 +-- NameError
 |    +-- UnboundLocalError
 +-- AttributeError
 +-- SyntaxError
      +-- IndentationError
```

2-4 Arithmetic problems: dividing by zero, or a result too large to represent.

5-7 Lookup problems: a bad index, or a missing key.

issubclass() answers is-a-kind-of questions

```
1  print(issubclass(ZeroDivisionError, ArithmeticError))   # True
2  print(issubclass(KeyError, LookupError))                # True
3  print(issubclass(KeyError, IndexError))                 # False: siblings
4  print(issubclass(KeyboardInterrupt, Exception))         # False
5  print(issubclass(Exception, BaseException))             # True
```

3 KeyError and IndexError share a parent, but neither is a kind of the other.

4 KeyboardInterrupt is outside the Exception branch.

Parents that group their children

EXCEPTION	GROUPS	RAISED DIRECTLY BY BUILT-INS?
`BaseException`	every exception there is	no
`Exception`	every everyday error	no
`ArithmeticError`	ZeroDivisionError, OverflowError	no: you meet its children
`LookupError`	IndexError, KeyError	no: you meet its children

ON THE EXAM

An abstract exception is one you handle, as a group, rather than one you expect to see raised.

Arithmetic and lookup errors

CODE	RAISES	PARENT
`10 / 0 10 // 0 10 % 0`	ZeroDivisionError	ArithmeticError
`2.0 ** 10000`	OverflowError	ArithmeticError
`[1][5] "ab"[9] (1,)[2]`	IndexError	LookupError
`{"a": 1}["b"]`	KeyError	LookupError

ON THE EXAM

Too large to represent is OverflowError. Very large ints are fine: only floats overflow.

Type, value, name and attribute errors

CODE	RAISES	BECAUSE
`"1" + 1 len(5) int(None)`	TypeError	the wrong type for the operation
`int("x") [1].remove(9) math.sqrt(-1)`	ValueError	the right type, but an unusable value
`print(undefined)`	NameError	no such name anywhere
`"abc".foo()`	AttributeError	the object has no such method

THE TIEBREAKER

Wrong type: TypeError. Right type, wrong value: ValueError. The most useful tiebreaker in the block.

The same function, two different errors

WRONG TYPE

TypeError

```
1  print(int(None))
```

int() cannot convert None at all: it is not a string or a number.

RIGHT TYPE, WRONG VALUE

ValueError

```
1  print(int("x"))
```

int() accepts strings, but "x" does not spell a whole number.

ON THE EXAM

Ask: could any value of this type have worked? Yes: ValueError. No: TypeError.

Other exceptions you have met

EXCEPTION	RAISED WHEN	WHERE IT SITS
`RecursionError`	recursion goes too deep	under RuntimeError
`StopIteration`	next() on a used-up generator	under Exception
`EOFError`	input() finds no more input	under Exception
`AssertionError`	an assert condition is false	under Exception
`IndentationError, TabError`	bad indentation	under SyntaxError

assert raises AssertionError when a condition is false

```
1  assert 2 + 2 == 4                  # true: nothing happens
2  assert len([]) > 0, "list is empty"
3  # AssertionError: list is empty
```

1. assert checks a condition that should always be true.
2. If it is false, AssertionError is raised, with the optional message after the comma.

The last line of a traceback names the exception

- A traceback starts **Traceback (most recent call last):** and lists the calls that led to the error.
- The **last line** gives the exception name, a colon, and a message.
- Since 3.11, markers under the code point to the exact part of the line that failed.
- Since 3.10, NameError and AttributeError often add: Did you mean …?

> ON THE EXAM
>
> Exam options name exceptions. Match the name; treat the message as a clue, not the answer.

What to carry into the exam

- **BaseException** is the root. SystemExit and KeyboardInterrupt sit **beside** Exception.
- ArithmeticError: ZeroDivisionError, OverflowError. LookupError: IndexError, KeyError.
- Abstract exceptions group their children; built-ins raise the children.
- Wrong type: **TypeError**. Right type, bad value: **ValueError**.
- UnboundLocalError is a NameError; IndentationError is a SyntaxError.

> Consider Python's built-in exception hierarchy.

Which exception is the direct parent of KeyError?

A `Exception`

B `ArithmeticError`

C `LookupError`

D `IndexError`

◆ PAUSE AND DECIDE

Which exception is the direct parent of KeyError?

A `Exception`

An ancestor, but not the direct parent. LookupError sits between them.

B `ArithmeticError`

Groups calculation problems, not lookups.

C `LookupError`

KeyError and IndexError are both kinds of LookupError.

D `IndexError`

IndexError is a sibling of KeyError, not its parent.

THE SIGNAL

Direct parent means one level up. KeyError, then LookupError, then Exception, then BaseException.

> Each expression is evaluated on its own.

Which two raise an exception that is a subclass of ArithmeticError? (Choose two.)

A `7 // 0`

B `int("7.0")`

C `2.0 ** 5000`

D `[7][1]`

◆ PAUSE AND DECIDE

Which two raise an exception that is a subclass of ArithmeticError? (Choose two.)

A `7 // 0`

ZeroDivisionError, a kind of ArithmeticError.

B `int("7.0")`

ValueError: a str that does not spell a whole number.

C `2.0 ** 5000`

OverflowError: the float result is too large. A kind of ArithmeticError.

D `[7][1]`

IndexError, a kind of LookupError.

THE SIGNAL

Name the exception first, then place it in the tree.

Which exception does this program raise?

```
1  data = {"a": [1, 2]}
2  print(data["a"][2])
```

A KeyError

B TypeError

C ValueError

D IndexError

◆ PAUSE AND DECIDE

Which exception does this program raise?

```
1  data = {"a": [1, 2]}
2  print(data["a"][2])
```

data["a"] succeeds: it is [1, 2]. Then [2] is past the end of that list.

A `KeyError`
The key "a" exists. The failure is in the second lookup.

B `TypeError`
A list can be indexed by an int, so the types are fine.

C `ValueError`
Bad positions are IndexError, not ValueError.

D `IndexError`
The list [1, 2] has indexes 0 and 1 only.

THE SIGNAL

With chained lookups, find which step fails, and what kind of object it was working on.

> Consider Python's built-in exception hierarchy.

Which statement is true?

A KeyboardInterrupt is a subclass of Exception

B Every exception class derives from BaseException

C A bad list index raises LookupError itself

D ValueError is a subclass of TypeError

◆ PAUSE AND DECIDE

Which statement is true?

A KeyboardInterrupt is a subclass of Exception
It sits beside Exception, directly under BaseException.

B Every exception class derives from BaseException
BaseException is the root of the whole tree.

C A bad list index raises LookupError itself
It raises IndexError, a child. LookupError is abstract.

D ValueError is a subclass of TypeError
They are siblings, both directly under Exception.

THE SIGNAL

BaseException is the root; Exception is one branch of it; abstract parents are handled, not raised.

MODULE 21 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Handling exceptions

try and except, the order of except branches, else and finally, raise, and exceptions travelling through function calls.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 21 of 28

admin@sthithapragna.com

When an exception occurs in a try block, the rest of the block is skipped, and Python runs the **first** except branch that matches. **At most one** branch runs.

The try block stops at the first exception

```
1  try:
2      print("start")
3      print(1 / 0)            # raises here
4      print("never")          # skipped
5  except ZeroDivisionError:
6      print("caught")
7  print("after")              # start, caught, after
```

1-4 Put code that might fail in the try block. Everything after the failing line is skipped.

5-6 The except branch handles the exception, and the program carries on after it.

The first matching branch wins, and parents match

```
1  try:
2      print(10 // 0)
3  except ArithmeticError:        # matches: ZeroDivisionError is a kind of it
4      print("A")
5  except ZeroDivisionError:      # never reached
6      print("Z")
7  except:
8      print("?")                  # output: A
```

3 A branch matches the exception itself, or any of its parents.

5 Once a branch has matched, the others are skipped, even a more exact one.

Broad first hides narrow. A bare except first is illegal

BROAD BEFORE NARROW

Legal, but the narrow branch is dead

```
try:
    raise Exception
except BaseException:
    print("1")
except Exception:
    print("2")
```

Prints 1. The Exception branch can never run, but this is not an error.

BARE EXCEPT FIRST

SyntaxError

```
try:
    raise ValueError
except:
    print("x")
except ValueError:
    print("v")
```

SyntaxError: default 'except:' must be last. Nothing runs.

THE RULE

Order branches from the most specific to the most general, with any bare except: last.

A tuple of exceptions in one except

```
1  for v in ["x", "0", "4"]:
2      try:
3          print(10 / int(v))
4      except (ValueError, ZeroDivisionError) as e:
5          print("problem:", e)
```

4 Either exception is handled by this one branch. as e names the exception object.

5 Printing e shows the exception's message.

Output: problem: invalid literal for int() with base 10: 'x', then problem: division by zero, then 2.5

Brackets around several exceptions became optional

PYTHON 3.13 AND EARLIER

SyntaxError

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError, ValueError:
4      print("caught")
```

SyntaxError: multiple exception types must be parenthesized.

PYTHON 3.14 (PEP 758)

Legal, without as

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError, ValueError:
4      print("caught")
```

Prints caught. With as e, brackets are still required, even on 3.14.

ON THE EXAM

Exam answer: the brackets are required. Choose the bracketed option; it is correct in every version.

else runs if nothing went wrong. finally always runs

```
1  try:
2      x = int("5")
3  except ValueError:
4      print("E", end="")
5  else:
6      print("L", end="")          # no exception, so else runs
7  finally:
8      print("F", end="")          # runs in every case
9  print("!")                     # output: LF!
```

5-6 else runs only when the try block finished without an exception.

7-8 finally runs last, whatever happened: exception, no exception, even a return.

The order is fixed: try, except, else, finally. else needs at least one except; try with only finally is legal.

finally runs even when a branch returns

```
 1  def f():
 2      try:
 3          print("a", end="")
 4          x = [1][3]              # IndexError
 5          print("b", end="")      # skipped
 6      except IndexError:
 7          print("c", end="")
 8          return
 9      finally:
10          print("d", end="")      # still runs, before the return completes
11      print("e", end="")          # never reached
12  f()                            # output: acd
```

8-10 The return is held up until finally has run.

11 After the return, the rest of the function is skipped.

return inside finally now gives a warning

- A **return**, **break** or **continue** that leaves a finally block silently discards any exception, or overrides the earlier return.
- Python 3.14 (PEP 765) reports this with **SyntaxWarning: 'return' in a 'finally' block**.
- It is a warning, not an error. The code still runs exactly as before.

> ON THE EXAM
>
> try: return 1, finally: return 2 still returns 2. Only the warning is new.

raise signals a problem your code has detected

```
1  def check(age):
2      if age < 0:
3          raise ValueError("age cannot be negative")
4      return age
5  try:
6      check(-1)
7  except ValueError as e:
8      print("error:", e)        # error: age cannot be negative
```

3 raise creates the exception, with an optional message, and starts it on its way.

6-8 It is handled exactly like a built-in one.

In Python terminology, exceptions are raised, not thrown. raise ValueError, without brackets, also works.

An unhandled exception moves up to the caller

```
1  def half(n):
2      return 10 / n            # no try here: the exception leaves half
3  def safe(n):
4      try:
5          return half(n)       # ... and arrives here, inside a try
6      except ZeroDivisionError:
7          return -1
8  print(safe(0), safe(4))     # -1 2.5
```

2 half does not handle the exception, so it leaves half immediately.

5-7 It arrives in safe, inside the try, so safe's except branch handles it.

Nested handlers: the inner one gets the first chance

```
1  try:
2      try:
3          [][0]                # IndexError
4      except KeyError:
5          print("inner")     # does not match
6  except IndexError:
7      print("outer")         # output: outer
```

4-5 The inner handler does not match, so the exception passes outwards.

6-7 The outer handler matches and deals with it.

Errors that except branches do not catch

SITUATION	WHAT HAPPENS
A SyntaxError anywhere in the file	nothing runs: syntax is checked before any try exists
break inside try, but not inside a loop	SyntaxError: 'break' outside loop
an exception type no branch names	it travels on, and may end the program
an exception inside an except branch	a new exception, not handled by the same try
except Exception, and the user presses Ctrl+C	KeyboardInterrupt is not an Exception: it gets through

THE TRAP

try handles exceptions raised while its block runs. It cannot rescue code that never compiled.

Words about exceptions and debugging

- Exceptions are **raised**, not thrown. They are **handled** in except branches.
- A **debugger** runs a program step by step so you can trace it precisely.
- **Print debugging** means adding print() calls to see values while a program runs.
- Running without errors does not prove code is correct: some paths may never have run.
- Python's style is to try the operation and handle failure, rather than check everything first.

What to carry into the exam

- The try block stops at the first exception. The **first matching** except branch runs; parents match.
- At most one except branch runs. A bare except: must be last, or it is a SyntaxError.
- else: no exception happened. finally: always runs, even after return.
- Several exceptions: **except (A, B):** with brackets, for the exam.
- Unhandled exceptions travel up through callers. SyntaxErrors cannot be caught.

What is the output of this program?

```
try:
    print(10 // 0)
except ArithmeticError:
    print("A")
except ZeroDivisionError:
    print("Z")
except:
    print("?")
```

A. A

B. Z

C. A, then Z

D. The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      print(10 // 0)
3  except ArithmeticError:
4      print("A")
5  except ZeroDivisionError:
6      print("Z")
7  except:
8      print("?")
```

ZeroDivisionError is a kind of ArithmeticError, so the first branch matches.

A A

The first branch matches through the parent, and ends the search.

B Z

The exact match comes second. The first match wins.

C A, then Z

At most one except branch runs.

D The code raises a SyntaxError

The bare except is last, which is legal. Broad before narrow is not an error.

THE SIGNAL

Parents match. The first matching branch wins, even when a more exact one follows.

What happens when this program runs?

```
1  print("go")
2  try:
3      raise ValueError
4  except:
5      print("x")
6  except ValueError:
7      print("v")
```

A go, then x

B go, then v

C go, then x, then v

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("go")
2  try:
3      raise ValueError
4  except:
5      print("x")
6  except ValueError:
7      print("v")
```

SyntaxError: default 'except:' must be last. Found before line 1 runs.

A go, then x

What you would get if Python allowed this order. It does not.

B go, then v

The file never compiles, so nothing runs.

C go, then x, then v

Wrong twice: it does not compile, and only one branch could ever run.

D Nothing is printed; the program raises a SyntaxError

A bare except must be last, so even go is never printed.

THE SIGNAL

A bare except: that is not last is a SyntaxError, so nothing in the file runs.

The user types 4 and presses Enter.

What happens when this program runs?

```
try:
    n = input()
    print(n / 2)
except ValueError:
    print("bad value")
except ZeroDivisionError:
    print("zero")
print("done")
```

A bad value, then done

B zero, then done

C The program stops with an unhandled TypeError

D 2.0, then done

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      n = input()
3      print(n / 2)
4  except ValueError:
5      print("bad value")
6  except ZeroDivisionError:
7      print("zero")
8  print("done")
```

n is the str "4". A str divided by an int is a TypeError, and no branch names TypeError.

A. bad value, then done
The problem is the type, str, not a bad value. That is TypeError.

B. zero, then done
Nothing divides by zero here.

C. The program stops with an unhandled TypeError
No branch matches TypeError, so it travels on and ends the program.

D. 2.0, then done
Forgets that input() returns a str. Nothing converts it.

THE SIGNAL

If no except branch matches, the try might as well not be there. Name the exception first.

What is the output of this program?

```
try:
    x = int("5")
except ValueError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
print("!")
```

A ELF!

B LF!

C F!

D L!

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      x = int("5")
3  except ValueError:
4      print("E", end="")
5  else:
6      print("L", end="")
7  finally:
8      print("F", end="")
9  print("!")
```

int("5") succeeds: except skipped, else runs, finally runs.

A ELF!

Runs the except branch too. There was no exception.

B LF!

No exception: else runs, then finally, then the next line.

C F!

Forgets that else runs when the try block succeeds.

D L!

Forgets that finally always runs.

THE SIGNAL

No exception: try, else, finally. Exception handled: try, except, finally.

What is the output of this program?

```
1  def half(n):
2      return 10 / n
3
4  def safe(n):
5      try:
6          return half(n)
7      except ZeroDivisionError:
8          return -1
9
10 print(safe(0), safe(4))
```

A `-1 2.5`

B `-1 2`

C The code raises a ZeroDivisionError

D `None 2.5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def half(n):
2      return 10 / n
3
4  def safe(n):
5      try:
6          return half(n)
7      except ZeroDivisionError:
8          return -1
9
10 print(safe(0), safe(4))
```

The exception leaves half, arrives in safe's try, and is handled there.

A `-1 2.5`

The error travels up to safe, which handles it; 10 / 4 is 2.5.

B `-1 2`

/ always gives a float: 2.5.

C The code raises a ZeroDivisionError

half has no handler, but its caller does.

D `None 2.5`

The except branch returns -1, not None.

THE SIGNAL

An exception with no handler in its own function travels to the caller's handler.

What is the output of this program?

```
1  def f():
2      try:
3          print("a", end="")
4          x = [1][3]
5          print("b", end="")
6      except IndexError:
7          print("c", end="")
8          return
9      finally:
10         print("d", end="")
11     print("e", end="")
12
13 f()
```

A acde

B abcd

C acd

D ad

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f():
2      try:
3          print("a", end="")
4          x = [1][3]
5          print("b", end="")
6      except IndexError:
7          print("c", end="")
8          return
9      finally:
10         print("d", end="")
11     print("e", end="")
12
13 f()
```

A `acde`

The return in the except branch leaves f before e.

B `abcd`

The try block stops at the IndexError, so b is skipped.

C `acd`

a, then the error skips b, c, finally d, and return skips e.

D `ad`

The except branch does run and prints c.

a; IndexError skips b; c; return waits for finally: d; e is never reached.

THE SIGNAL

Trace three things: where the try block stops, which branch runs, and whether anything returns.

What is the output of this program?

```
try:
    raise KeyError("k")
except LookupError:
    print("lookup")
except KeyError:
    print("key")
```

A key

B lookup

C lookup, then key

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      raise KeyError("k")
3  except LookupError:
4      print("lookup")
5  except KeyError:
6      print("key")
```

KeyError is a kind of LookupError, so the first branch matches.

A **key**
The exact match comes second, and is never reached.

B **lookup**
KeyError's parent, LookupError, matches first.

C **lookup, then key**
Only one except branch ever runs.

D **The code raises a KeyError**
It is handled, by the LookupError branch.

THE SIGNAL

Module 20's tree decides module 21's handlers: a parent branch catches all its children.

> Consider try statements in Python in general.

Which two statements are true? (Choose two.)

A At most one except branch runs for one exception

B A SyntaxError inside a try block is caught by a bare except

C except Exception: catches KeyboardInterrupt

D The finally branch runs whether or not an exception occurred

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A At most one except branch runs for one exception

The first matching branch runs; the rest are skipped.

B A SyntaxError inside a try block is caught by a bare except

Syntax is checked before anything runs, so no try exists yet.

C except Exception: catches KeyboardInterrupt

KeyboardInterrupt sits beside Exception, not under it.

D The finally branch runs whether or not an exception occurred

That is what finally is for, even after a return.

THE SIGNAL

One branch at most; finally always; SyntaxErrors and KeyboardInterrupt get past the usual handlers.

MODULE 22 · EXAM STRATEGY - ALL BLOCKS

Spot the error

A three-step method for every 'what happens?' question, the signals of a syntax error, and twelve mixed drills.

BLOCKS all four FOCUS errors MODULE 22 of 28

admin@sthithapragna.com

Before tracing any code, ask three questions in order. **Does it compile?** Does it run to the end? Only then: what does it print?

Three passes over the code

01

1. Grammar

Colons, brackets, quotes, indentation, keywords, placement. Any fault: nothing prints.

›

02

2. Run

Types, indexes, keys, names, zero, argument counts. Find the first failing line.

›

03

3. Output

Everything printed before the failing line appears; nothing after it does.

›

04

4. Answer

Match the exception by name. Messages are clues, not answers.

THE KEY DIFFERENCE

A syntax error means no output at all. A runtime error means partial output, then the exception.

Syntax errors in statements

SIGNAL	EXAMPLE
missing colon	if count > 2
unclosed bracket or quote	print("hi"
bad indentation, or a block with only a comment	if x: # nothing else
keyword used as a name	class = 3
= in a condition	if x = 1:
else if instead of elif	else if x > 3:
break or continue outside a loop	if x > 2: break
return outside a function	return 5 at the top level

> ON THE EXAM
>
> Every row stops the whole file before line 1 runs.

Syntax errors in functions and exception handling

SIGNAL	EXAMPLE
def without brackets	def fun:
a default before a plain parameter	def f(a=1, b):
positional argument after a keyword argument	f(a=1, 2)
bare except: that is not last	except: then except ValueError:
global after the name is used	print(x) then global x
else with no except	try: … else: …
several exceptions without brackets	except A, B: before Python 3.14

ON THE EXAM

Again, nothing runs. Watch especially for argument order and bare except placement.

Runtime symptoms and the exception they raise

SYMPTOM	EXCEPTION
`str + int; calling a non-function; wrong argument count`	TypeError
`right type, unusable value: int("x"), [1].remove(9)`	ValueError
`position past the end of a list, string or tuple`	IndexError
`missing dictionary key`	KeyError
`name that does not exist`	NameError
`local read before it is assigned`	UnboundLocalError
`method the object does not have`	AttributeError
`divide or remainder by zero`	ZeroDivisionError

ON THE EXAM

Runtime errors happen only when their line is reached, so earlier output has already appeared.

How much gets printed before the error

SYNTAX ERROR ON LINE 3

Nothing at all

```
1  print("one")
2  print("two")
3  if True print("three")
```

Output: nothing. SyntaxError before line 1 runs.

RUNTIME ERROR ON LINE 3

Everything before it

```
1  print("one")
2  print("two")
3  print("three" + 3)
```

Output: one, two, then TypeError. Line 3 is where it stops.

THE TIEBREAKER

If an option says some lines print and then an error occurs, the error must be a runtime error.

Code that looks wrong, but runs

CODE	WHAT ACTUALLY HAPPENS
`code after a return`	legal; it simply never runs
`except Exception: before except ValueError:`	legal; the second branch is dead
`"a" == 1`	False, not an error
`x = x == x`	x becomes True
`[1, 2][1:9]`	[2]: slices never raise IndexError
`for i in range(3, 3): ... else: ...`	zero passes; the else still runs
`f(b=3, a=2)`	legal: keywords in any order
`a function defined below the one that calls it`	fine, if both defs run before the call

THE TRAP

Before choosing 'the code is erroneous', make sure the rule you think is broken really exists.

Error messages that point at the answer

- Since 3.10: NameError and AttributeError often add **Did you mean: ...?**
- Since 3.11: markers under the line show the exact part that failed.
- Python 3.14: keyword typos get **Did you mean 'while'?**, and string errors explain escaped quotes.
- Use these when you practise. On the exam, answer with the exception's name.

What to carry into the exam

- Pass 1: grammar. Any fault means **nothing prints**.
- Pass 2: run. Find the **first** line that raises, and name the exception.
- Pass 3: output. Everything before the failing line has printed.
- A handled exception is not an error: check for try blocks that catch it.
- Do not choose 'erroneous' unless you can name the rule that is broken.

What happens when this program runs?

```
1  print("Total")
2  count = 3
3  if count > 2
4      print("big")
```

A Total, then big

B Total, then a SyntaxError

C Nothing is printed; the program raises a SyntaxError

D Total only

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("Total")
2  count = 3
3  if count > 2
4      print("big")
```

SyntaxError: expected ':'. Found on the grammar pass, before line 1 runs.

A Total, then big

The if line has no colon.

B Total, then a SyntaxError

Syntax errors stop everything, including line 1.

C Nothing is printed; the program raises a SyntaxError

The missing colon is a grammar fault, so no line runs.

D Total only

Python never skips a broken line and carries on.

THE SIGNAL

Pass 1 finds it: a missing colon means nothing prints.

What happens when this program runs?

```
1  items = ["a", "b"]
2  print(len(items))
3  print("Count: " + len(items))
4  print("end")
```

A 2, then Count: 2, then end

B 2, then a TypeError

C Nothing is printed; the program raises a TypeError

D 2, then a ValueError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  items = ["a", "b"]
2  print(len(items))
3  print("Count: " + len(items))
4  print("end")
```

TypeError: can only concatenate str (not "int") to str.

A 2, then Count: 2, then end

+ never converts an int to a str for you.

B 2, then a TypeError

Line 2 runs; line 3 joins a str and an int, which is a type problem.

C Nothing is printed; the program raises a TypeError

A TypeError is a runtime error, so line 2 has already printed.

D 2, then a ValueError

The problem is the type of len's result, not its value.

THE SIGNAL

Pass 2 finds it on line 3. Pass 3: line 2 has already printed.

What happens when this program runs?

```
for i in range(3, 3):
    print("loop")
else:
    print("else")
```

A loop, then else

B Nothing is printed

C The code raises a SyntaxError

D else is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  for i in range(3, 3):
2      print("loop")
3  else:
4      print("else")
```

range(3, 3) is empty: zero passes. No break, so the else runs.

A loop, then else
An empty range gives zero passes.

B Nothing is printed
The loop's else runs even after zero passes.

C The code raises a SyntaxError
else on a for loop is legal.

D **else is printed**
Zero passes, no break: only the else runs.

THE SIGNAL

Looks odd, runs fine. An empty range and a loop else are both legal.

What happens when this program runs?

```
1  def f(a, b=2):
2      return a * b
3
4  print(f(b=3, a=2), f(4))
```

A 6 8 is printed

B SyntaxError: positional argument follows keyword argument

C The code raises a TypeError

D 6 4 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def f(a, b=2):
2      return a * b
3
4  print(f(b=3, a=2), f(4))
```

f(b=3, a=2) is 2 * 3. f(4) uses the default b = 2: 4 * 2.

A 6 8 is printed
Keywords in any order are legal; the second call uses the default.

B SyntaxError: positional argument follows keyword argument
Every argument in the first call is a keyword. No positional follows one.

C The code raises a TypeError
Both calls supply a, and b is filled or defaulted.

D 6 4 is printed
Forgets that f(4) still multiplies by the default 2.

THE SIGNAL

Keyword arguments may come in any order. Only a positional after a keyword is illegal.

What happens when this program runs?

```
1  stock = {"tea": 2}
2  print(stock.get("jam", 0))
3  print(stock["jam"])
```

A 0, then 0

B 0, then None

C 0, then a KeyError

D Nothing is printed; the program raises a KeyError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  stock = {"tea": 2}
2  print(stock.get("jam", 0))
3  print(stock["jam"])
```

get() never raises. Square brackets on a missing key do.

A 0, then 0
Square brackets have no default.

B 0, then None
Only get() returns None. Square brackets raise.

C 0, then a KeyError
Line 2 uses get's default; line 3 fails on the missing key.

D Nothing is printed; the program raises a KeyError
A KeyError is a runtime error, so line 2 has printed.

THE SIGNAL

get with a default is safe. Square brackets on a missing key are a KeyError.

What happens when this program runs?

```
1  total = 0
2  def add(n):
3      total += n
4
5  add(5)
6  print(total)
```

A 5 is printed

B 0 is printed

C The code raises an UnboundLocalError

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  total = 0
2  def add(n):
3      total += n
4
5  add(5)
6  print(total)
```

+= assigns total, so total is local in add, and it is read before it has a value.

A 5 is printed

Would need global total inside add.

B 0 is printed

The call to add fails before print is reached.

C The code raises an UnboundLocalError

total is local to add, and += reads it first.

D The code raises a SyntaxError

The grammar is fine. The problem only appears when add runs.

THE SIGNAL

Module 18's rule: assigning a name anywhere in a function makes it local there.

What happens when this program runs?

```
1  x = 3
2  print(x)
3  if x > 2:
4      break
```

A 3 is printed

B Nothing is printed; the program raises a SyntaxError

C 3, then a SyntaxError

D 3, then a RuntimeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  x = 3
2  print(x)
3  if x > 2:
4      break
```

SyntaxError: 'break' outside loop.

A 3 is printed

break outside a loop is a grammar fault, so nothing runs.

B Nothing is printed; the program raises a SyntaxError

break must be inside a loop, and grammar is checked first.

C 3, then a SyntaxError

A syntax error stops line 2 as well.

D 3, then a RuntimeError

It is found before running, not while running.

THE SIGNAL

break outside a loop is grammar, not behaviour. The file never starts.

What happens when this program runs?

```
1  point = (1, 2)
2  x, y = point
3  print(x + y)
4  point[0] = 5
```

A 3, then a TypeError

B 3, then an AttributeError

C 3 is printed, and the program ends normally

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  point = (1, 2)
2  x, y = point
3  print(x + y)
4  point[0] = 5
```

TypeError: 'tuple' object does not support item assignment.

A. 3, then a TypeError
Unpacking and printing work; assigning to a tuple element does not.

B. 3, then an AttributeError
That is for a missing method, like append. This is item assignment.

C. 3 is printed, and the program ends normally
Tuples cannot be changed.

D. Nothing is printed; the program raises a SyntaxError
Assigning to an element is grammatical. It fails when it runs.

THE SIGNAL

Tuple element assignment: TypeError. A list-only method on a tuple: AttributeError.

What happens when this program runs?

```
1  x = 5
2  x = x == 5
3  print(x, type(x))
```

A `5 <class 'int'>`

B `False <class 'bool'>`

C The code raises a SyntaxError

D `True <class 'bool'>`

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  x = 5
2  x = x == 5
3  print(x, type(x))
```

The comparison x == 5 is worked out first: True. Then x is assigned True.

A `5 <class 'int'>`
x is reassigned on line 2, to the comparison's result.

B `False <class 'bool'>`
x is 5 when the comparison runs, so it is True.

C The code raises a SyntaxError
= then == on one line is legal: assign the result of a comparison.

D `True <class 'bool'>`
5 == 5 is True, and that bool is assigned to x.

THE SIGNAL

Looks odd, runs fine. The right side is evaluated completely before the assignment.

What happens when this program runs?

```
try:
    value = int("7") + "1"
except ValueError:
    print("value")
except:
    print("other")
```

A value is printed

B other is printed

C 8 is printed

D The program stops with an unhandled TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      value = int("7") + "1"
3  except ValueError:
4      print("value")
5  except:
6      print("other")
```

int("7") succeeds. 7 + "1" is a TypeError, caught by the bare except.

A value is printed
int("7") works. The failure is int + str, a TypeError.

B other is printed
The TypeError skips the ValueError branch and is caught by the bare except.

C 8 is printed
An int and a str are never added.

D The program stops with an unhandled TypeError
The bare except catches everything that reaches it.

THE SIGNAL

Name the exception, then check the handlers. A caught exception is not an error.

What happens when this program runs?

```
1  names = ["Ann", "Bob"]
2  for i in range(len(names) + 1):
3      print(names[i])
```

A Ann, then Bob

B Nothing is printed; the program raises an IndexError

C Ann, then Bob, then a KeyError

D Ann, then Bob, then an IndexError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  names = ["Ann", "Bob"]
2  for i in range(len(names) + 1):
3      print(names[i])
```

range(3) gives 0, 1, 2. names[2] does not exist.

A. Ann, then Bob
The loop makes a third pass, with i = 2.

B. Nothing is printed; the program raises an IndexError
The first two passes succeed and print before the error.

C. Ann, then Bob, then a KeyError
Lists raise IndexError. KeyError is for dictionaries.

D. Ann, then Bob, then an IndexError
Passes 0 and 1 print; pass 2 is past the end.

THE SIGNAL

Off by one in a range is a runtime IndexError, so earlier passes have already printed.

> Each line is saved and run as a whole program on its own.

Which two programs raise a SyntaxError, so that nothing runs? (Choose two.)

A `def f(a=1, b): pass`

B `print(undefined_name)`

C `x = [1, 2]; print(x[5])`

D `while True print(1)`

◆ PAUSE AND DECIDE

Which two programs raise a SyntaxError, so that nothing runs? (Choose two.)

A `def f(a=1, b): pass`

A plain parameter after a default. SyntaxError.

B `print(undefined_name)`

Grammatical. NameError when it runs.

C `x = [1, 2]; print(x[5])`

Grammatical. IndexError when it runs.

D `while True print(1)`

Missing colon. SyntaxError.

THE SIGNAL

Grammar faults: SyntaxError. Faults that depend on values or names: runtime exceptions.

MODULE 23 · EXAM STRATEGY - ALL BLOCKS

Confusion pairs

The look-alikes the exam puts side by side, and a one-line rule for telling each pair apart.

BLOCKS all four FOCUS look-alikes MODULE 23 of 28

admin@sthithapragna.com

When two options differ by one small thing, the question is **testing that difference**. Find it, apply its rule, and the answer follows.

Operators that look alike

PAIR	THE RULE	EXAMPLE
`/ versus //`	/ always gives a float; // floors	7 / 2 is 3.5; 7 // 2 is 3
`// versus int()`	// rounds down; int() cuts towards zero	-7 // 2 is -4; int(-3.5) is -3
`= versus ==`	= assigns; == compares	if x = 5: is a SyntaxError
`and versus &`	and returns an operand; & works on bits	1 and 2 is 2; 1 & 2 is 0
`is versus ==`	is: same object; ==: equal values	[1] == [1] True; [1] is [1] False
`"2" + "4" versus 2 + 4`	+ joins strings, adds numbers	"24" versus 6

Control flow that looks alike

PAIR	THE RULE	EXAMPLE, OVER "AB3CD" STOPPING AT 3
`break`	leaves the loop at once	ab
`continue`	skips to the next pass	abcd
`pass`	does nothing at all	ab3cd
`elif chain versus separate ifs`	one branch at most; each if decides alone	count the chains first
`loop else versus if else`	loop else: no break happened	runs even after zero passes
`range(5) versus range(5, 0)`	one argument is stop; no step counts up	0 to 4, versus empty

List operations that look alike

PAIR	THE RULE	EXAMPLE, FROM [1, 2]
`append versus extend`	append adds one element; extend adds each	[1, 2, [3]] versus [1, 2, 3]
`sort() versus sorted()`	sort changes the list, returns None; sorted returns a new list	n.sort() is None
`pop() versus remove() versus del`	pop by position and returns; remove by value; del by position	pop() returns the element
`b = a versus b = a[:]`	alias shares one list; slice copies it	a change through b shows in a
`[[0]*2]*2 versus a comprehension`	* repeats one inner list; a comprehension builds new ones	[[1, 0], [1, 0]] after one change
`x += [3] versus x = x + [3]`	+= changes the shared list; = + builds a new one	matters only with an alias

Tuples, strings and dictionaries

PAIR	THE RULE	EXAMPLE
`(1) versus (1,)`	the comma makes a tuple	int versus tuple
`find() versus index()`	find gives -1; index raises ValueError	"banana".find("x") is -1
`d[k] versus d.get(k)`	[] raises KeyError; get gives None or a default	{"a": 1}.get("b") is None
`k in d versus v in d.values()`	in on a dict checks keys only	1 in {"a": 1} is False
`s.upper() versus s = s.upper()`	a string method's result must be kept	s itself never changes
`len() versus count()`	len counts everything; count one value	[1, 1, 2]: 3 versus 2

Functions and exceptions that look alike

PAIR	THE RULE	EXAMPLE
`return versus print`	return gives a value back; print only shows it	print(print("x")) shows x, then None
`local versus global`	assigned in the function: local, unless global	total += n needs global total
`f(a=1, 2) versus f(1, a=2)`	wrong order: SyntaxError; filled twice: TypeError	one never runs; one fails at the call
`TypeError versus ValueError`	wrong type, versus right type with a bad value	int(None) versus int("x")
`IndexError versus KeyError`	bad position, versus missing dictionary key	[1][5] versus {}["k"]
`SyntaxError versus runtime error`	nothing runs, versus stops at the failing line	partial output only for runtime

admin@sthithapragna.com

break, continue and pass on the same loop

```
1  for word in ("break", "continue", "pass"):
2      out = ""
3      for ch in "ab3cd":
4          if ch == "3":
5              if word == "break": break
6              elif word == "continue": continue
7              else: pass
8          out += ch
9      print(word, out)     # break ab, continue abcd, pass ab3cd
```

5 break: the rest of the string is never reached.

6 continue: only the 3 is skipped.

7 pass: nothing happens, so the 3 is added like any other character.

What to carry into the exam

- Find the **one difference** between the options, then apply that pair's rule.
- // floors; int() cuts to zero. append adds one; extend adds each.
- sort() returns None; sorted() returns a list. find gives -1; index raises.
- The comma makes a tuple. in on a dict checks keys. pass does nothing.
- Wrong argument order: SyntaxError. Wrong count or filled twice: TypeError.

What is the output of this program?

```
1  a = [1, 2]
2  b = a
3  c = a[:]
4  a += [3]
5  c.append(4)
6  print(len(a), len(b), len(c))
```

A 3 2 3

B 3 3 3

C 3 3 2

D 2 2 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  b = a
3  c = a[:]
4  a += [3]
5  c.append(4)
6  print(len(a), len(b), len(c))
```

a and b share [1, 2, 3]. c is a separate [1, 2, 4].

A 3 2 3

b is an alias, so it sees a's += as well.

B 3 3 3

+= grows the shared list to 3; the copy grows separately to 3.

C 3 3 2

c's own append makes it 3 long too.

D 2 2 3

+= changes the list a and b share.

THE SIGNAL

Alias versus copy, and += changing the shared list: two pairs in one question.

What is the output of this program?

```
1  out = ""
2  for ch in "a1b2":
3      if ch.isdigit():
4          continue
5      out += ch
6  for ch in "a1b2":
7      if ch.isdigit():
8          pass
9      out += ch
10 print(out)
```

A ab

B abab

C aba1b2

D a

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  out = ""
2  for ch in "a1b2":
3      if ch.isdigit():
4          continue
5      out += ch
6  for ch in "a1b2":
7      if ch.isdigit():
8          pass
9      out += ch
10 print(out)
```

Loop 1 skips digits: "ab". Loop 2 keeps everything: "a1b2".

A `ab`

Forgets the second loop adds to the same out.

B `abab`

Treats pass like continue. pass skips nothing.

C `aba1b2`

continue skips the digits; pass does nothing, so they are kept.

D `a`

Treats continue like break.

THE SIGNAL

continue skips the rest of the pass. pass does nothing at all.

What is the output of this program?

```
1  x = -7
2  print(x / 2, x // 2, int(x / 2), round(x / 2))
```

A -3.5 -4 -3 -4

B -3.5 -3 -3 -4

C -3.5 -4 -4 -4

D -3.5 -4 -3 -3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = -7
2  print(x / 2, x // 2, int(x / 2), round(x / 2))
```

-3.5 exactly. Floor: -4. Truncate: -3. Round half to even: -4.

A `-3.5 -4 -3 -4`

/ exact; // floors down; int cuts to zero; round goes to the even -4.

B `-3.5 -3 -3 -4`

// rounds down, towards minus infinity. -4.

C `-3.5 -4 -4 -4`

int() cuts towards zero: -3.

D `-3.5 -4 -3 -3`

round(-3.5) goes to the even neighbour, -4.

THE SIGNAL

Four ways to turn -3.5 into a whole number, and three different answers.

What is the output of this program?

```
1  words = ["b", "a"]
2  x = words.sort()
3  y = sorted("cab")
4  print(x, y, "cab".find("z"))
```

A `['a', 'b'] ['a', 'b', 'c'] -1`

B `None abc -1`

C `None ['a', 'b', 'c'] -1`

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["b", "a"]
2  x = words.sort()
3  y = sorted("cab")
4  print(x, y, "cab".find("z"))
```

sort() returns None. sorted() always returns a list. find() returns -1.

A `['a', 'b'] ['a', 'b', 'c'] -1`
x holds sort's return value, which is None.

B `None abc -1`
sorted() returns a list, even from a string.

C `None ['a', 'b', 'c'] -1`
Three pairs: sort vs sorted, str vs list, find vs index.

D The code raises a ValueError
find returns -1. Only index would raise.

THE SIGNAL

sort returns None; sorted returns a list; find returns -1.

What is the output of this program?

```
a = (1)
b = (1,)
c = [1]
print(type(a) == int, len(b), c == [1], c is [1])
```

A `True 1 True True`

B `False 1 True False`

C The code raises a TypeError

D `True 1 True False`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = (1)
2  b = (1,)
3  c = [1]
4  print(type(a) == int, len(b), c == [1], c is [1])
```

(1) is the int 1. (1,) is a one-element tuple. [1] makes a new list each time.

A `True 1 True True`
Each [1] builds a new list, so is gives False.

B `False 1 True False`
(1) has no comma, so a is an int.

C The code raises a TypeError
a is an int, but nothing takes its len. b's len is fine.

D `True 1 True False`
No comma: int. Comma: tuple of length 1. Equal lists; different objects.

THE SIGNAL

The comma makes a tuple. == compares values; is compares identity.

> Each line is run as a whole program on its own.

Which two programs print the word None? (Choose two.)

A `print(print("x"))`

B `print({"a": 1}.get("b"))`

C `print([3, 1].sort() == None)`

D `print(len([None]))`

◆ PAUSE AND DECIDE

Which two programs print the word None? (Choose two.)

A `print(print("x"))`

Prints x, then the inner print's return value, None.

B `print({"a": 1}.get("b"))`

A missing key with no default: get returns None.

C `print([3, 1].sort() == None)`

sort does return None, but the comparison prints True.

D `print(len([None]))`

The list has one element, so this prints 1.

THE SIGNAL

Something that returns None only prints None if None itself is what gets printed.

MODULE 24 · EXAM STRATEGY - ALL BLOCKS

Exam mechanics and question formats

The exam's format and rules, every question type and how to approach it, timing, and what to do when Python 3.14 and the exam disagree.

BLOCKS all four FOCUS the exam itself MODULE 24 of 28

admin@sthithapragna.com

Thirty questions, forty minutes, seventy per cent to pass, and **no notes of any kind**. Everything you trace on the day, you trace in your head.

The exam in facts

ITEM	DETAIL
Version	PCEP-30-02, active. PCEP-30-03 is listed as in development.
Questions and time	30 questions in 40 minutes, plus a 5 minute NDA and tutorial
Passing score	70%. Items can carry different points, up to 40 each
Delivery	TestNow, online and proctored, through the Python Institute
Retakes	After a 7 day wait, with no limit on attempts
Validity	5 years. Cost from 69 US dollars

BEFORE YOU BOOK

Before you book, check the version code on your voucher. If PCEP-30-03 is live, check its syllabus against this course.

ALL BLOCKS / WHERE THE MARKS ARE

Four blocks, thirty questions

BLOCK	WEIGHT	QUESTIONS	COVERED IN
1: Fundamentals	18%	7	modules 1 to 5
2: Control flow	29%	8	modules 6 to 10
3: Data collections	25%	7	modules 11 to 15
4: Functions and exceptions	28%	8	modules 16 to 21

ON THE EXAM

Blocks 2 and 4 together are more than half the exam. Both are mostly tracing.

Every format the exam can use

FORMAT	WHAT IT LOOKS LIKE	APPROACH
Single-select	four options, one right	three passes: grammar, run, output
Multi-select	choose two, or choose three	judge each option on its own
Gap fill, code fill	a blank inside the code	try each option in the gap
Code insertion	pick the line that gives a stated output	run each option against the output
Drag and drop, sort	put items or lines in order	fix the first and last positions first
Interactive, scenario	a short task described in words	find the decision, then the rule

Multi-select: every option is its own question

- Decide **true or false for each option separately**, then pick exactly the number asked.
- Do not look for a pattern. The right options rarely share a theme.
- If more options seem true than you are asked for, one of them breaks a rule: find it.
- Count your selections before moving on. Choose two means two.

Try each option in place, grammar first

GAP FILL

What completes the code?

```
1  i = 0
2  ___ i < 3:
3      print(i, end=" ")
4      i += 1
```

Expected: 0 1 2. while works; if prints only 0; for and elif are SyntaxErrors.

CODE INSERTION

Which line gives this output?

```
1  nums = [1, 2, 3]
2  # insert line here
3  print(total)
```

Output 6: sum(nums). max is 3, nums[-1] * 3 is 9, len is 3.

ON THE EXAM

Eliminate on grammar first, it is fastest. Then trace only the options that are left.

Drag and drop, and sort

- Fix the ends first: what must come first, and what must come last.
- For code lines: a def before its call, a variable assigned before it is read, consistent indentation.
- For operator priority, use the real order: ** , unary, * / // %, + -, comparisons, not, and, or.
- Re-read the finished order as code, top to bottom, before moving on.

Rules of the online exam that change how you work

- **No notes of any kind**: no paper, no whiteboard, nothing written.
- **No scheduled breaks.** You stay seated and in view of the camera throughout.
- Full screen, one monitor, no other tabs, no phone or smartwatch in the room.
- A quiet, well-lit room where you are alone. A webcam and microphone are required.
- The score shown at the end is preliminary until the session has been reviewed.

About 80 seconds a question

QUESTION KIND	TYPICAL TIME	HOW TO KEEP IT THERE
Definition or recognition	20 to 40 seconds	answer and move on
Short output trace	60 to 80 seconds	three passes, grammar first
Loop or recursion trace	90 to 120 seconds	shortcuts: stop minus start, multiply or add
Choose two	60 to 90 seconds	one true or false judgement per option

EXAM TECHNIQUE

The quick questions pay for the long ones. If a trace runs long, choose using the method, and keep moving.

Turning this course's habits into mental ones

- Every time this course said write it down, do exactly that while studying.
- Then repeat the same question without paper, saying each value to yourself.
- Track as few things as possible: usually one variable and a count.
- Use shortcuts instead of full traces where a rule exists: ranges, nested loops, the first true branch.
- Take the three practice papers in this course without writing anything down.

When current Python and the exam disagree

WHAT CHANGED	PYTHON 3.14	ANSWER ON THE EXAM
`except A, B:` without brackets	legal (PEP 758)	choose the bracketed form
`return` inside `finally`	SyntaxWarning (PEP 765)	it still works: the warning changes nothing
error message wording	several reworded	answer by the exception's name
soft keywords `match`, `case`, `_`, `type`	legal names	there are still 35 keywords

ON THE EXAM

The exam was written for the classic rules. Where they differ, the classic rule is the safe answer.

Where study material can mislead you

- Logical **not** sits below every comparison, not beside unary minus (module 4).
- Only **one** except branch can run for an exception (module 21).
- For a string, the exam treats s[:] as a copy; CPython may return the same object (module 15).
- Certification lasts 5 years. Some third-party sites wrongly say it never expires.

A short checklist

- Check your voucher's exam version, and the current syllabus, on pythoninstitute.org.
- Run the TestNow equipment check: browser, webcam, microphone, a stable connection.
- Practise in a real interpreter: Python, IDLE, or the free Edube sandbox.
- Optional: the official PCEP-30-02 practice test is sold separately.
- Take this course's three practice papers, paper-free, before you book.

What to carry into the exam

- 30 questions, 40 minutes, 70% to pass. **No notes**, no breaks.
- About 80 seconds a question: quick ones pay for long traces.
- Choose N: judge each option separately, pick exactly N.
- Gaps and insertions: grammar first, then trace what is left.
- Where Python 3.14 and the exam differ, give the classic answer.

Which line, inserted at the comment, makes the program print 6?

```
1  nums = [1, 2, 3]
2  # insert line here
3  print(total)
```

EXPECTED OUTPUT

```
6
```

A `total = sum(nums)`

B `total = max(nums)`

C `total = nums[-1] * 3`

D `total = len(nums)`

◆ PAUSE AND DECIDE

Which line, inserted at the comment, makes the program print 6?

```
1  nums = [1, 2, 3]
2  # insert line here
3  print(total)
```

EXPECTED OUTPUT

```
6
```

A `total = sum(nums)`

1 + 2 + 3 is 6.

B `total = max(nums)`

The largest element is 3.

C `total = nums[-1] * 3`

*The last element is 3, and 3 * 3 is 9.*

D `total = len(nums)`

Three elements: 3.

THE SIGNAL

For code insertion, evaluate every option against the output. Do not stop at the first plausible one.

Which keyword fills the gap so the program prints 0 1 2?

```
1  i = 0
2  ___ i < 3:
3      print(i, end=" ")
4      i += 1
```

A if

B for

C while

D elif

◆ PAUSE AND DECIDE

Which keyword fills the gap so the program prints 0 1 2?

```
1  i = 0
2  ___ i < 3:
3      print(i, end=" ")
4      i += 1
```

A `if`

Legal, but runs the block once: prints 0 only.

B `for`

for needs name in sequence. SyntaxError.

C `while`

Repeats while i < 3: prints 0 1 2.

D `elif`

elif cannot start a statement. SyntaxError.

THE SIGNAL

Grammar removed two options instantly. Tracing decided between the last two.

> The operators are: and, **, ==, not

Which order lists these operators from highest to lowest priority?

A `** not == and`

B `not ** == and`

C `== ** and not`

D `** == not and`

◆ PAUSE AND DECIDE

Which order lists these operators from highest to lowest priority?

A `** not == and`

Puts not above ==. not sits below every comparison.

B `not ** == and`

The bank's mistake: not grouped with unary minus, at the top.

C `== ** and not`

*** is above every comparison, and not is above and.*

D `** == not and`

Power, then comparisons, then not, then and.

THE SIGNAL

Arithmetic, then comparisons, then not, then and, then or.

> These statements are about the PCEP-30-02 online exam session.

Which two statements are true? (Choose two.)

A. You may write working notes on scrap paper during the exam

B. The exam has 30 questions in 40 minutes, plus a short NDA and tutorial

C. The passing score is 70%

D. There is a scheduled break halfway through

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A You may write working notes on scrap paper during the exam

The testing policies prohibit notes of any kind.

B The exam has 30 questions in 40 minutes, plus a short NDA and tutorial

As published for PCEP-30-02.

C The passing score is 70%

As published for PCEP-30-02.

D There is a scheduled break halfway through

The PCEP exam has no scheduled breaks.

THE SIGNAL

No notes, no breaks, 30 questions, 40 minutes, 70%. Now the practice papers.

MODULE 25 · EXAM STRATEGY - ALL BLOCKS

Practice paper 1

Thirty new questions in the exam's proportions: 7 from Block 1, 8 from Block 2, 7 from Block 3, 8 from Block 4.

QUESTIONS 30 TIMING about 80 s each MODULE 25 of 28

admin@sthithapragna.com

Treat it as the real thing

- **No paper.** Trace everything in your head, as you will on the day.
- Commit to an answer before each walkthrough. Pause the video if you need more time.
- Keep a tally of your misses by block. The scoring guide at the end uses it.
- Watching the answers without choosing first teaches almost nothing.

What is the output of this program?

```
1  a = 7.
2  b = "7"
3  c = 7e0
4  print(type(a) == type(c), type(b) == type(a))
```

A True False

B False False

C True True

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 7.
2  b = "7"
3  c = 7e0
4  print(type(a) == type(c), type(b) == type(a))
```

A **True False**
7. and 7e0 are both floats; "7" is a str.

B **False False**
A point or an E makes a float, so a and c match.

C **True True**
Quotes make a str, whatever the digits.

D **False True**
Both comparisons are reversed.

BLOCK 1

A point or an E makes a float. Quotes make a str.

How many stars does this program print?

```
for i in range(1, 10, 4):
    print("*", end="")
```

A Two

B Three

C Four

D Nine

◆ PAUSE AND DECIDE

How many stars does this program print?

```
for i in range(1, 10, 4):
    print("*", end="")
```

A Two

range(1, 10, 4) has three values: 1, 5 and 9.

B Three

1, 5, 9. The next would be 13, past the stop.

C Four

13 is past the stop of 10.

D Nine

The step is 4, not 1.

BLOCK 2

With a step other than 1, list the values; stop before reaching the stop.

What is the output of this program?

```
1  nums = [10, 20, 30, 40, 50]
2  print(nums[-1:1:-2])
```

A [50, 40, 30]

B []

C [50, 30]

D [50, 30, 10]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [10, 20, 30, 40, 50]
2  print(nums[-1:1:-2])
```

A `[50, 40, 30]`
The step is -2, so every second element.

B `[]`
A negative step can count down from -1 to 1.

C **`[50, 30]`**
Indexes 4 and 2. Index 1 is the stop, excluded.

D `[50, 30, 10]`
Index 0 is beyond the stop of 1.

BLOCK 3

Negative step: start included, stop excluded, every step-th element.

What is the output of this program?

```
def grade(n):
    if n >= 50:
        return "pass"
    elif n >= 40:
        return "close"

print(grade(45), grade(30))
```

A `close fail`

B `close None`

C `None None`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grade(n):
2      if n >= 50:
3          return "pass"
4      elif n >= 40:
5          return "close"
6
7  print(grade(45), grade(30))
```

A `close fail`

Nothing in the code produces the word fail.

B `close None`

45 returns close; 30 reaches no return, so None.

C `None None`

45 does reach the elif's return.

D The code raises a TypeError

A missing return is legal: it gives None.

BLOCK 4

Any path through a function without a return gives None.

What is the output of this program?

```
1  print(0o12 + 0b11)
```

A 23

B 15

C 1211

D 13

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(0o12 + 0b11)
```

A 23

Reads 0o12 as decimal 12.

B 15

Reads 0b11 as 5. Binary 11 is 3.

C 1211

Prefixed literals are ints, so + adds them.

D 13

Octal 12 is 10, binary 11 is 3.

BLOCK 1

Convert each prefixed literal with place values, then add normally.

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i):
        count += 1
print(count)
```

A 9

B 6

C 3

D 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 0
2  for i in range(3):
3      for j in range(i):
4          count += 1
5  print(count)
```

A 9

That would need range(3) for the inner loop as well.

B 6

Adds 0 + 1 + 2 + 3: the outer range stops at 2.

C 3

Inner passes: 0, then 1, then 2.

D 0

The inner loop runs when i is 1 and 2.

BLOCK 2

Inner range uses the outer variable: add each pass, do not multiply.

> Each name is considered on its own.

Which two are legal variable names? (Choose two.)

A `pass_`

B `for`

C `None`

D `nOne`

◆ PAUSE AND DECIDE

Which two are legal variable names? (Choose two.)

A `pass_`

A keyword with an underscore is a different name.

B `for`

for is a keyword.

C `None`

None is a keyword, capitalised.

D `nOne`

Case matters: nOne is not None.

BLOCK 1

Check the exact spelling against the 35 keywords, case included.

What is the output of this program?

```
try:
    print(int("abc"))
except (TypeError, IndexError):
    print("A")
except ArithmeticError:
    print("B")
except Exception:
    print("C")
```

A. A

B. B

C. C

D. The program stops with an unhandled ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    print(int("abc"))
except (TypeError, IndexError):
    print("A")
except ArithmeticError:
    print("B")
except Exception:
    print("C")
```

A. A
int("abc") raises ValueError, which is not in the tuple.

B. B
ValueError is not an ArithmeticError.

C. C
ValueError falls through to the Exception branch.

D. The program stops with an unhandled ValueError
Exception catches every everyday error.

BLOCK 4

Name the exception, then take the first branch that matches it or a parent.

What is the output of this program?

```
1  text = "Hello World"
2  print(text.lower().count("o"), text.find("World"))
```

A 2 6

B 1 6

C 2 7

D 2 -1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  text = "Hello World"
2  print(text.lower().count("o"), text.find("World"))
```

A `2 6`

Two o's after lowering; World starts at index 6.

B `1 6`

lower() keeps both o's: hello world.

C `2 7`

Indexes start at 0, so W is at 6.

D `2 -1`

find searches the original text, which contains World.

BLOCK 3

Count from 0. find returns the index of the first match, or -1.

What is the output of this program?

```
n = 1
while n < 50:
    n *= 3
    if n == 27:
        break
else:
    print("done")
print(n)
```

A done, then 27

B 81

C done, then 81

D 27

◆ PAUSE AND DECIDE

What is the output of this program?

```
n = 1
while n < 50:
    n *= 3
    if n == 27:
        break
else:
    print("done")
print(n)
```

A `done, then 27`
break skips the loop's else.

B `81`
break stops the loop at 27.

C `done, then 81`
Ignores the break entirely.

D **27**
n: 3, 9, 27, then break. The else is skipped.

BLOCK 2

A loop's else runs only when no break fired.

What is the output of this program?

```
x = 10
def change():
    global x
    x = x // 3
    return x * 2

y = change()
print(x, y)
```

A 10 6

B 3 6

C 3 3

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 10
2  def change():
3      global x
4      x = x // 3
5      return x * 2
6
7  y = change()
8  print(x, y)
```

A `10 6`

global x makes the function change the outer x.

B `3 6`

x becomes 10 // 3, which is 3; then 3 * 2 is returned.

C `3 3`

*The function returns x * 2.*

D The code raises an UnboundLocalError

global x makes x the global, so it has a value.

BLOCK 4

With global, assigning inside a function changes the outer name.

What is the output of this program?

```
1  print(-9 // 4, -9 % 4)
```

A -2 -1

B -3 -3

C -3 3

D -2 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(-9 // 4, -9 % 4)
```

A `-2 -1`

Truncates towards zero, like C. Python floors.

B `-3 -3`

% takes the sign of the divisor, 4.

C `-3 3`

−2.25 floors to −3; −3 * 4 + 3 is −9.

D `-2 1`

Neither half follows Python's floor rule.

BLOCK 1

// rounds towards minus infinity; % takes the sign of the divisor.

What is the output of this program?

```
1  a = ["x"]
2  a.extend("yz")
3  a.append("yz")
4  a.insert(1, 0)
5  print(len(a), a[1])
```

A 4 0

B 5 y

C 3 0

D 5 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["x"]
2  a.extend("yz")
3  a.append("yz")
4  a.insert(1, 0)
5  print(len(a), a[1])
```

A `4 0`

extend("yz") adds two elements, not one.

B `5 y`

insert(1, 0) puts 0 at index 1.

C `3 0`

append adds one more element, and insert another.

D `5 0`

x, y, z, then "yz", then 0 inserted at index 1.

BLOCK 3

extend adds each item of a sequence, even a string's characters. append adds one.

What is the output of this program?

```
i = 0
total = 0
while i < 6:
    i += 1
    if i % 3 == 0:
        continue
    total += i
print(total)
```

A 21

B 12

C 15

D 9

◆ PAUSE AND DECIDE

What is the output of this program?

```
i = 0
total = 0
while i < 6:
    i += 1
    if i % 3 == 0:
        continue
    total += i
print(total)
```

A 21

Ignores the continue.

B 12

1 + 2 + 4 + 5: the multiples of 3 are skipped.

C 15

Includes 3, or leaves out 6, but not both.

D 9

Skips more than the multiples of 3.

BLOCK 2

continue skips the rest of the pass. Check the update happens before it.

What is the output of this program?

```
def f(n):
    if n < 10:
        return n
    return f(n // 10) + n % 10

print(f(905))
```

A 14

B 509

C 5

D 905

◆ PAUSE AND DECIDE

What is the output of this program?

```
def f(n):
    if n < 10:
        return n
    return f(n // 10) + n % 10

print(f(905))
```

A 14

9 + 0 + 5: the digits of 905, summed.

B 509

Builds the reverse instead of a sum.

C 5

Stops after one step. The base case is reached later.

D 905

Nothing returns n unchanged unless n < 10.

BLOCK 4

Trace calls going down, then add the returns coming back up.

What is the output of this program?

```
1  print(2 + 3 * 2 ** 2 - 4 // 3)
```

A 12

B 21

C 13.0

D 13

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(2 + 3 * 2 ** 2 - 4 // 3)
```

A `12`
4 // 3 is 1, not 2.

B `21`
*Does + before * somewhere. Power, then * and //, then + and -.*

C `13.0`
No / and no float: the result is an int.

D **13**
2 ** 2 is 4; 3 * 4 is 12; 4 // 3 is 1; 2 + 12 - 1.

BLOCK 1

Power, then * / // %, then + -. Then ask whether it is an int or a float.

What is the output of this program?

```
1  t = (1, 2) * 2
2  print(t[1:3], len(t))
```

A (2, 1) 4

B [2, 1] 4

C (2, 1) 2

D (2,) 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (1, 2) * 2
2  print(t[1:3], len(t))
```

A `(2, 1) 4`

(1, 2, 1, 2): slice indexes 1 and 2.

B `[2, 1] 4`

Slicing a tuple gives a tuple.

C `(2, 1) 2`

** 2 repeats the elements: the tuple has 4.*

D `(2,) 4`

The slice stops before index 3, so two elements.

BLOCK 3

A slice has the same type as what you sliced.

What is the output of this program?

```
x = 15
if x % 3 == 0:
    print("Fizz", end="")
if x % 5 == 0:
    print("Buzz", end="")
elif x > 10:
    print("Big", end="")
else:
    print("-", end="")
```

A Fizz

B FizzBuzzBig

C FizzBuzz

D Buzz

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 15
2  if x % 3 == 0:
3      print("Fizz", end="")
4  if x % 5 == 0:
5      print("Buzz", end="")
6  elif x > 10:
7      print("Big", end="")
8  else:
9      print("-", end="")
```

A `Fizz`

The second if is a separate decision.

B `FizzBuzzBig`

Big is in the same chain as Buzz, so it is skipped.

C **`FizzBuzz`**

Two decisions: Fizz from the first, Buzz ends the second chain.

D `Buzz`

15 % 3 == 0, so Fizz prints first.

BLOCK 2

Every if that is not an elif starts a new decision.

What happens when this program runs?

```
1  def area(w, h=2):
2      return w * h
3
4  print(area(3), area(h=4, w=1), area(2, w=5))
```

A 6 4 10 is printed

B 6 4 is printed, then a TypeError

C Nothing is printed; the program raises a TypeError

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def area(w, h=2):
2      return w * h
3
4  print(area(3), area(h=4, w=1), area(2, w=5))
```

A 6 4 10 is printed
area(2, w=5) fills w twice.

B 6 4 is printed, then a TypeError
print evaluates every argument before printing anything.

C Nothing is printed; the program raises a TypeError
The third call fails before print can run.

D Nothing is printed; the program raises a SyntaxError
No positional follows a keyword here, so it compiles.

BLOCK 4

print evaluates all its arguments first. One failing argument means no output from that print.

> d = {"a": 1, "b": 2} has been assigned.

Which two expressions evaluate to True? (Choose two.)

A `1 in d`

B `"b" in d`

C `d.get("c", 3) == 3`

D `len(d.items()) == 4`

◆ PAUSE AND DECIDE

Which two expressions evaluate to True? (Choose two.)

A `1 in d`

in checks keys. 1 is a value.

B `"b" in d`

b is a key.

C `d.get("c", 3) == 3`

c is missing, so get returns the default 3.

D `len(d.items()) == 4`

Two pairs, so 2.

BLOCK 3

in checks keys. get returns its default only when the key is missing.

What is the output of this program?

```
1  print(1, 2, sep="+", end="=")
2  print(3)
```

A `1 + 2 = 3`

B `1+2+=3`

C 1+2=, then 3 on a new line

D `1+2=3`

◆ PAUSE AND DECIDE

What is the output of this program?

```
print(1, 2, sep="+", end="=")
print(3)
```

A `1 + 2 = 3`

sep replaces the space, so there are no spaces.

B `1+2+=3`

sep goes between arguments only, never after the last.

C `1+2=`, then `3` on a new line

end replaced the newline, so 3 follows on the same line.

D **`1+2=3`**

sep between, end at the end, then 3 continues the line.

BLOCK 1

sep only between arguments; end replaces the newline.

What is the output of this program?

```
for i in range(2, 8, 3):
    pass
print(i)
```

A 8

B 7

C 5

D 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(2, 8, 3):
2      pass
3  print(i)
```

A 8

range never produces its stop.

B 7

7 is not a value of this range.

C 5

2, then 5. The loop variable keeps its last value.

D 2

The loop runs twice, so i moves on to 5.

BLOCK 2

After a for loop, the variable holds the last value produced.

What is the output of this program?

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n * n

print(list(evens(5)))
```

A [0, 2, 4]

B [0, 4, 16]

C [4, 16]

D [0, 4, 16, 36]

◆ PAUSE AND DECIDE

What is the output of this program?

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n * n

print(list(evens(5)))
```

A `[0, 2, 4]`
*The generator yields n * n, not n.*

B **`[0, 4, 16]`**
Even n in range(5) are 0, 2, 4; squared.

C `[4, 16]`
*0 is even, and 0 * 0 is 0.*

D `[0, 4, 16, 36]`
range(5) stops at 4.

BLOCK 4

Trace the loop inside a generator; each yield adds one value.

What is the output of this program?

```
1  words = ["apple", "fig", "kiwi"]
2  print([w[0] for w in words if len(w) > 3])
```

A `['a', 'k']`

B `['a', 'f', 'k']`

C `['apple', 'kiwi']`

D `['f']`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["apple", "fig", "kiwi"]
2  print([w[0] for w in words if len(w) > 3])
```

A `['a', 'k']`

apple and kiwi pass the filter; w[0] takes their first letters.

B `['a', 'f', 'k']`

fig has 3 letters, so it is filtered out.

C `['apple', 'kiwi']`

The expression is w[0], not w.

D `['f']`

The test is len(w) > 3, not len(w) == 3.

BLOCK 3

Read a comprehension as: sequence, then filter, then expression.

What is the output of this program?

```
n = 1000
c = 0
while n > 0:
    c += 1
    n //= 10
print(c)
```

A 3

B 4

C 1000

D 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 1000
2  c = 0
3  while n > 0:
4      c += 1
5      n //= 10
6  print(c)
```

A 3

Counts zeros. The loop counts every digit.

B 4

1000, 100, 10, 1: four passes.

C 1000

c counts passes; n is not printed.

D 1

The loop runs until n reaches 0.

BLOCK 2

n //= 10 drops one digit per pass, so counting passes counts digits.

What is the output of this program?

```
def add(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box

a = add(1)
b = add(2)
c = add(3, a)
print(a, b, c is a)
```

A [1] [2] False

B [1, 2, 3] [1, 2] True

C [1, 3] [2] False

D [1, 3] [2] True

◆ PAUSE AND DECIDE

What is the output of this program?

```
def add(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box

a = add(1)
b = add(2)
c = add(3, a)
print(a, b, c is a)
```

A `[1] [2] False`
c = add(3, a) appends to a itself.

B `[1, 2, 3] [1, 2] True`
None as the default makes a fresh list each time.

C `[1, 3] [2] False`
add returns the list it was given, so c is a.

D **`[1, 3] [2] True`**
Fresh lists for a and b; the third call extends a.

BLOCK 4

The None default gives a fresh list per call. A list passed in is changed in place.

What is the output of this program?

```
1  print(12 & 10 | 1, 5 ^ 1, ~4)
```

A 8 4 -5

B 9 6 -4

C 9 4 -4

D 9 4 -5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(12 & 10 | 1, 5 ^ 1, ~4)
```

A `8 4 -5`
| then sets the last bit: 8 | 1 is 9.

B `9 6 -4`
5 ^ 1 clears the last bit: 4. And ~4 is -5.

C `9 4 -4`
~x is -x - 1, so ~4 is -5.

D `9 4 -5`
(12 & 10) | 1 is 9; 101 ^ 001 is 100; -4 - 1.

BLOCK 1

& before ^ before |. And ~x is -x - 1.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b = b + [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [9, 2, 3, 4]`

B `[9, 2, 3] [1, 2, 3, 4]`

C `[9, 2, 3, 4] [9, 2, 3, 4]`

D `[1, 2, 3] [1, 2, 3, 4]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b = b + [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [9, 2, 3, 4]`
b = b + [4] made b a new list before a changed.

B **`[9, 2, 3] [1, 2, 3, 4]`**
+ built a new list for b; a[0] = 9 changes a only.

C `[9, 2, 3, 4] [9, 2, 3, 4]`
+ does not extend the shared list.

D `[1, 2, 3] [1, 2, 3, 4]`
a[0] = 9 changes a.

BLOCK 3

b = b + [...] builds a new list. Only +=, methods and index assignment change a shared one.

What is the output of this program?

```
for n in [4, 6, 9, 10]:
    if n % 3 == 0 and n % 2 == 1:
        print("found", n)
        break
else:
    print("none")
```

A found 9

B found 6

C none

D found 9, then none

◆ PAUSE AND DECIDE

What is the output of this program?

```
for n in [4, 6, 9, 10]:
    if n % 3 == 0 and n % 2 == 1:
        print("found", n)
        break
else:
    print("none")
```

A **found 9**
The first odd multiple of 3 in the list.

B **found 6**
6 is a multiple of 3, but even.

C **none**
9 matches, so break fires and the else is skipped.

D **found 9, then none**
break skips the loop's else.

BLOCK 2

and needs both sides true. A break skips the loop's else.

> Consider Python's built-in exception hierarchy.

Which two exceptions are subclasses of LookupError? (Choose two.)

A `KeyError`

B `ValueError`

C `ZeroDivisionError`

D `IndexError`

◆ PAUSE AND DECIDE

Which two exceptions are subclasses of LookupError? (Choose two.)

A `KeyError`
A missing key is a lookup problem.

B `ValueError`
ValueError sits directly under Exception.

C `ZeroDivisionError`
That is an ArithmeticError.

D `IndexError`
A bad position is a lookup problem.

BLOCK 4

LookupError: KeyError and IndexError. ArithmeticError: ZeroDivisionError, OverflowError.

How did you do?

SCORE OUT OF 30	WHAT IT MEANS	WHAT TO DO NEXT
27 to 30	comfortably above the pass mark	take paper 2 within a few days
21 to 26	at or above 70%, the pass mark	revisit the block where most misses fell, then paper 2
Below 21	below the pass mark	rework the weakest block's modules before paper 2

ON THE EXAM

21 out of 30 is 70%. Aim for a margin above it, not for the line itself.

Find the block, then the modules

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 7, 12, 16, 21, 27	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 25, 29	6 to 10
3: Data collections	3, 9, 13, 17, 20, 24, 28	11 to 15
4: Functions and exceptions	4, 8, 11, 15, 19, 23, 26, 30	16 to 21

WHAT MATTERS MOST

Where your misses cluster matters more than the total. Three misses in one block is a pattern; spread out, it is noise.

MODULE 26 · EXAM STRATEGY - ALL BLOCKS

Practice paper 2

Thirty new questions, none repeating paper 1's topics, with three deliberate pairs that change one detail from a paper 1 question.

QUESTIONS 30 TIMING about 80 s each MODULE 26 of 28

admin@sthithapragna.com

Same rules as paper 1

- **No paper.** Trace in your head.
- Commit to an answer before each walkthrough. Pause if you need more time.
- Tally your misses by block, and compare them with paper 1.
- If paper 1 showed a weak block, check whether it has improved.

The user types 3.5 and presses Enter.

What is the output of this program?

```
1  x = input()
2  print(float(x) * 2, x * 2)
```

A `7.0 7.0`

B `7.0 3.53.5`

C `7 3.53.5`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = input()
2  print(float(x) * 2, x * 2)
```

A `7.0 7.0`
*x is a str; x * 2 repeats it.*

B `7.0 3.53.5`
float(x) * 2 is 7.0; the str x repeated is 3.53.5.

C `7 3.53.5`
float() gives a float, so 7.0.

D The code raises a TypeError
*float * int and str * int are both legal.*

BLOCK 1

input() returns a str. Only the converted copy is a number.

What is the output of this program?

```
1  count = 5
2  while count < 3:
3      count += 1
4  else:
5      count *= 2
6  print(count)
```

A 5

B 6

C 10

D 12

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 5
2  while count < 3:
3      count += 1
4  else:
5      count *= 2
6  print(count)
```

A 5

The loop's else runs even after zero passes.

B 6

5 < 3 is False at the first test: no passes.

C 10

Zero passes, no break, so the else doubles count.

D 12

The body never runs.

BLOCK 2

A loop's else runs whenever no break fired, including after zero passes.

What is the output of this program?

```
a = [1, 2, 3, 4, 5]
a[1:3] = []
del a[-1]
print(a)
```

A [1, 4, 5]

B [1, 2, 3, 4]

C [4, 5]

D [1, 4]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:3] = []
3  del a[-1]
4  print(a)
```

A `[1, 4, 5]`
del a[-1] removes the 5.

B `[1, 2, 3, 4]`
a[1:3] = [] removes 2 and 3.

C `[4, 5]`
The 1 at index 0 is not in the slice.

D `[1, 4]`
Remove indexes 1 and 2, then the last element.

BLOCK 3

Assigning [] to a slice deletes it.

What is the output of this program?

```
1  def add(item, box=[]):
2      box.append(item)
3      return box
4
5  a = add(1)
6  b = add(2)
7  print(a, b, a is b)
```

A [1] [2] False

B [1] [1, 2] False

C [1, 2] [1, 2] True

D [1, 2] [2] True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def add(item, box=[]):
2      box.append(item)
3      return box
4
5  a = add(1)
6  b = add(2)
7  print(a, b, a is b)
```

A `[1] [2] False`
That was paper 1's answer, with a None default.

B `[1] [1, 2] False`
a and b are the same list.

C `[1, 2] [1, 2] True`
One default list, shared by both calls.

D `[1, 2] [2] True`
Both names refer to the one list.

BLOCK 4

Paired with paper 1, question 26: a default list is created once and shared.

What is the output of this program?

```
1  print(0 or "" or "x", 3 > 2 > 1)
```

A True True

B 0 True

C x False

D x True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(0 or "" or "x", 3 > 2 > 1)
```

A `True True`
or returns an operand, not a bool.

B `0 True`
0 is falsy, so or moves on.

C `x False`
3 > 2 > 1 means 3 > 2 and 2 > 1: True.

D `x True`
or returns the first truthy operand; the chain is True.

BLOCK 1

or returns an operand. A chain is an and of neighbouring pairs.

What is the output of this program?

```
score = 85
if score > 50:
    grade = "C"
elif score > 80:
    grade = "B"
else:
    grade = "F"
print(grade)
```

A C

B B

C F

D C, then B

◆ PAUSE AND DECIDE

What is the output of this program?

```
score = 85
if score > 50:
    grade = "C"
elif score > 80:
    grade = "B"
else:
    grade = "F"
print(grade)
```

A C

85 > 50 is checked first and is True.

B B

The more specific test comes second, too late.

C F

else only runs if both tests fail.

D C, then B

Only one branch of a chain runs.

BLOCK 2

The first true branch wins. Put the most demanding test first.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b += [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [1, 2, 3, 4]`

B `[9, 2, 3, 4] [9, 2, 3, 4]`

C `[9, 2, 3] [9, 2, 3, 4]`

D `[1, 2, 3, 4] [9, 2, 3, 4]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b += [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [1, 2, 3, 4]`
That was paper 1's answer, with b = b + [4].

B **`[9, 2, 3, 4] [9, 2, 3, 4]`**
+= extends the shared list; then a[0] = 9 shows in both.

C `[9, 2, 3] [9, 2, 3, 4]`
a and b are still one list.

D `[1, 2, 3, 4] [9, 2, 3, 4]`
a and b are one list, so they print the same.

BLOCK 3

Paired with paper 1, question 28: += changes the shared list; b = b + … builds a new one.

What is the output of this program?

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
```

A ELF

B LF

C E

D EF

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
```

A ELF

else runs only when there was no exception.

B LF

There was an exception, so except runs, not else.

C E

finally always runs.

D EF

Exception: except, then finally.

BLOCK 4

Exception handled: except, then finally. No exception: else, then finally.

What is the output of this program?

```
1  print("cat" < "Cat", "2" > "10")
```

A True False

B True True

C False True

D False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("cat" < "Cat", "2" > "10")
```

A `True False`
Lower case letters come after capitals.

B `True True`
c (99) is after C (67).

C **`False True`**
c is greater than C; "2" beats "1" at the first character.

D `False False`
"2" > "10" compares "2" with "1" first.

BLOCK 1

Strings compare by code point, character by character: digits, capitals, lower case.

What is the output of this program?

```
lines = 0
for i in range(3):
    for j in range(3):
        if j > i:
            break
        lines += 1
print(lines)
```

A 9

B 3

C 1

D 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  lines = 0
2  for i in range(3):
3      for j in range(3):
4          if j > i:
5              break
6          lines += 1
7  print(lines)
```

A 9

break cuts each inner loop short.

B 3

break ends the inner loop only, not the outer.

C 1

The outer loop still makes three passes.

D 6

Inner passes: 1, then 2, then 3.

BLOCK 2

break ends the innermost loop only.

What is the output of this program?

```
1  d = {"x": 1}
2  for k in ["x", "y", "x"]:
3      d[k] = d.get(k, 0) + 1
4  print(d)
```

A `{'x': 2, 'y': 1}`

B `{'x': 1, 'y': 1}`

C `{'x': 3, 'y': 1}`

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {"x": 1}
2  for k in ["x", "y", "x"]:
3      d[k] = d.get(k, 0) + 1
4  print(d)
```

A `{'x': 2, 'y': 1}`
x is counted twice in the loop, plus its starting 1.

B `{'x': 1, 'y': 1}`
Each pass adds 1 to the stored count.

C **`{'x': 3, 'y': 1}`**
x: 1 + 1 + 1. y: get gives 0, plus 1.

D The code raises a KeyError
get supplies 0 for the missing y.

BLOCK 3

get(k, 0) + 1 counts safely, even for new keys.

What is the output of this program?

```
def check(n):
    if n < 0:
        raise ValueError("negative")
    return n ** 0.5

try:
    print(check(-4))
except ValueError as e:
    print("error:", e)
```

A `2.0`

B `error: negative`

C The program stops with an unhandled ValueError

D `error: ValueError`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def check(n):
2      if n < 0:
3          raise ValueError("negative")
4      return n ** 0.5
5
6  try:
7      print(check(-4))
8  except ValueError as e:
9      print("error:", e)
```

A `2.0`
-4 fails the test, so the function raises first.

B `error: negative`
raise ValueError with a message; the handler prints it.

C The program stops with an unhandled ValueError
The except branch matches ValueError.

D `error: ValueError`
e prints the message, not the class name.

BLOCK 4

raise sends the exception to the caller's handler. Printing e shows its message.

What is the output of this program?

```
1  print(round(4.5), round(5.5), int(-4.9))
```

A 4 6 -4

B 5 6 -5

C 4 5 -5

D 5 6 -4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(round(4.5), round(5.5), int(-4.9))
```

A `4 6 -4`

Halves go to the even neighbour; int() cuts towards zero.

B `5 6 -5`

round sends 4.5 to 4, and int(-4.9) is -4.

C `4 5 -5`

5.5 rounds to the even 6.

D `5 6 -4`

4.5 is an exact half. it goes to 4.

BLOCK 1

round: halves to even. int(): towards zero.

What is the output of this program?

```
for i in range(10, 0, -4):
    print(i, end=" ")
```

A 10 6 2 -2

B 10 6 2

C 6 2

D 10 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(10, 0, -4):
2      print(i, end=" ")
```

A `10 6 2 -2`

-2 is past the stop of 0.

B `10 6 2`

Start included; keep stepping by -4 while above 0.

C `6 2`

The start, 10, is always the first value.

D `10 6`

2 is still above the stop.

BLOCK 2

Start is included; stop is never reached, in either direction.

What is the output of this program?

```
1  s = "banana"
2  s.replace("a", "o")
3  t = s.upper()
4  print(s, t[1:4])
```

A banana ANA

B bonono ONO

C bonono ANA

D banana BAN

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "banana"
2  s.replace("a", "o")
3  t = s.upper()
4  print(s, t[1:4])
```

A `banana ANA`

replace's result is thrown away; t is the upper-case copy.

B `bonono ONO`

s.replace on its own line changes nothing.

C `bonono ANA`

s is unchanged, so it still prints banana.

D `banana BAN`

t[1:4] is indexes 1, 2 and 3.

BLOCK 3

A string method's result must be assigned, or it is lost.

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          print(n, end="")
4          show(n - 1)
5          print(n, end="")
6
7  show(2)
```

A 21

B 1221

C 2112

D 2121

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          print(n, end="")
4          show(n - 1)
5          print(n, end="")
6
7  show(2)
```

A 21

The print after the call also runs, on the way back.

B 1221

The first print happens before going deeper.

C 2112

Down: 2, 1. Back up: 1, 2.

D 2121

The second prints run in reverse order, deepest first.

BLOCK 4

Work before the recursive call runs going down; work after it runs coming back up.

What is the output of this program?

```
1  print(1.5e3, 2e-5)
```

A 1500 0.00002

B 1500.0 2e-05

C 1.5e3 2e-5

D 1500.0 0.00002

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(1.5e3, 2e-5)
```

A `1500 0.00002`
An E makes a float, and tiny floats print in E notation.

B `1500.0 2e-05`
1.5e3 is the float 1500.0; 2e-5 prints in E notation.

C `1.5e3 2e-5`
Python prints the value, not the way it was written.

D `1500.0 0.00002`
Below 1e-4, Python switches to E notation.

BLOCK 1

E always means float. Very small and very large floats print in E notation.

What happens when this program runs?

```
1  i = 10
2  while i != 0:
3      i -= 3
4  print(i)
```

A 0 is printed

B -2 is printed

C 1 is printed

D Nothing is printed: the loop never ends

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  i = 10
2  while i != 0:
3      i -= 3
4  print(i)
```

A 0 is printed

i never equals 0.

B -2 is printed

The loop does not stop at -2: -2 != 0.

C 1 is printed

1 != 0, so the loop continues.

D Nothing is printed: the loop never ends

10, 7, 4, 1, -2 … skips 0 for ever.

BLOCK 2

With != as the test, check the stop value can be hit exactly.

What is the output of this program?

```
1  a, b, c = (1, 2, 3)
2  a, c = c, a
3  print(a, b, c)
```

A 1 2 3

B 3 2 3

C 1 2 1

D 3 2 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
a, b, c = (1, 2, 3)
a, c = c, a
print(a, b, c)
```

A `1 2 3`

The second line swaps a and c.

B `3 2 3`

Both values are read before either name changes.

C `1 2 1`

The swap is simultaneous.

D `3 2 1`

Unpack 1, 2, 3, then swap a and c.

BLOCK 3

The right side is packed first, then unpacked: a true swap.

What is the output of this program?

```
x = 5
def f(x):
    x += 1
    return x

print(f(x), x)
```

A 6 5

B 6 6

C 5 5

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  def f(x):
3      x += 1
4      return x
5
6  print(f(x), x)
```

A `6 5`

The parameter x is local; the global x is untouched.

B `6 6`

x += 1 changes only the parameter.

C `5 5`

f returns the incremented local x.

D `The code raises an UnboundLocalError`

x is a parameter, so it has a value from the start.

BLOCK 4

A parameter is a local variable, even when it shares a global's name.

What is the output of this program?

```
1  x = 3
2  x *= x + 1
3  x -= 2
4  print(x)
```

A 7

B 8

C 12

D 10

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 3
2  x *= x + 1
3  x -= 2
4  print(x)
```

A 7

*x *= x + 1 means x = x * (x + 1).*

B 8

*Reads line 2 as x * x + 1, then misses the - 2.*

C 12

Misses the final x -= 2.

D **10**

3 * (3 + 1) is 12, then 12 - 2.

BLOCK 1

x op= expr means x = x op (expr).

What is the output of this program?

```
for i in range(5):
    if i == 2:
        continue
    if i == 4:
        break
    print(i, end="")
```

A 01234

B 0134

C 013

D 01

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(5):
    if i == 2:
        continue
    if i == 4:
        break
    print(i, end="")
```

A `01234`
Ignores both continue and break.

B `0134`
break stops the loop before 4 prints.

C **`013`**
0, 1; 2 skipped by continue; 3; break at 4.

D `01`
continue skips only 2; the loop carries on.

BLOCK 2

continue skips one pass; break ends the loop.

What is the output of this program?

```
1  m = [[1, 2], [3, 4], [5, 6]]
2  print(m[2][0] + m[-2][1], len(m[0]))
```

A 8 2

B 9 2

C 9 3

D The code raises an IndexError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  m = [[1, 2], [3, 4], [5, 6]]
2  print(m[2][0] + m[-2][1], len(m[0]))
```

A `8 2`
m[-2] is [3, 4], so m[-2][1] is 4.

B `9 2`
m[2][0] is 5; m[-2][1] is 4; each row has 2.

C `9 3`
len(m[0]) counts one row's elements: 2.

D `The code raises an IndexError`
m has rows 0, 1 and 2.

BLOCK 3

Row first, then column. len(m) counts rows; len(m[0]) counts one row.

What is the output of this program?

```
def span(a):
    return min(a), max(a)

lo, hi = span([3, 8, 1])
print(hi - lo, type(span([1])))
```

A `7 <class 'list'>`

B `1 <class 'tuple'>`

C `The code raises a TypeError`

D `7 <class 'tuple'>`

◆ PAUSE AND DECIDE

What is the output of this program?

```
def span(a):
    return min(a), max(a)

lo, hi = span([3, 8, 1])
print(hi - lo, type(span([1])))
```

A `7 <class 'list'>`

return a, b packs a tuple, not a list.

B `1 <class 'tuple'>`

hi - lo is 8 - 1.

C `The code raises a TypeError`

A function may return a tuple, and it unpacks into two names.

D `7 <class 'tuple'>`

min 1, max 8; a comma-separated return is one tuple.

BLOCK 4

return a, b returns one tuple, which can be unpacked.

> Each literal is typed on its own.

Which two are valid integer literals? (Choose two.)

A `1_000`

B `0o78`

C `0x1F`

D `1__0`

◆ PAUSE AND DECIDE

Which two are valid integer literals? (Choose two.)

A `1_000`

One underscore between digits is fine.

B `0o78`

8 is not an octal digit. SyntaxError.

C `0x1F`

Hex digits include A to F: 31.

D `1__0`

Two underscores in a row are not allowed.

BLOCK 1

Each base allows only its own digits. Underscores: one at a time, between digits.

What is the output of this program?

```
n = 1
while n < 50:
    n *= 2
    if n == 27:
        break
else:
    print("done")
print(n)
```

A 64

B done, then 64

C 27

D done, then 27

◆ PAUSE AND DECIDE

What is the output of this program?

```
n = 1
while n < 50:
    n *= 2
    if n == 27:
        break
else:
    print("done")
print(n)
```

A 64

No break fired, so the else runs first.

B done, then 64

n doubles past 27 without hitting it: no break, so else runs.

C 27

Doubling from 1 never gives 27.

D done, then 27

n is never 27.

BLOCK 2

Paired with paper 1, question 10: whether break fires decides whether the else runs.

> Each line is run on its own.

Which two lines print None? (Choose two.)

A `print([1].append(2))`

B `print([3, 1].pop())`

C `print(sorted([2, 1]))`

D `print([2, 1].reverse())`

◆ PAUSE AND DECIDE

Which two lines print None? (Choose two.)

A `print([1].append(2))`
append changes the list and returns None.

B `print([3, 1].pop())`
pop returns the removed element: 1.

C `print(sorted([2, 1]))`
sorted returns a new list: [1, 2].

D `print([2, 1].reverse())`
reverse changes the list in place and returns None.

BLOCK 3

Methods that change a list return None. pop is the exception: it returns the element.

What is the output of this program?

```
1  def count(n):
2      while n > 0:
3          yield n
4          n -= 1
5
6  g = count(3)
7  print(next(g), sum(g))
```

A 3 6

B 3 3

C 3 0

D [3] 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
def count(n):
    while n > 0:
        yield n
        n -= 1

g = count(3)
print(next(g), sum(g))
```

A `3 6`

next already took the 3.

B `3 3`

next takes 3; sum adds what is left: 2 + 1.

C `3 0`

The generator still holds 2 and 1.

D `[3] 3`

next gives a value, not a list.

BLOCK 4

A generator remembers its place. Each value can be taken only once.

The user types 4, 9, 3, then -1, pressing Enter after each.

What is the output of this program?

```
1  best = int(input())
2  n = best
3  while n >= 0:
4      if n > best:
5          best = n
6      n = int(input())
7  print(best)
```

A 9

B 4

C -1

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  best = int(input())
2  n = best
3  while n >= 0:
4      if n > best:
5          best = n
6      n = int(input())
7  print(best)
```

A 9

best keeps the largest value seen; -1 stops the loop.

B 4

9 is larger, so best is replaced.

C -1

The sentinel stops the loop and is never compared.

D 3

best keeps the largest, not the last.

BLOCK 2

The sentinel is read and tested, never processed.

> Consider try statements in Python in general.

Which two statements about except branches are true? (Choose two.)

A. A bare except: must be the last branch

B. Two except branches can run for one exception

C. except (A, B): handles either exception

D. finally runs only when no exception occurs

◆ PAUSE AND DECIDE

Which two statements about except branches are true? (Choose two.)

A. A bare except: must be the last branch
Anywhere else, it is a SyntaxError.

B. Two except branches can run for one exception
At most one branch runs.

C. except (A, B): handles either exception
A tuple names several exceptions.

D. finally runs only when no exception occurs
finally always runs.

BLOCK 4

One branch at most; bare except last; finally always.

How did you do?

SCORE OUT OF 30	WHAT IT MEANS	WHAT TO DO NEXT
27 to 30	comfortably above the pass mark	take paper 3
21 to 26	at or above 70%, the pass mark	compare blocks with paper 1, revisit the weakest, then paper 3
Below 21	below the pass mark	rework the weakest block before paper 3

ON THE EXAM

Compare with paper 1. A score that rose after revisiting one block shows the method works.

Find the block, then the modules

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

WHAT MATTERS MOST

Misses that cluster in the same block across two papers are a real weakness. Fix that block first.

MODULE 27 · EXAM STRATEGY - ALL BLOCKS

Practice paper 3

Thirty new questions, none repeating papers 1 or 2, with four deliberate pairs. The scoring guide reads all three papers together.

QUESTIONS 30 TIMING about 80 s each MODULE 27 of 28

admin@sthithapragna.com

Same rules, and keep all three tallies

- **No paper.** Trace in your head.
- Commit to an answer before each walkthrough.
- Tally your misses by block, next to papers 1 and 2.
- Try to keep to about 80 seconds per question, the exam's pace.

What is the output of this program?

```
1  print(True + True * 3, bool("0"), bool(0.0))
```

A 4 True False

B 6 True False

C 4 False False

D 4 True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(True + True * 3, bool("0"), bool(0.0))
```

A `4 True False`

True counts as 1: 1 + 1 * 3 is 4. "0" is non-empty; 0.0 is zero.

B `6 True False`

*Adds first: (1 + 1) * 3. Multiplication comes first, so 1 + 3.*

C `4 False False`

"0" is a non-empty string, so it is truthy.

D `4 True True`

0.0 is a zero, so it is falsy.

BLOCK 1

True is 1 in arithmetic. Only empty strings and zeros are falsy.

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= i
4  print(nums)
```

A [1, 2, 3]

B [0, 1, 2]

C [0, 2, 6]

D [1, 4, 9]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= i
4  print(nums)
```

A `[1, 2, 3]`
*nums[i] *= i changes each slot in the list.*

B `[0, 1, 2]`
Multiplies by i, not replaces with i.

C **`[0, 2, 6]`**
1 * 0, 2 * 1, 3 * 2.

D `[1, 4, 9]`
Each element is multiplied by its index, not itself.

BLOCK 2

Assigning through an index changes the list; assigning to a loop variable does not.

What is the output of this program?

```
1  parts = "a,b,,c".split(",")
2  print(len(parts), "-".join(parts))
```

A 3 a-b-c

B 4 a,b,,c

C 3 a-b--c

D 4 a-b--c

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  parts = "a,b,,c".split(",")
2  print(len(parts), "-".join(parts))
```

A `3 a-b-c`

Two commas in a row leave an empty piece between them.

B `4 a,b,,c`

join uses "-", not a comma.

C `3 a-b--c`

There are 4 pieces, one of them empty.

D `4 a-b--c`

a, b, an empty string, c: joined with -.

BLOCK 3

split(sep) keeps empty pieces; split() with no argument does not.

What is the output of this program?

```
def inner():
    return 1 / 0

def outer():
    try:
        return inner()
    except ZeroDivisionError:
        return "caught"
    finally:
        print("F", end=" ")

print(outer())
```

A caught

B F, then a ZeroDivisionError

C F caught

D caught F

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def inner():
2      return 1 / 0
3
4  def outer():
5      try:
6          return inner()
7      except ZeroDivisionError:
8          return "caught"
9      finally:
10         print("F", end=" ")
11
12 print(outer())
```

A `caught`
finally prints F before the return completes.

B `F, then a ZeroDivisionError`
outer's handler catches what inner raised.

C **`F caught`**
inner's error is caught in outer; finally prints, then caught is returned.

D `caught F`
finally runs before outer actually returns.

BLOCK 4

Errors travel up to the caller's handler. finally runs before the return completes.

> A developer notices a __pycache__ folder containing a file whose name ends in .pyc.

What does a .pyc file in the __pycache__ folder contain?

A The original source code, compressed

B Machine code for the processor

C A copy of the Python interpreter

D Compiled bytecode for an imported module

◆ PAUSE AND DECIDE

What does a .pyc file in the __pycache__ folder contain?

A The original source code, compressed

The source stays in the .py file.

B Machine code for the processor

Bytecode targets the Python virtual machine, not the processor.

C A copy of the Python interpreter

The interpreter is still needed to run it.

D Compiled bytecode for an imported module

Cached so later imports can skip compiling.

BLOCK 1

A .pyc file holds bytecode for an imported module.

What is the output of this program?

```
a, b = 0, 10
while a < b and b > 5:
    a += 2
    b -= 1
print(a, b)
```

A 6 7

B 10 5

C 4 8

D 8 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a, b = 0, 10
2  while a < b and b > 5:
3      a += 2
4      b -= 1
5  print(a, b)
```

A 6 7

6 < 7 and 7 > 5 is still True: one more pass.

B 10 5

The loop stops once a < b fails.

C 4 8

Two passes are not enough to fail the test.

D 8 6

Passes give (2, 9), (4, 8), (6, 7), (8, 6); then 8 < 6 fails.

BLOCK 2

With and in a loop test, the loop ends as soon as either side is false.

What is the output of this program?

```
d = {1: "a", True: "b"}
d[1.0] = "c"
print(len(d), d[True])
```

A 1 c

B 2 b

C 3 c

D 1 a

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {1: "a", True: "b"}
2  d[1.0] = "c"
3  print(len(d), d[True])
```

A `1 c`

1, True and 1.0 are one key; the last value wins.

B `2 b`

True == 1, so they are the same key.

C `3 c`

Equal keys share one entry.

D `1 a`

Each assignment replaces the value.

BLOCK 3

Equal keys collide: one entry, and the last value assigned wins.

What happens when this program runs?

```
1  count = 0
2  def show():
3      print(count)
4      count = 1
5
6  show()
```

A 0 is printed

B 1 is printed

C Nothing is printed; the program raises a SyntaxError

D The program raises an UnboundLocalError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  count = 0
2  def show():
3      print(count)
4      count = 1
5
6  show()
```

A 0 is printed

count is assigned in show, so it is local throughout show.

B 1 is printed

The print comes before the assignment.

C Nothing is printed; the program raises a SyntaxError

It compiles. It fails when show runs.

D The program raises an UnboundLocalError

count = 1 makes count local, and print reads it first.

BLOCK 4

Paired with paper 2, question 20: assigning anywhere in a function makes the name local everywhere in it.

What happens when this program runs?

```
1  print("start")
2  message = "unfinished
3  print(message)
```

A start, then unfinished

B Nothing is printed; the program raises a SyntaxError

C start, then a SyntaxError

D start, then a NameError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("start")
2  message = "unfinished
3  print(message)
```

A start, then unfinished

The second string is never closed.

B Nothing is printed; the program raises a SyntaxError

An unclosed string is a grammar fault, so line 1 never runs.

C start, then a SyntaxError

Syntax errors stop the whole file, including line 1.

D start, then a NameError

The fault is in the grammar, found before running.

BLOCK 1

Any syntax error, anywhere: nothing runs.

What is the output of this program?

```
x = 4
if x > 2:
    if x > 5:
        print("A")
    else:
        print("B")
print("C")
```

A. A, then C

B. B, then C

C. Only C

D. C, then B

◆ PAUSE AND DECIDE

What is the output of this program?

```
x = 4
if x > 2:
    if x > 5:
        print("A")
    else:
        print("B")
print("C")
```

A. A, then C
4 > 5 is False.

B. B, then C
The else pairs with the inner if, by indentation.

C. Only C
That would need the else to line up with the outer if.

D. C, then B
C prints after the whole if statement.

BLOCK 2

An else belongs to the if it lines up with.

What is the output of this program?

```
1  a = [3, 1, 2]
2  b = sorted(a)
3  a.reverse()
4  print(a, b)
```

A [1, 2, 3] [1, 2, 3]

B [3, 2, 1] [1, 2, 3]

C [2, 1, 3] [1, 2, 3]

D [2, 1, 3] [3, 2, 1]

◆ PAUSE AND DECIDE

What is the output of this program?

```
a = [3, 1, 2]
b = sorted(a)
a.reverse()
print(a, b)
```

A `[1, 2, 3] [1, 2, 3]`
reverse() does not sort; and sorted made a separate list.

B `[3, 2, 1] [1, 2, 3]`
reverse() flips the current order. 3, 1, 2 becomes 2, 1, 3.

C `[2, 1, 3] [1, 2, 3]`
sorted copied before the reverse; reverse flips 3, 1, 2.

D `[2, 1, 3] [3, 2, 1]`
b was sorted before a changed, and is a separate list.

BLOCK 3

sorted() makes a new list. reverse() flips the current order; it does not sort.

What happens when this program runs?

```
1  def power(b, e):
2      if e == 0:
3          return 1
4      return b * power(b, e - 2)
5
6  print(power(2, 5))
```

A The program raises a RecursionError

B 32 is printed

C 8 is printed

D 1 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def power(b, e):
2      if e == 0:
3          return 1
4      return b * power(b, e - 2)
5
6  print(power(2, 5))
```

A The program raises a RecursionError

e goes 5, 3, 1, -1 ... and never equals 0.

B 32 is printed

That needs e - 1, not e - 2.

C 8 is printed

The recursion does not stop at e = 1.

D 1 is printed

The base case is never reached.

BLOCK 4

Check the steps can land exactly on the base case.

What is the output of this program?

```
1  print(str(2.50), int("0042"), float("  1e2 "))
```

A 2.50 42 100.0

B 2.5 0042 100.0

C 2.5 42 100.0

D The code raises a ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(str(2.50), int("0042"), float("  1e2 "))
```

A `2.50 42 100.0`
str() of a float prints its shortest form: 2.5.

B `2.5 0042 100.0`
int() of a str gives a number, so leading zeros vanish.

C `2.5 42 100.0`
str(2.50) is "2.5"; int drops leading zeros; float strips spaces and reads E.

D `The code raises a ValueError`
Each conversion is valid.

BLOCK 1

Spaces around a number string are ignored. Leading zeros in a str are fine for int().

What is the output of this program?

```
count = 0
for c in "Programming":
    if c in "aeiou":
        count += 1
print(count)
```

A 3

B 4

C 2

D 11

◆ PAUSE AND DECIDE

What is the output of this program?

```
count = 0
for c in "Programming":
    if c in "aeiou":
        count += 1
print(count)
```

A 3

o, a and i are the lower-case vowels in Programming.

B 4

There are only three vowels in Programming.

C 2

Counts only o and a; i counts too.

D 11

Only characters in "aeiou" are counted.

BLOCK 2

in on a string checks for a substring, here a single character.

What is the output of this program?

```
1  t = ([1], 2)
2  t[0].append(3)
3  print(t, len(t[0]))
```

A ([1], 2) 1

B ([1, 3], 2) 2

C The code raises a TypeError

D ([1, 3], 2) 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = ([1], 2)
2  t[0].append(3)
3  print(t, len(t[0]))
```

A `([1], 2) 1`
append changes the list inside the tuple.

B `([1, 3], 2) 2`
The tuple holds the same list, which grew.

C `The code raises a TypeError`
No tuple slot is replaced; only the list changes.

D `([1, 3], 2) 3`
len(t[0]) is the length of the inner list: 2.

BLOCK 3

A tuple fixes which objects it holds, not what happens inside them.

What is the output of this program?

```
1  def f(a, b=2, c=3):
2      return a + b * c
3
4  print(f(1, c=10), f(c=1, a=2, b=5))
```

A 10 25

B 21 7

C 21 10

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f(a, b=2, c=3):
2      return a + b * c
3
4  print(f(1, c=10), f(c=1, a=2, b=5))
```

A `10 25`
b keeps its default, 2, in the first call.

B `21 7`
1 + 2 * 10, then 2 + 5 * 1.

C `21 10`
Keywords match names: a = 2, b = 5, c = 1.

D `The code raises a TypeError`
Every parameter gets exactly one value.

BLOCK 4

Keywords fill by name, in any order. Unfilled parameters use their defaults.

What is the output of this program?

```
1  print(not 1 + 1 == 2, not (1 + 1) == 3)
```

A True False

B False False

C True True

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(not 1 + 1 == 2, not (1 + 1) == 3)
```

A `True False`
not applies last, after the comparison.

B `False False`
1 + 1 == 3 is False, so not gives True.

C `True True`
1 + 1 == 2 is True, so not gives False.

D **`False True`**
not (2 == 2) is False; not (2 == 3) is True.

BLOCK 1

not sits below arithmetic and every comparison.

What is the output of this program?

```
n = 0
while True:
    n += 1
    if n * n > 20:
        break
print(n)
```

A 4

B 20

C 5

D The loop never ends

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 0
2  while True:
3      n += 1
4      if n * n > 20:
5          break
6  print(n)
```

A `4`
*4 * 4 is 16, not more than 20.*

B `20`
n counts up, not to 20.

C `5`
5 * 5 is 25, the first square over 20.

D `The loop never ends`
break fires at n = 5.

BLOCK 2

while True needs a break that is certain to fire.

What is the output of this program?

```
s = "Python"
print(s[-2] + s[1], chr(ord(s[0]) + 1))
```

A oy P

B ny Q

C hy Q

D oy Q

◆ PAUSE AND DECIDE

What is the output of this program?

```
s = "Python"
print(s[-2] + s[1], chr(ord(s[0]) + 1))
```

A `oy P`
chr(ord("P") + 1) is the next code point: Q.

B `ny Q`
s[-2] is the second last character, o.

C `hy Q`
s[-2] is o, not h.

D `oy Q`
s[-2] is o, s[1] is y; P shifted by one is Q.

BLOCK 3

-1 is the last character. chr(ord(c) + n) moves n characters along.

What is the output of this program?

```
def first_long(words):
    for w in words:
        if len(w) > 3:
            return w

print(first_long(["hi", "hello", "world"]), first_long(["a"]))
```

A hello world

B hello -1

C hello None

D hi None

◆ PAUSE AND DECIDE

What is the output of this program?

```
def first_long(words):
    for w in words:
        if len(w) > 3:
            return w

print(first_long(["hi", "hello", "world"]), first_long(["a"]))
```

A `hello world`
return ends the function at the first match.

B `hello -1`
Nothing returns -1.

C **`hello None`**
First long word is returned; no match falls off the end.

D `hi None`
hi has only 2 letters.

BLOCK 4

return ends the function at once. No return reached: None.

What is the output of this program?

```
1  print(hex(255) + bin(3))
```

A `0xff0b11`

B `258`

C `0xff 0b11`

D `The code raises a TypeError`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(hex(255) + bin(3))
```

A `0xff0b11`
hex and bin return strings, so + joins them.

B `258`
The results are strings, not numbers.

C `0xff 0b11`
+ joins with no space.

D `The code raises a TypeError`
str + str is legal.

BLOCK 1

bin(), oct() and hex() return strings.

What happens when this program runs?

```
1  for i in range(3.0):
2      print(i)
```

A 0, 1 and 2 are printed

B 0.0, 1.0 and 2.0 are printed

C Nothing is printed; the program raises a SyntaxError

D The program raises a TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  for i in range(3.0):
2      print(i)
```

A 0, 1 and 2 are printed
range() does not accept a float.

B 0.0, 1.0 and 2.0 are printed
range only produces ints, and only accepts them.

C Nothing is printed; the program raises a SyntaxError
The grammar is fine. It fails when range is called.

D The program raises a TypeError
range(3.0): 'float' object cannot be interpreted as an integer.

BLOCK 2

range() takes ints only. A float, even 3.0, is a TypeError.

What is the output of this program?

```
a = ["x"]
a.extend(["yz"])
a += "ab"
print(len(a), a[-1])
```

A 4 b

B 6 b

C 3 ab

D 5 b

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["x"]
2  a.extend(["yz"])
3  a += "ab"
4  print(len(a), a[-1])
```

A `4 b`

extend(["yz"]) adds one element; += "ab" adds two characters.

B `6 b`

extend of a list adds that list's single element, "yz".

C `3 ab`

+= with a string adds each character separately.

D `5 b`

"yz" goes in whole: x, yz, a, b.

BLOCK 3

Paired with paper 1, question 13: extend adds each item of what it is given.

What is the output of this program?

```
try:
    print({"a": 1}["b"])
except IndexError:
    print("I")
except LookupError:
    print("L")
except KeyError:
    print("K")
```

A I

B K

C I, then L

D L

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    print({"a": 1}["b"])
except IndexError:
    print("I")
except LookupError:
    print("L")
except KeyError:
    print("K")
```

A `I`
A missing key is a KeyError, not an IndexError.

B `K`
LookupError matches first, as KeyError's parent.

C `I, then L`
Only one except branch runs.

D **L**
The KeyError matches its parent, LookupError, first.

BLOCK 4

The first branch that matches the exception or a parent wins.

> Each line is run on its own.

Which two lines print True? (Choose two.)

A `print(0.1 + 0.2 == 0.3)`

B `print(round(0.1 + 0.2, 2) == 0.3)`

C `print(1 == 1.0)`

D `print("1" == 1)`

◆ PAUSE AND DECIDE

Which two lines print True? (Choose two.)

A `print(0.1 + 0.2 == 0.3)`

The sum is 0.30000000000000004.

B `print(round(0.1 + 0.2, 2) == 0.3)`

Rounding removes the tiny error.

C `print(1 == 1.0)`

int and float compare by value.

D `print("1" == 1)`

A str never equals an int.

BLOCK 1

Never compare floats exactly. Numbers compare by value; str never equals int.

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i, 3):
        count += 1
print(count)
```

A 6

B 3

C 9

D 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i, 3):
        count += 1
print(count)
```

A 6

Inner passes: 3, then 2, then 1.

B 3

That was paper 1's range(i). Here it is range(i, 3).

C 9

The inner range shrinks, so do not multiply.

D 4

Adds the passes incorrectly.

BLOCK 2

Paired with paper 1, question 6: when the inner range uses i, count each pass and add.

> d = {"a": 1, "b": 2} has been assigned. Each line is run on its own.

Which two lines remove the key "a" from d? (Choose two.)

A `del d["a"]`

B `d.pop("a")`

C `d.remove("a")`

D `d.discard("a")`

◆ PAUSE AND DECIDE

Which two lines remove the key "a" from d? (Choose two.)

A `del d["a"]`

del removes the pair.

B `d.pop("a")`

pop removes it and returns 1.

C `d.remove("a")`

Dictionaries have no remove method: AttributeError.

D `d.discard("a")`

Dictionaries have no discard method: AttributeError.

BLOCK 3

Dictionaries: del d[k], d.pop(k), d.popitem(), d.clear(). No remove().

What is the output of this program?

```
def g():
    yield 1
    return 2
    yield 3

print(list(g()))
```

A [1]

B [1, 2]

C [1, 3]

D [1, 2, 3]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def g():
2      yield 1
3      return 2
4      yield 3
5
6  print(list(g()))
```

A **[1]**

return ends the generator after the first yield.

B `[1, 2]`

A return value is not yielded.

C `[1, 3]`

Nothing after the return runs.

D `[1, 2, 3]`

return stops the generator.

BLOCK 4

return inside a generator simply ends it.

What is the output of this program?

```
x = 15
if x % 3 == 0:
    print("Fizz", end="")
elif x % 5 == 0:
    print("Buzz", end="")
else:
    print("-", end="")
```

A FizzBuzz

B Fizz

C Buzz

D -

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 15
2  if x % 3 == 0:
3      print("Fizz", end="")
4  elif x % 5 == 0:
5      print("Buzz", end="")
6  else:
7      print("-", end="")
```

A `FizzBuzz`

That was paper 1's answer, with two separate ifs.

B `Fizz`

One chain: the first true branch wins, and Buzz is skipped.

C `Buzz`

The Fizz test comes first and is true.

D `-`

else runs only if both tests fail.

BLOCK 2

Paired with paper 1, question 18: elif joins a chain; a new if starts a new decision.

> Consider Python functions in general.

Which two statements are true? (Choose two.)

A. A parameter's default is evaluated each time the function is called

B. A function can read a global variable without the global keyword

C. Calling a function that contains yield returns a generator object

D. return can appear only once in a function

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A A parameter's default is evaluated each time the function is called

It is evaluated once, when def runs.

B A function can read a global variable without the global keyword

Only assigning needs global.

C Calling a function that contains yield returns a generator object

The body does not run until a value is requested.

D return can appear only once in a function

It can appear many times.

BLOCK 4

Defaults: once. Globals: read freely. yield: a generator. return: as often as needed.

Paper 3, by block

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

The trend matters more than any single score

ACROSS PAPERS 1 TO 3	WHAT IT MEANS	WHAT TO DO
27 or more on all three	ready	book the exam; take paper 4 a few days before
rising, now 24 or more	the method is working	revisit any block still weak, then paper 4
one block weak in two or more papers	a real gap, not bad luck	rework that block's modules, then module 22
below 21 on paper 3	not ready yet	rework the weakest block, then repeat papers 1 to 3

WHAT MATTERS MOST

Where the misses cluster across papers matters more than the totals. Fix the block that keeps appearing.

MODULE 28 · EXAM STRATEGY - ALL BLOCKS

Practice paper 4: final rehearsal

Thirty new questions under exam conditions: no paper, about 80 seconds each, in the exam's proportions.

QUESTIONS 30 TIMING about 80 s each MODULE 28 of 28

admin@sthithapragna.com

Exam conditions

- **No paper, no notes.** Exactly as on the day.
- About **80 seconds** per question. Pause only to keep to that pace.
- Commit to an answer before each walkthrough, even when unsure.
- Score it, then compare with papers 1 to 3.

What is the output of this program?

```
1  print(-3 ** 2, (-3) ** 2, 2 ** -1)
```

A 9 9 0.5

B -9 9 0.5

C -9 9 -2

D -9 -9 0.5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(-3 ** 2, (-3) ** 2, 2 ** -1)
```

A `9 9 0.5`
*** binds tighter than the minus on its left: -(3 ** 2).*

B `-9 9 0.5`
-(3 ** 2); then (-3) squared; then 1 / 2 as a float.

C `-9 9 -2`
*2 ** -1 is one half, not -2.*

D `-9 -9 0.5`
The brackets make (-3) the base, so it is squared to 9.

BLOCK 1

A minus on the left of ** is applied after the power.

What is the output of this program?

```
for ch in "hello":
    if ch == "l":
        break
    print(ch, end="")
```

A he

B hel

C heo

D hello

◆ PAUSE AND DECIDE

What is the output of this program?

```
for ch in "hello":
    if ch == "l":
        break
    print(ch, end="")
```

A he

h and e print; the first l triggers break.

B hel

break comes before the print, so l is not printed.

C heo

break ends the loop; it does not skip to the next character.

D hello

The loop stops at the first l.

BLOCK 2

break ends the whole loop; continue would skip one pass.

What is the output of this program?

```
1  data = [1, [1, 2], 2]
2  print(data.count(1), 2 in data, len(data[1]))
```

A 2 True 2

B 1 True 2

C 1 False 2

D 2 True 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [1, [1, 2], 2]
2  print(data.count(1), 2 in data, len(data[1]))
```

A `2 True 2`

count looks only at top-level elements: the 1 inside the list does not count.

B `1 True 2`

One top-level 1; 2 is a top-level element; the inner list has 2.

C `1 False 2`

The last element is 2, so 2 in data is True.

D `2 True 3`

len(data[1]) is the inner list's length.

BLOCK 3

count, in and len look at top-level elements only.

What is the output of this program?

```
def f(x=None):
    return x is None

print(f(), f(0))
```

A True False

B True True

C False False

D None 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f(x=None):
2      return x is None
3
4  print(f(), f(0))
```

A `True False`

f() uses the default None; 0 is not None.

B `True True`

0 is falsy, but it is not None.

C `False False`

With no argument, x is None, so is gives True.

D `None 0`

The function returns the result of the is test.

BLOCK 4

is None tests for None itself, not for any falsy value.

What is the output of this program?

```
1  s = "a#b"  # a comment
2  print(len(s))
```

A 1

B The code raises a SyntaxError

C 5

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "a#b"  # a comment
2  print(len(s))
```

A `1`

A # inside quotes is just a character.

B `The code raises a SyntaxError`

Comments may follow code on the same line.

C `5`

The comment after the code is not part of the string.

D `3`

a, # and b; the real comment is ignored.

BLOCK 1

inside quotes is text. # outside quotes starts a comment.

What is the output of this program?

```
n = 7
while n > 0:
    n -= 2
else:
    print(n)
```

A 1

B -1

C Nothing is printed

D 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
n = 7
while n > 0:
    n -= 2
else:
    print(n)
```

A `1`

The loop runs once more: 1 > 0, so n becomes -1.

B `-1`

n: 5, 3, 1, -1. Then the test fails and the else prints.

C `Nothing is printed`

No break, so the loop's else runs.

D `0`

Odd minus 2 stays odd: n never equals 0.

BLOCK 2

The else sees the value that failed the test.

What is the output of this program?

```
1  text = "  Hi There "
2  print(text.strip().split()[1].upper())
```

A THERE

B HI

C There

D The code raises an IndexError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  text = "  Hi There "
2  print(text.strip().split()[1].upper())
```

A THERE

strip, split on spaces, take index 1, then upper.

B HI

Index 1 is the second word.

C There

upper() capitalises every letter.

D The code raises an IndexError

split gives two words, so index 1 exists.

BLOCK 3

Chained methods apply left to right.

What is the output of this program?

```
def total(nums):
    if not nums:
        return 0
    return nums[0] + total(nums[1:])

print(total([2, 4, 6]))
```

A 2

B 246

C 12

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def total(nums):
2      if not nums:
3          return 0
4      return nums[0] + total(nums[1:])
5
6  print(total([2, 4, 6]))
```

A 2

The recursion adds every element.

B 246

The values are numbers, so + adds them.

C 12

2 + (4 + (6 + 0)).

D `The code raises a RecursionError`

Each call has a shorter list, reaching the empty list.

BLOCK 4

Each recursive call must move closer to the base case.

What is the output of this program?

```
1  x, y = 1, 2
2  x, y = y, x + y
3  print(x, y)
```

A 2 3

B 2 4

C 1 3

D 3 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
x, y = 1, 2
x, y = y, x + y
print(x, y)
```

A 2 3

The right side uses the old x and y: 2, then 1 + 2.

B 2 4

x + y uses the old x, 1, not the new one.

C 1 3

x is given the old y, 2.

D 3 2

The pair is assigned in order: x first, then y.

BLOCK 1

The right side is evaluated completely before any name changes.

What is the output of this program?

```
for i in range(1, 4):
    if i == 2:
        continue
    for j in range(i):
        print(i, end="")
```

A 122333

B 13

C 1223

D 1333

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(1, 4):
    if i == 2:
        continue
    for j in range(i):
        print(i, end="")
```

A 122333

continue skips the whole of the i = 2 pass.

B 13

The inner loop prints i, i times.

C 1223

When i is 3, the inner loop prints 3 three times.

D **1333**

i = 1 prints 1; i = 2 is skipped; i = 3 prints 333.

BLOCK 2

continue in the outer loop skips the inner loop for that pass.

What is the output of this program?

```
1  prices = {"b": 2, "a": 1}
2  prices["c"] = 3
3  del prices["b"]
4  print(list(prices), sum(prices.values()))
```

A ['b', 'a', 'c'] 6

B ['a', 'b', 'c'] 4

C [1, 3] 4

D ['a', 'c'] 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  prices = {"b": 2, "a": 1}
2  prices["c"] = 3
3  del prices["b"]
4  print(list(prices), sum(prices.values()))
```

A `['b', 'a', 'c'] 6`
del removed b.

B `['a', 'b', 'c'] 4`
b was deleted, and keys are not sorted.

C `[1, 3] 4`
list(prices) gives the keys.

D `['a', 'c'] 4`
b is deleted; keys stay in insertion order; 1 + 3.

BLOCK 3

list(d) gives keys, in insertion order.

What is the output of this program?

```
def safe_div(a, b):
    try:
        r = a / b
    except ZeroDivisionError:
        return "inf"
    else:
        return round(r, 1)

print(safe_div(1, 3), safe_div(1, 0))
```

A 0.3 0.0

B 0.3 inf

C 0.33 inf

D The code raises a ZeroDivisionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def safe_div(a, b):
2      try:
3          r = a / b
4      except ZeroDivisionError:
5          return "inf"
6      else:
7          return round(r, 1)
8
9  print(safe_div(1, 3), safe_div(1, 0))
```

A `0.3 0.0`
The except branch returns the string inf.

B `0.3 inf`
1 / 3 rounds to 0.3 in else; 1 / 0 is handled.

C `0.33 inf`
round(r, 1) keeps one decimal place.

D `The code raises a ZeroDivisionError`
It is caught inside safe_div.

BLOCK 4

else runs only when the try block succeeded.

What is the output of this program?

```
1  print(int(7.9) + int("7") * 2)
```

A 22

B The code raises a ValueError

C 21

D 28

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(int(7.9) + int("7") * 2)
```

A 22
int(7.9) cuts to 7; it does not round to 8.

B The code raises a ValueError
int("7") is a valid conversion.

C 21
7 + 7 * 2: int() truncates, then * before +.

D 28
** comes before +: 7 + 14.*

BLOCK 1

int() of a float truncates. Then precedence as usual.

What is the output of this program?

```
items = []
if items:
    print("full")
elif items == []:
    print("empty")
else:
    print("?")
```

A `full`

B `?`

C `empty`

D Nothing is printed

◆ PAUSE AND DECIDE

What is the output of this program?

```
items = []
if items:
    print("full")
elif items == []:
    print("empty")
else:
    print("?")
```

A `full`
An empty list is falsy.

B ?
The elif test is True.

C **empty**
if [] is False; [] == [] is True.

D Nothing is printed
One branch of the chain matches.

BLOCK 2

Empty collections are falsy.

What is the output of this program?

```
1  print([x * y for x in range(1, 3) for y in range(2)])
```

A [0, 1, 2]

B [0, 2]

C [0, 1, 0, 2]

D [1, 2, 2, 4]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print([x * y for x in range(1, 3) for y in range(2)])
```

A `[0, 1, 2]`
Two loops: every x is paired with every y.

B `[0, 2]`
y takes both 0 and 1 for each x.

C **`[0, 1, 0, 2]`**
x = 1: 0, 1. x = 2: 0, 2.

D `[1, 2, 2, 4]`
range(2) gives y = 0 and 1, not 1 and 2.

BLOCK 3

Two for clauses in a comprehension work like nested loops.

What happens when this program runs?

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print(undefined)
print("end")
```

A end is printed

B The ZeroDivisionError is handled, and end is printed

C Nothing is printed; the program raises a SyntaxError

D The program stops with a NameError; end is not printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError:
4      print(undefined)
5  print("end")
```

A end is printed
The except branch itself fails.

B The ZeroDivisionError is handled, and end is printed
Handling starts, but the branch raises a new error.

C Nothing is printed; the program raises a SyntaxError
An undefined name is a runtime error.

D The program stops with a NameError; end is not printed
The error inside except is new, and nothing handles it.

BLOCK 4

An exception raised inside an except branch is a new, unhandled problem.

What is the output of this program?

```
1  print("ab" * 2 < "abc", len("ab" * 0))
```

A False 0

B True 0

C True 2

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("ab" * 2 < "abc", len("ab" * 0))
```

A `False 0`
Compare character by character: a=a, b=b, then a < c.

B `True 0`
"abab" < "abc" at the third character; "ab" * 0 is empty.

C `True 2`
Repeating zero times gives an empty string.

D `The code raises a TypeError`
*str * int is repetition.*

BLOCK 1

Strings compare at the first differing character, not by length.

What happens when this program runs?

```
1  for k in range(0):
2      pass
3  print(k)
```

A -1 is printed

B The program raises a NameError

C 0 is printed

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  for k in range(0):
2      pass
3  print(k)
```

A -1 is printed

range(0) produces nothing.

B The program raises a NameError

Zero passes: k is never assigned.

C 0 is printed

range(0) has no values, not a 0.

D Nothing is printed; the program raises a SyntaxError

The grammar is fine.

BLOCK 2

Paired with paper 1, question 22: zero passes means the loop variable never exists.

What is the output of this program?

```
1  t = (5)
2  u = (5,)
3  print(t * 2, u * 2)
```

A (5, 5) (5, 5)

B 10 (5, 5)

C 10 10

D (10,) (5, 5)

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (5)
2  u = (5,)
3  print(t * 2, u * 2)
```

A `(5, 5) (5, 5)`
(5) has no comma, so t is the int 5.

B **`10 (5, 5)`**
t is 5, so t * 2 is 10; u is a tuple, repeated.

C `10 10`
*(5,) is a tuple, so * repeats it.*

D `(10,) (5, 5)`
t is not a tuple at all.

BLOCK 3

The comma makes a tuple.

What is the output of this program?

```
1  hits = 0
2  def hit():
3      global hits
4      hits += 1
5      return hits
6
7  for _ in range(3):
8      hit()
9  print(hits, hit())
```

A 3 4

B 3 3

C 0 1

D 4 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  hits = 0
2  def hit():
3      global hits
4      hits += 1
5      return hits
6
7  for _ in range(3):
8      hit()
9  print(hits, hit())
```

A `3 4`

Three calls make hits 3; the fourth call, in print, makes it 4.

B `3 3`

hit() inside print runs again.

C `0 1`

global hits makes the function change the outer counter.

D `4 4`

print evaluates hits before calling hit().

BLOCK 4

Arguments are evaluated left to right.

What is the output of this program?

```
1  print(7.5 // 2, 7.5 % 2)
```

A 3 1.5

B 3.75 1.5

C 3.0 1

D 3.0 1.5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(7.5 // 2, 7.5 % 2)
```

A `3 1.5`
A float operand makes // return a float: 3.0.

B `3.75 1.5`
// floors; it does not divide exactly.

C `3.0 1`
% on a float gives a float: 1.5.

D **`3.0 1.5`**
7.5 // 2 is 3.0; 3.0 * 2 + 1.5 is 7.5.

BLOCK 1

// and % on floats give floats.

What is the output of this program?

```
1  for i in range(3):
2      print("*" * (i + 1), end=" ")
```

A `* ** ***`

B `*** ** *`

C `** *** ****`

D `* * *`

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(3):
    print("*" * (i + 1), end=" ")
```

A `* ** ***`

Pass i prints i + 1 stars, then a space.

B `*** ** *`

The count grows with i.

C `** *** ****`

range(3) starts at 0, so the first line has 1 star.

D `* * *`

"" * (i + 1) repeats the star.*

BLOCK 2

range(3) starts at 0. String * int repeats.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a[::-1]
3  b[0] = 9
4  print(a, b)
```

A [9, 2, 3] [9, 2, 1]

B [1, 2, 3] [9, 2, 3]

C [3, 2, 1] [9, 2, 1]

D [1, 2, 3] [9, 2, 1]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a[::-1]
3  b[0] = 9
4  print(a, b)
```

A `[9, 2, 3] [9, 2, 1]`
A slice is a copy; changing b cannot reach a.

B `[1, 2, 3] [9, 2, 3]`
[::-1] reverses: b starts as [3, 2, 1].

C `[3, 2, 1] [9, 2, 1]`
Slicing never changes the original list.

D **`[1, 2, 3] [9, 2, 1]`**
b is a reversed copy; a is untouched.

BLOCK 3

Paired with paper 2, question 7: a slice makes a copy; += keeps one shared list.

What is the output of this program?

```
def nums():
    n = 0
    while True:
        yield n
        n += 1

for x in nums():
    if x > 2:
        break
    print(x, end="")
```

A 0123

B 012

C The loop never ends

D 123

◆ PAUSE AND DECIDE

What is the output of this program?

```
def nums():
    n = 0
    while True:
        yield n
        n += 1

for x in nums():
    if x > 2:
        break
    print(x, end="")
```

A `0123`

The break happens before 3 is printed.

B `012`

The generator is infinite; break stops the loop at 3.

C `The loop never ends`

break ends the loop once x is 3.

D `123`

n starts at 0, and 0 is yielded first.

BLOCK 4

A generator produces values only when asked, so an infinite one is safe with break.

> Each expression is evaluated on its own.

Which two expressions raise a TypeError? (Choose two.)

A `"3" * "3"`

B `"3" * 3`

C `len(42)`

D `int("3")`

◆ PAUSE AND DECIDE

Which two expressions raise a TypeError? (Choose two.)

A `"3" * "3"`

You cannot multiply a str by a str.

B `"3" * 3`

*str * int repeats: "333".*

C `len(42)`

An int has no length.

D `int("3")`

A valid conversion: 3.

BLOCK 1

Wrong type for the operation: TypeError.

What is the output of this program?

```
x = 1
while x < 100:
    x = x * 2 + 1
print(x)
```

A 63

B 100

C 64

D 127

◆ PAUSE AND DECIDE

What is the output of this program?

```
x = 1
while x < 100:
    x = x * 2 + 1
print(x)
```

A 63

63 < 100, so the loop runs once more.

B 100

x never equals 100; it jumps from 63 to 127.

C 64

*The update is x * 2 + 1, not x * 2.*

D 127

1, 3, 7, 15, 31, 63, 127: the first value not below 100.

BLOCK 2

A while loop leaves with the first value that fails the test.

> Each line is run on its own.

Which two lines print True? (Choose two.)

A `print("Abc".isupper())`

B `print("abc".isalpha())`

C `print("123".isdigit())`

D `print("a b".isalnum())`

◆ PAUSE AND DECIDE

Which two lines print True? (Choose two.)

A `print("Abc".isupper())`
b and c are lower case.

B `print("abc".isalpha())`
Letters only.

C `print("123".isdigit())`
Digits only.

D `print("a b".isalnum())`
The space is not a letter or a digit.

BLOCK 3

The is methods test every character.

What is the output of this program?

```
1  def add(d, k):
2      d[k] = d.get(k, 0) + 1
3
4  inventory = {}
5  add(inventory, "a")
6  add(inventory, "a")
7  print(inventory)
```

A {}

B {'a': 1}

C The code raises a KeyError

D {'a': 2}

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def add(d, k):
2      d[k] = d.get(k, 0) + 1
3
4  inventory = {}
5  add(inventory, "a")
6  add(inventory, "a")
7  print(inventory)
```

A `{}`
The function changes the dictionary it is given.

B `{'a': 1}`
The second call adds one more.

C `The code raises a KeyError`
get supplies 0 the first time.

D `{'a': 2}`
The same dictionary is updated twice through d.

BLOCK 4

A function can change a dictionary or list it is given.

What is the output of this program?

```
for n in range(2, 5):
    if 7 % n == 0:
        break
else:
    print("prime")
```

A Nothing is printed

B prime, three times

C prime

D The code raises a ZeroDivisionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
for n in range(2, 5):
    if 7 % n == 0:
        break
else:
    print("prime")
```

A `Nothing is printed`
No divisor is found, so no break fires: the else runs.

B `prime, three times`
The else runs once, after the loop.

C `prime`
7 % 2, 7 % 3 and 7 % 4 are all non-zero: no break.

D `The code raises a ZeroDivisionError`
range(2, 5) never gives 0.

BLOCK 2

A search with for-else: the else means nothing was found.

> Consider Python exceptions in general.

Which two statements are true? (Choose two.)

A except ArithmeticError: catches a ZeroDivisionError

B IndexError is a subclass of KeyError

C An exception not handled in a function passes to its caller

D assert raises a ValueError when its condition is false

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A except ArithmeticError: catches a ZeroDivisionError

ZeroDivisionError is a kind of ArithmeticError.

B IndexError is a subclass of KeyError

They are siblings under LookupError.

C An exception not handled in a function passes to its caller

It travels up until something handles it.

D assert raises a ValueError when its condition is false

assert raises AssertionError.

BLOCK 4

Parents catch children. Unhandled exceptions travel to the caller.

Paper 4, by block

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

Reading the final rehearsal

PAPER 4 SCORE	WHAT IT MEANS	WHAT TO DO
27 or more	ready, with a margin	sit the exam while it is fresh
24 to 26	ready, with a small margin	revisit the block where misses fell; then sit it
21 to 23	at the pass line, with no margin	rework your weakest block and modules 22 and 23 first
Below 21	not ready yet	rework the weakest block, then retake papers 1 to 4

BEFORE YOU BOOK

Aim to sit the exam with a margin above 70%, not on the line. Retakes are allowed after 7 days, but passing first time is the goal.